Student Privacy Protection (NER) - A-Z
零基础实战教程

从环境配置到训练、测试、Web、验收与面试的可执行教材

目录

Phase 00：项目边界、最终成果与学习方法 7

1. 本阶段学习目标 . . . 7
2. 本阶段需要掌握的知识 . . . 7
3. 从零解释知识点 . . . 7
4. 本阶段开始前的项目状态 . . . 7
5. 本阶段需要使用的工具 . . . 8
    6. Lesson 00.1：把需求改写成输入与输出 . . . 8
    7. Lesson 00.2：建立证据层级 . . . 8
    8. Lesson 00.3：识别禁止的捷径 . . . 8
    9. Lesson 00.4：定义完成条件 . . . 8
10. 完整代码回顾 . . . 9
11. 跨文件调用关系 . . . 9
12. 数据流解释 . . . 9
13. 常见错误 . . . 9
14. Checkpoint . . . 9
15. 学生练习 . . . 9
16. 面试准备 . . . 9
17. 下一阶段预告 . . . 9

本阶段完整可复制文件 . . . 9

Phase 01：Windows 11、Python、VS Code 与虚拟环境 13

1. 本阶段学习目标 . . . 13
2. 本阶段需要掌握的知识 . . . 13
3. 从零解释知识点 . . . 13
4. 本阶段开始前的项目状态 . . . 14
5. 本阶段需要使用的工具 . . . 14
    6. Lesson 01.1：打开 PowerShell并检查 Python . . . 14
    7. Lesson 01.2：安装并打开 VS Code . . . 14
    8. Lesson 01.3：创建、激活并选择 Virtual Environment . . . 15
    9. Lesson 01.4：安装 uv、PyTorch 与依赖 . . . 15
    9. Lesson 01.5：创建并运行第一个 Python 文件 . . . 16
    9. Lesson 01.6：退出与重新进入项目 . . . 16
10. 完整代码回顾 . . . 16
11. 跨文件调用关系 . . . 17
12. 数据流解释 . . . 17
13. 常见错误 . . . 17
14. Checkpoint . . . 17

15. 学生练习 . . . 17
16. 面试准备 . . . 17
17. 下一阶段预告 . . . 17
本阶段完整可复制文件 . . . 17
Phase 02：为机器学习补齐 Python 基础 21
1. 本阶段学习目标 . . . 21
2. 本阶段需要掌握的知识 . . . 21
3. 从零解释知识点 . . . 21
4. 本阶段开始前的项目状态 . . . 22
5. 本阶段需要使用的工具 . . . 22
6. Lesson 02.1：list、loop 与 dict . . . 22
7. Lesson 02.2：把逻辑变成 function . . . 22
8. Lesson 02.3：class、object、self 与 `__init__` . . . 22
9. Lesson 02.4：inheritance 与 `super()` . . . 23
9. Lesson 02.5：dataclass 与半开区间 . . . 23
9. Lesson 02.6：method、union type 与 main guard . . . 23
10. 完整代码回顾 . . . 24
11. 跨文件调用关系 . . . 24
12. 数据流解释 . . . 24
13. 常见错误 . . . 24
14. Checkpoint . . . 24
15. 学生练习 . . . 24
16. 面试准备 . . . 24
17. 下一阶段预告 . . . 25
本阶段完整可复制文件 . . . 25
Phase 03：从零认识 NLP、NER 与 BIO 26
1. 本阶段学习目标 . . . 26
2. 本阶段需要掌握的知识 . . . 26
3. 从零解释知识点 . . . 26
4. 本阶段开始前的项目状态 . . . 26
5. 本阶段需要使用的工具 . . . 26
6. Lesson 03.1：手工建立 token/label 对齐 . . . 27
7. Lesson 03.2：实现 `validate_bio` . . . 27
8. Lesson 03.3：把 BIO 合并为 entity . . . 27
9. Lesson 03.4：实际运行与检查输出 . . . 27
10. 完整代码回顾 . . . 28
11. 跨文件调用关系 . . . 28
12. 数据流解释 . . . 28

13. 常见错误 . . . 28
14. Checkpoint . . . 28
15. 学生练习 . . . 28
16. 面试准备 . . . 28
17. 下一阶段预告 . . . 29
本阶段完整可复制文件 . . . 29
Phase 04：创建项目骨架、配置与 Git 30
1. 本阶段学习目标 . . . 30
2. 本阶段需要掌握的知识 . . . 30
3. 从零解释知识点 . . . 30
4. 本阶段开始前的项目状态 . . . 30
5. 本阶段需要使用的工具 . . . 30
6. Lesson 04.1：创建 package 与目录 . . . 31
7. Lesson 04.2：逐行创建 constants . . . 31
8. Lesson 04.3：建立 `.gitignore` 与依赖入口 . . . 31
9. Lesson 04.4：从不同目录验证路径 . . . 31
10. 完整代码回顾 . . . 31
11. 跨文件调用关系 . . . 32
12. 数据流解释 . . . 32
13. 常见错误 . . . 32
14. Checkpoint . . . 32
15. 学生练习 . . . 32
16. 面试准备 . . . 32
17. 下一阶段预告 . . . 32
本阶段完整可复制文件 . . . 32
Phase 05：建立并验证一般与 Malaysian 数据集 35
1. 本阶段学习目标 . . . 35
2. 本阶段需要掌握的知识 . . . 35
3. 从零解释知识点 . . . 35
4. 本阶段开始前的项目状态 . . . 35
5. 本阶段需要使用的工具 . . . 35
6. Lesson 05.1：设计 sample 与 `_annotate` . . . 35
7. Lesson 05.2：独立 entity pools 与 deterministic random . . . 35
8. Lesson 05.3：Malaysian 独立 split . . . 35
9. Lesson 05.4：写 JSONL 与 leakage fail-fast . . . 36
10. 完整代码回顾 . . . 36
11. 跨文件调用关系 . . . 36
12. 数据流解释 . . . 36

13. 常见错误 . . . 36
14. Checkpoint . . . 36
15. 学生练习 . . . 36
16. 面试准备 . . . 36
17. 下一阶段预告 . . . 37
本阶段完整可复制文件 . . . 37
Phase 06：Tokenization、BIO 校验与 Vocabulary 42
本阶段完整可复制文件 . . . 43
Phase 07：Tensor、Token ID、特征与 Padding 47
本阶段完整可复制文件 . . . 47
Phase 08：PyTorch Dataset、DataLoader 与 Batch 52
本阶段完整可复制文件 . . . 52
Phase 09：Neural Network、Embedding、LSTM 与完整 BiLSTM 56
1. 本阶段学习目标 . . . 56
2. 本阶段需要掌握的知识 . . . 56
3. 从零解释知识点 . . . 56
4. 本阶段开始前的项目状态 . . . 56
5. 本阶段需要使用的工具 . . . 56
6. Lesson 1：什么是 Neural Network? . . . 57
7. Lesson 2：什么是 Layer? . . . 57
8. Lesson 3：什么是 Parameter? . . . 57
9. Lesson 4：什么是 PyTorch Module? . . . 57
10. Lesson 5：Python Class、Inheritance、self 与 super . . . 57
11. Lesson 6：Version 1 - 创建第一个模型类 . . . 57
12. Lesson 7：理解 Embedding . . . 58
13. Lesson 8：Version 2 - 添加 Embedding . . . 58
14. Lesson 9：理解 RNN 与 LSTM . . . 58
15. Lesson 10：理解 Bidirectional LSTM . . . 58
16. Lesson 11：Version 3 - 添加 BiLSTM . . . 58
17. Lesson 12：理解 Linear Layer . . . 58
18. Lesson 13：Version 4 - 添加 Linear . . . 59
19. Lesson 14：理解 Dropout . . . 59
20. Lesson 15：Version 5 - 实现 forward() . . . 59
21. Lesson 16：完整 Tensor Shape Tracking . . . 59
22. Lesson 17：Version 6 - 连接真实 casing features . . . 59
23. Lesson 18：连接真实 Dataset 并完整 Forward . . . 60
24. Lesson 19：常见错误与 Debugging . . . 60
25. Lesson 20：阶段验收与参数数量 . . . 60

10. 完整代码回顾 . . . 60
11. 跨文件调用关系 . . . 60
12. 数据流解释 . . . 60
13. 常见错误 . . . 61
14. Checkpoint . . . 61
15. 学生练习 . . . 61
16. 面试准备 . . . 61
17. 下一阶段预告 . . . 61
补充 A：`BiLSTMNER.__init__` 完整函数教学卡 . . . 61
补充 B：`forward` 完整函数教学卡 . . . 62
补充 C：一次完整数值 Shape 演算 . . . 62
补充 D：LSTM 参数为什么比想象多 . . . 62
补充 E：Version 1 到 Version 6 的逐行变更清单 . . . 63
补充 F：五个可复现 Debug Lab . . . 63
补充 G：为什么当前选择 BiLSTM Token Classifier 而不是 CRF . . . 63
补充 H：本阶段掌握证明 . . . 63
补充 I：逐行阅读最终 forward 的口述练习 . . . 64
补充 J：代码审查清单 . . . 64
本阶段完整可复制文件 . . . 64
Phase 10：Loss、Gradient、Backpropagation 与 Training Step 70
本阶段完整可复制文件 . . . 71
Phase 11：Epoch、Validation、Checkpoint 与可复现训练 74
本阶段完整可复制文件 . . . 74
Phase 12：Checkpoint Loading、Raw Text Prediction 与 BIO Decoding 79
本阶段完整可复制文件 . . . 80
Phase 13：字符 Span 与可靠 Text Redaction 82
本阶段完整可复制文件 . . . 83
Phase 14：Malaysian 学生数据合入训练与独立测试 86
本阶段完整可复制文件 . . . 86
Phase 15：Entity-level Evaluation、失败案例与限制 94
本阶段完整可复制文件 . . . 94
Phase 16：FastAPI Text API 与真实 HTTP 验收 97
本阶段完整可复制文件 . . . 98
Phase 17：TXT/PDF 文件处理、安全限制与错误响应 101
本阶段完整可复制文件 . . . 101
Phase 18：Web Interface 与端到端数据流 104
本阶段完整可复制文件 . . . 105

Phase 19：自动化测试、Windows 调试与错误修复 110
本阶段完整可复制文件 110
Phase 20：README、验收报告、GitHub 发布准备与文档同步 121
本阶段完整可复制文件 122
Phase 21：最终复现、30 天计划、英文演示与面试 132
本阶段完整可复制文件 133

Phase 00：项目边界、最终成果与学习方法

1. 本阶段学习目标

这一阶段不训练模型。你要先知道“将要造什么、什么算完成、什么绝对不能宣传”。完成后，你应能用自己的话说明：输入是一段英文字符串；输出是 PERSON、LOCATION、ORGANISATION 三类实体、字符位置与脱敏文本；核心预测必须来自本仓库训练的模型，而不是现成 NER API。

为什么先做边界？因为如果一开始把“所有个人资料”误解成 PER/LOC/ORG，后面即使代码全部运行，也会做出错误产品。CoNLL 风格的实体标签不包含电话、邮箱、护照或学号。本项目是教学/比赛原型，不是医院可直接部署的生产系统。

2. 本阶段需要掌握的知识

- Project requirement：必须实现的功能约束。 - Acceptance criterion：用什么证据判断功能完成。 - Software correctness 与 ML performance 的区别。 - Repository、source code、checkpoint、dataset、report 的含义。 - 为什么“不编造指标”本身是工程要求。

3. 从零解释知识点

Source code 是人编写的程序文本，例如 `src/model.py`。Dataset 是带输入与答案的数据；训练集用来更新参数，验证集用来选模型，测试集只用于最终考试。Checkpoint 是训练后保存的参数和推理所需配置。Metric 是量化表现的数字，但数字必须注明在哪个数据集上得到。

Software correctness 回答“程序是否按规格运行”，例如空输入是否返回错误、PDF 是否能提取文字。ML performance 回答“预测是否正确”，例如 Person recall。`pytest` 全通过不代表模型不会漏人名；反过来，F1 很高也不代表文件上传安全。

本项目的真实边界：Embedding + casing features + BiLSTM + Linear，从随机权重训练；没有现成 NER pipeline。它支持文本、TXT、文字层 PDF、网页和 redaction。扫描 PDF 没有 OCR。合成模板测试 F1 很高，但手动挑战句确实出现漏检，因此必须人工复核。

4. 本阶段开始前的项目状态

学生开始时只需要一个空文件夹。正式完成版本会逐步出现：

```
Intern-MachineLearning/
  learning/              # 前置练习
  src/                   # 模型与数据代码
  app/                   # FastAPI 和网页
  tests/                 # 自动化测试
  data/                  # 数据集
  models/                # checkpoint
  reports/               # 训练与评估证据
  tutorial/              # 本教材源文件与快照
```

本阶段不要求这些目录已经存在。下一阶段才实际创建第一个文件。

5. 本阶段需要使用的工具

先准备纸笔或文本编辑器即可。若你已经打开 VS Code：点击左上角 Explorer 图标，观察当前文件夹；不要随意运行不理解的命令。本阶段命令只用于读取，不会训练或修改模型。

6. Lesson 00.1：把需求改写成输入与输出

输入示例：

```
Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru.
```

目标实体是 Sarah/PER、Microsoft/ORG、Kuala Lumpur/LOC。脱敏目标是：

```
[PER] works at [ORG] in [LOC].
```

这里的方括号不是模型输入，而是应用层根据模型预测的字符范围生成的结果。模型先预测 BIO 标签，redaction 后处理才替换文字。

在纸上完成练习：给句子 `Ali studies at Universiti Malaya.` 标出实体。不要用关键词表；你只是先理解预期输出。

实际验收：最终模型对这类合成分布样例可返回实体，但我们不会因为一个样例正确就宣称模型性能优秀。

7. Lesson 00.2：建立证据层级

打开项目根目录的 `MODEL_ACCEPTANCE_REPORT.md`。先找 Software Tests，再找 Machine Learning Evaluation，最后找 Manual Demonstrations。三个区域不可合并。

在项目根目录 PowerShell 执行：

```
Get-Content MODEL_ACCEPTANCE_REPORT.md -TotalCount 40
```

`Get-Content` 读取文本；`-TotalCount 40` 只显示前 40 行。它不会修改文件。实际项目输出开头包含执行日期、平台和 requirement checklist。

若提示找不到文件：先执行 `Get-Location`，确认 Working Directory 是项目根目录。若你从空目录学习，本文件要到 Phase 20 才创建，因此本练习标记为“教师演示，不适用于学生空目录”，不是错误。

8. Lesson 00.3：识别禁止的捷径

以下做法不合格：调用 OpenAI/Gemini/Google Cloud 识别实体；直接加载 spaCy 或 Hugging Face 已训练 NER pipeline；查看 test label 后反复修改模型；只展示伪造运行结果。

允许使用 PyTorch、NumPy、FastAPI、pypdf 等辅助库，因为分类权重仍由自己的训练产生。正则表达式可以负责 tokenization，但不能硬编码 `Microsoft -> ORG` 作为核心预测。

运行只读检查：

```
rg -n "spacy|pipeline\(|openai|gemini" src app
```

`rg` 是快速文本搜索工具；`-n` 显示行号。最终验收代码没有这些 NER 调用，因此正常结果可能为空。命令找不到 `rg` 时可使用 VS Code 的 Ctrl+Shift+F 搜索。

9. Lesson 00.4：定义完成条件

本教程只有同时具备以下证据才完成：数据校验、真实训练日志、冻结测试指标、checkpoint 重载、字符 span、redaction、Malaysian 独立测试、TXT/PDF、真实 HTTP、自动化测试、README、验收报告、教学快照与 PDF 全页检查。

公开 GitHub 需要仓库所有者授权。若没有 URL，只能写“发布准备完成，尚未发布”，不能写 PASS。

10. 完整代码回顾

本阶段没有业务代码，这是唯一合理的不适用项：目标是建立需求与证据边界。可执行内容是只读检查命令。没有创建模型文件，因此不存在可展示的 Python 完整代码。下一阶段会创建第一个 Python 文件。

11. 跨文件调用关系

当前没有运行时调用。未来数据流是：`app/main.py` 接收文本 -> `src/predict.py` 分词并加载 checkpoint -> `src/model.py` 计算标签 -> `src/predict.py` 合并实体 -> `src/redact.py` 替换字符区间 -> API 返回 JSON。

验收证据流是：`reports/*.json` + `pytest` + 手测结果 -> `MODEL_ACCEPTANCE_REPORT.md` -> 教程 Phase 20/21。

12. 数据流解释

原始输入类型是 Python `str`。Tokenizer 产生 `list[str]` 和字符区间。Vocabulary 把 token 转成 `list[int]`。Dataset 把它变成 `torch.Tensor`。模型输出 logits tensor。Argmax 产生标签 ID，再映射到 BIO 字符串。实体抽取器产生 `list[dict]`，最后 redaction 返回新的 `str`。

这一阶段只认识流程名称，不要求掌握 Tensor；Phase 07 会从数字和 shape 开始教学。

13. 常见错误

1. 把 NER 当成所有 PII：修复方法是列出不支持的电话、邮箱、ID、地址。 2. 把合成 test F1=1.0 当生产表现：查看手动失败案例。 3. 把 PDF 提取器当 NER：pypdf 只提供文本，分类仍由模型完成。 4. 把测试通过当公开 GitHub：发布是独立外部动作。 5. 一开始复制最终代码：你会失去增量学习和错误定位能力。

14. Checkpoint

口头回答并记录：三种实体是什么？核心预测由谁完成？Software correctness 与 ML performance 有何不同？为什么扫描 PDF 不支持？为什么高风险场景仍需人工？五题全部能回答才通过。

教师验收命令（最终仓库）：

```
uv run python -m pytest -q
```

本轮最近一次真实结果是 `22 passed`。这只是已有最终工程的基线，不是学生 Phase 00 应自行得到的输出。

15. 学生练习

1. 写三个句子，各包含 PER、LOC、ORG。 2. 列出五种本模型无法检测的敏感标识。 3. 用两句话说明为什么不能用 test set 选最佳 epoch。 4. 解释“文件可以读取”和“实体预测正确”为何是两种证据。

16. 面试准备

简单英语："I built an educational NER prototype that detects people, locations and organisations. The core predictions come from a model trained in this repository. Software tests and machine-learning metrics are reported separately. The system is not a production PII solution and still requires human review."

17. 下一阶段预告

Phase 01 会从真正的 Windows 11 操作开始：检查 Python、安装 VS Code、创建 `.venv`、选择解释器、安装依赖并运行 `learning/hello_python.py`。完成后，你才拥有可靠的开发环境。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：README.md

为什么需要它：公开仓库入口。学生需要逐段填写项目介绍、安装、数据、训练、运行、测试、架构、指标与限制，而不是写一句占位说明。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 md 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-35 行

```
0001 | # Student Privacy Protection — Personal Information Detection and Redaction
0002 |
0003 | ## Project Overview
0004 |
0005 | 这是一个面向英文学生文本的命名实体识别（NER）项目，服务于 Student Incident Reports、Support Chats、Complaints、Feedback、Un
.... | >iversity Administrative Reports 与 Student Activity Reports。模型从随机权重开始训练，由 PyTorch 自定义 `Embedd
.... | >ing + casing features + BiLSTM + Linear` 实现，识别 `PERSON (PER)`、`LOCATION (LOC)`、`ORGANISATION
.... | > (ORG)`，返回字符位置并生成脱敏文本。
0006 |
0007 | 项目不调用外部 AI/NER API，也不加载 spaCy/Hugging Face 现成 NER pipeline。仓库直接包含经过训练的 `models/best_model.pt
.... | >`，Clone 后不需要重新训练即可预测。
0008 |
0009 | ## Competition Requirements
0010 |
0011 | | Requirement | Status |
0012 | |---|---|
0013 | | 自行定义并训练的 Machine Learning Model | PASS |
0014 | | PER / LOC / ORG | PASS |
0015 | | 原始英文文本输入及类型/位置输出 | PASS |
0016 | | Public GitHub repository | 本仓库 |
0017 | | Bonus: Text Redaction | PASS |
0018 | | Bonus: Malaysian local synthetic dataset | PASS |
0019 | | Bonus: TXT/PDF input | PASS |
0020 |
0021 | ## Features
0022 |
0023 | - CPU 可运行的自定义 BiLSTM token classifier。
0024 | - 原始英文文本预测、BIO tags、实体类型、token span、字符 `[start,end)`。
0025 | - `[PER]`、`[LOC]`、`[ORG]` 自动脱敏。
0026 | - Malaysian 学生姓名、地点、大学、学校、学生组织与教育机构合成标注数据。
0027 | - FastAPI JSON API 和响应式单页 Web 界面（Input Text、Redact Text、只读结果、Clear）。
0028 | - UTF-8 `.txt` 与带文本层 `.pdf` 上传；5 MB 限制和错误响应。
0029 | - 56 项自动化测试（包含阶段专用 Tensor、DataLoader、training step、text API、file API 与 Web UI 检查）、13 项学生场景固定
.... | >挑战矩阵及独立 general/Malaysian test evaluation。
0030 |
0031 | ## Model Architecture
0032 |
0033 | `token IDs -> trainable Embedding(64)` 与 `4 casing features -> Linear(8)` 拼接，进入一层双向 LSTM（每方向
.... | > 96），经过 Dropout 0.25，最后 Linear 输出 8 个 BIO 标签 logits。参数总数 143,536。
0034 |
0035 | 没有使用 CRF：采用比赛允许的 BiLSTM Token Classifier 备选，以保证 CPU 和初学者环境可复现。代价是模型没有联合学习 BIO 转移；抽取器会安全修复非法
.... | >始 I-tag。
```

源码第 36-77 行

```
0036 |
0037 | ## Dataset
0038 |
0039 | 本项目使用可重现的学生场景合成英文 BIO 数据，不声称是 CoNLL-2003、真实学生记录或真实用户数据：
0040 |
0041 | - General: 2,400 train / 300 validation / 300 test。
0042 | - Malaysian: 160 train / 40 validation / 60 test。
0043 | - 标签：`O, B/I-PER, B/I-LOC, B/I-ORG`。
0044 | - 语境覆盖学生报告、投诉、反馈、支持聊天、大学行政报告与学生活动。
0045 | - 完整句子与 general/Malaysian 合并实体池跨 split 隔离；句法模板仍共享，因此测试指标高估真实泛化。
0046 |
0047 | 数据由 `src/generate_data.py` 生成，可公开重新分发；仓库不包含受限制的第三方数据集。
```

0048 |
0049 | ## Installation - Windows 11 / PowerShell
0050 |
0051 | 推荐使用 **Visual Studio Code**，并安装 Microsoft 发布的 Python 扩展。运行项目还需要：
0052 |
0053 | - 64-bit Python 3.11 或 3.12（安装时勾选 `Add Python to PATH`）
0054 | - Git for Windows
0055 | - PowerShell（Windows 11 已包含）
0056 | - Chrome、Edge 或 Firefox 浏览器
0057 |
0058 | 不需要 Node.js、npm、React、Vue、数据库、NVIDIA GPU 或重新训练模型。以下命令已经按本项目结构验证：
0059 |
0060 | ```powershell
0061 | git clone https://github.com/sleeplessLh/NER-Personal-Information-Detection.git
0062 | cd NER-Personal-Information-Detection
0063 | python -m venv .venv
0064 | Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
0065 | .\.venv\Scripts\Activate.ps1
0066 | python -m pip install --upgrade pip
0067 | python -m pip install -r requirements.txt
0068 | ```
0069 |
0070 | ## Quick Start
0071 |
0072 | 不需要重新训练。仓库中的 checkpoint 会在第一次请求时自动加载到 CPU：
0073 |
0074 | ```powershell
0075 | $env:OMP_NUM_THREADS="1"
0076 | $env:MKL_NUM_THREADS="1"
0077 | python -m uvicorn app.main:app --reload

源码第 78-114 行

0078 | ```
0079 |
0080 | 打开 <http://127.0.0.1:8000>。停止服务按 `Ctrl+C`。
0081 |
0082 | ## Web Interface
0083 |
0084 | 网页只保留完成任务所需的流程：
0085 |
0086 | 1. 在 **Input Text** 输入或粘贴英文学生文本。
0087 | 2. 点击 **Redact Text**。
0088 | 3. JavaScript 将原文以 JSON 发送到 `POST /api/redact`。
0089 | 4. FastAPI 使用仓库中的自训练 checkpoint 推理。
0090 | 5. **Redacted Text** 只读框显示模型生成的结果。
0091 | 6. 点击 **Clear** 清空两个文本框。
0092 |
0093 | 网页没有 JavaScript 关键词表或姓名硬编码。核心 NER 始终在后端执行。空白、超过 100,000 字符、模型不可用、服务器错误和网络失败都会显示用户可见提示
0094 |
0095 | 也可以直接测试精简 API：
0096 |
0097 | ```powershell
0098 | $body = @{ text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru." } | C
.... | >onvertTo-Json
0099 | Invoke-RestMethod -Uri http://127.0.0.1:8000/api/redact -Method Post -ContentType "applicati
.... | >on/json" -Body $body
0100 | ```
0101 |
0102 | 预期响应：
0103 |
0104 | ```json
0105 | {"redacted_text":"[PER] studies at [ORG] in [LOC]."}
0106 | ```
0107 |
0108 | ## Model Prediction
0109 |
0110 | 输入 `Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru.`，实际模型返回 `Ahmad Ali/PE
.... | >R`、`Universiti Teknologi Malaysia/ORG`、`Johor Bahru/LOC` 及字符位置。
0111 |
0112 | ```powershell
0113 | $body = @{ text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru." } | C
.... | >onvertTo-Json
0114 | Invoke-RestMethod -Uri http://127.0.0.1:8000/api/predict -Method Post -ContentType "applicat
.... | >ion/json" -Body $body

源码第 115-152 行

````
```

```python
from src.predict import Predictor
result = Predictor().predict("Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Ba
>hru.")
print(result["entities"])
```

## Text Redaction

上述输入的实际脱敏结果为 `[PER] studies at [ORG] in [LOC].`。题目示例句也实际得到 `[PER], a student from [ORG], att
>ended an event in [LOC].`。实现按字符 span 从右向左替换，避免位置偏移。

## File Upload

后端仍保留 `POST /api/upload` 供 TXT/PDF 客户端调用。TXT 必须 UTF-8；PDF 必须有文本层；扫描件不支持 OCR；单文件最大 5 MB。为了保持本
>次网页极简，主页不显示文件上传控件。

## Model Training

Clone 后运行不需要训练。只有重新实验时才执行：

```powershell
python -m src.generate_data
python -m src.train --epochs 16 --batch-size 64 --learning-rate 0.003
python -m src.evaluate
```

训练只合入 general/Malaysian train；validation 用于选模；两个 test split 不参与训练或选模。checkpoint 保存权重、vocabul
>ary、labels、config、seed、数据计数和最佳 epoch。

## Model Evaluation

2026-10-29 在 Windows 11、Python 3.12.15、PyTorch 2.9.0+cpu 重新训练与评估：

- 16 epochs；batch size 64；learning rate 0.003；Adam；seed 42。
- CPU training loop: 14.2677 seconds。
- Best epoch: 16；validation entity F1: 0.9192。
- General student synthetic test (300): P/R/F1 = 0.9843/1.0000/0.9921。
- Malaysian synthetic test (60): P/R/F1 = 1.0000/1.0000/1.0000。
- PERSON precision/recall/F1 = 1.0000/1.0000/1.0000 on both synthetic tests。
````

源码第 153-191 行

````

学生场景固定挑战矩阵为 13/13 通过，覆盖 Malaysian 学生姓名、大学、学校、投诉、支持聊天、反馈、活动、重复实体、多句和未知词。它仍是小型人工检查矩
>告评估。完整实际输出在 `reports/manual_acceptance.json`。

## Testing

```powershell
$env:OMP_NUM_THREADS="1"
$env:MKL_NUM_THREADS="1"
python -m pytest -q
```

最新实测：`56 passed`。覆盖学生数据、合并实体池泄漏、preprocessing、vocabulary、BIO、Tensor/DataLoader 小实验、model、for
>ward、loss/backpropagation、checkpoint、prediction、三类实体、标点 span、长文本、重复实体、redaction、Malaysian da
>ta、TXT、单页/多页/损坏 PDF、API/Web、空白/超长输入、连续提交、模型不可用、CPU inference 与 end-to-end。

## Project Structure

```text
app/main.py                 FastAPI、Web UI、JSON API、TXT/PDF 上传
src/generate_data.py        General/Malaysian 合成 BIO 数据与 split 检查
src/preprocessing.py        Token、字符 span、BIO 校验、vocabulary
src/dataset.py              Dataset、padding、mask、word dropout
src/model.py                自定义 BiLSTM token classifier
src/train.py                Training、validation、checkpoint、history
src/evaluate.py             General/Malaysian 独立 test evaluation
src/predict.py              CPU checkpoint loading、BIO/entity extraction
src/redact.py               基于字符 span 的可靠脱敏
tests/                      56 项自动化测试
models/best_model.pt        已训练 checkpoint；Clone 后直接使用
data/processed/             可分发的合成 general 数据
data/malaysian/             可分发的合成 Malaysian 数据
reports/                    训练与评估 JSON 证据
````

```
MODEL_ACCEPTANCE_REPORT.md  完整工程验收记录
TUTORIAL_AUDIT_REPORT.md    22 阶段零基础教程审计与复现边界
```

```
## Limitations

- 模型针对学生报告、学生支持聊天、投诉、反馈、大学行政报告和学生活动进行优化；不提供医院病人、公司员工或客户场景的专项保证。
- 合成模板数据不等于真实学生 incident report、support chat 或 CoNLL-2003。
- PER/LOC/ORG 不覆盖电话、邮箱、Student ID、Passport、完整地址等所有 PII。
```

源码第 192-206 行

```
- 无 CRF；非法 BIO 转移主要由后处理修复。
- 不支持扫描 PDF OCR。
- 高风险大学、医院或企业资料必须人工复核，尤其要防止姓名漏检。

## Reproducibility

- 依赖固定在 `pyproject.toml`、`uv.lock`、`requirements.txt`。
- seed 42；每 epoch 指标在 `reports/training_log.json`。
- 测试指标在 `reports/test_metrics.json` 与 `reports/malaysian_test_metrics.json`。
- checkpoint 内含 vocabulary、label mapping 与 model configuration，不依赖外部下载。
- 完整证据和 Clone 验收状态见 `MODEL_ACCEPTANCE_REPORT.md`。

## License

项目源码使用 [MIT License](LICENSE)。仓库中的 general 与 Malaysian 数据均由本项目脚本生成，是用于教学和工程验收的合成数据，不是 CoNLL-2003 或真实个人资料。
```

立刻运行

```
Get-Content README.md | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 01：Windows 11、Python、VS Code 与虚拟环境

1. 本阶段学习目标

你将从没有打开任何开发工具开始，建立一个可重复进入的 Windows 11 开发环境。最终证明是：VS Code 使用项目 `.venv` 里的 Python，`learning/hello_python.py` 能打印版本、操作系统和解释器绝对路径。

为什么现在做？后面的 PyTorch、pytest、FastAPI 都依赖解释器和包。若系统 Python、Microsoft Store 占位符与 `.venv` 混用，会出现“明明安装过却 import 失败”的错误。

2. 本阶段需要掌握的知识

- Program、terminal、command、working directory。 - Python interpreter 与 `.py` source file。 - PATH、`where.exe`、版本号。 - Module、pip、virtual environment。 - VS Code Explorer、Extension、Integrated Terminal、Interpreter selector。 - CPU 版 PyTorch 与依赖版本锁定。

3. 从零解释知识点

PowerShell 是输入命令的程序，不是 Python。`PS D:\project>` 左侧称为 prompt；`D:\project` 是当前 Working Directory。Python interpreter 是读取 `.py` 并执行的程序。一个电脑可安装多个 Python，所以“python 命令存在”不代表它指向正确版本。

Virtual Environment 是项目专用 Python 环境。全局环境像公共工具箱：多个项目可能要求冲突版本；`.venv` 像本项目独立工具箱。激活环境只是把 `.venv\Scripts` 临时放到当前终端 PATH 前面，不会修改你的 Python 源码。

python -m venv .venv 中：python 启动解释器；-m 要求 Python 按模块运行；venv 是标准库模块；.venv 是新目录名。执行后会产生 Scripts/python.exe、激活脚本和包目录。.venv 体积大且与电脑相关，所以写入 .gitignore，不上传 GitHub。

4. 本阶段开始前的项目状态

```
Intern-MachineLearning/
  tutorial/chapters/phase_00.md   [已存在：上一阶段]
  learning/                       [本阶段新增]
    hello_python.py               [本阶段新增]
  .venv/                          [本阶段生成，不提交 Git]
```

如果你拿到完整仓库，会看到更多文件。学习时只关注本阶段列出的文件，不需要提前读 src/model.py。

5. 本阶段需要使用的工具

- Windows 11 Settings 或浏览器：安装 Python/VS Code。 - PowerShell：检查版本、创建环境、安装包、运行程序。 - VS Code：创建与编辑 learning/hello_python.py。 - 本阶段所有命令都在项目根目录执行；除安装检查外，应激活 .venv。

6. Lesson 01.1：打开 PowerShell并检查 Python

操作 1：按键盘 Windows 键，输入 PowerShell，点击 Windows PowerShell。不要在 Python >>> 提示符里输入 PowerShell 命令。

输入：

```
python --version
```

--version 要求程序只报告版本。正常例子是 Python 3.12.15。本项目验证范围是 Python >=3.11,<3.13；3.12 最稳妥。若出现 Microsoft Store 或 “file cannot be accessed”，继续执行：

```
where.exe python
```

它会列出候选路径。WindowsApps\python.exe 可能只是占位符。真正解释器常见于 AppData\Local\Programs\Python。若完全找不到 Python：浏览 https://www.python.org/downloads/windows/，选择 Python 3.12 64-bit installer；安装第一页勾选 Add python.exe to PATH，再选 Install Now。完成后关闭并重新打开 PowerShell，再检查版本。

检查 pip：

```
python -m pip --version
```

这里使用 python -m pip 而不是裸 pip，可以确认 pip 属于同一个解释器。预期输出包含 pip 版本和 Python site-packages 路径。若提示 No module named pip，运行 python -m ensurepip --upgrade。

7. Lesson 01.2：安装并打开 VS Code

按 Windows 键搜索 Visual Studio Code。若没有，浏览 https://code.visualstudio.com/ 下载 Windows User Installer。安装时建议勾选 Add to PATH、Open with Code 和 Register Code as an editor。

打开 VS Code，点击左侧 Extensions（四个方块），搜索 Python，确认发布者是 Microsoft，点击 Install。这个 extension 提供语法提示、解释器选择和调试，但它本身不是 Python。

创建项目文件夹时，本机策略要求 vibe coding 项目默认放在 D:\BBB_vibecoding_project；本教学仓库目前位于 D:\AIROST\Intern-MachineLearning，因为用户明确提供了现有工作区。学生的新副本可执行：

```
New-Item -ItemType Directory -Path 'D:\BBB_vibecoding_project\personal-information-ner'
Set-Location 'D:\BBB_vibecoding_project\personal-information-ner'
```

`New-Item` 创建目录；`Set-Location` 改变当前目录。路径有连字符但无空格，单引号仍能避免 PowerShell 误解析。若 D: 不存在，先在 File Explorer 确认磁盘，或向老师询问允许的位置。

在 VS Code 点击 File -> Open Folder，选择项目目录，点击 Select Folder。点击 Terminal -> New Terminal。底部应出现 PowerShell，prompt 中路径应是项目根目录。输入：

```
Get-Location
```

预期 Path 与打开的文件夹一致。若不一致，使用 `Set-Location` 修正。

8. Lesson 01.3：创建、激活并选择 Virtual Environment

在 VS Code Integrated Terminal、项目根目录执行：

```
python -m venv .venv
```

命令成功通常没有大量输出，但 Explorer 会出现 `.venv`。如果 Access denied，确认项目不在受保护的 Program Files；如果找不到 venv，确认你用的是完整 python.org 安装。

激活：

```
.\.venv\Scripts\Activate.ps1
```

开头 `.` 表示当前目录，反斜线是 Windows 路径。成功后 prompt 前出现 `(.venv)`。若提示 execution policy 禁止脚本，只为当前 PowerShell 进程执行：

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

`-Scope Process` 保证关闭窗口后失效，不永久降低系统安全设置。

验证：

```
python -c "import sys; print(sys.executable)"
```

`-c` 执行引号里的短代码。预期路径以项目的 `.venv\Scripts\python.exe` 结尾。若仍是系统 Python，关闭终端，新建 PowerShell，再激活。

在 VS Code 按 Ctrl+Shift+P，输入 `Python: Select Interpreter`，选择包含 `.venv` 的条目。右下角状态栏应显示 `.venv`。Terminal 激活和编辑器解释器选择是两件事，两者都要确认。

9. Lesson 01.4：安装 uv、PyTorch 与依赖

本仓库用 `uv.lock` 锁定版本。若已安装 uv：

```
uv --version
uv sync --all-groups
```

第一条只读；第二条读取 `pyproject.toml` 和 `uv.lock`，安装 runtime 与 dev dependencies。它会创建或更新 `.venv`。完整依赖包括 PyTorch 2.9.0、FastAPI、pypdf、pytest、ReportLab 等。

若没有 uv，可以先在已激活 `.venv` 使用：

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

`install` 下载并安装；`-r` 表示从文件读取多项依赖。网络错误时检查校园代理；权限错误通常说明没有激活 `.venv`。不要用管理员权限掩盖路径错误。

验证关键依赖：

```
python -c "import torch, fastapi, pypdf, pytest; print(torch.__version__); print(torch.cuda.is_available(
))"
```

本机真实验证版本是 Python 3.12.15、PyTorch 2.9.0+cpu；`cuda.is_available()` 为 False，项目按 CPU 设计。其他电脑的小版本可能不同，但必须满足 `pyproject.toml`。

9. Lesson 01.5：创建并运行第一个 Python 文件

在 VS Code Explorer 点击 New Folder，输入 `learning`；右键该目录 -> New File -> 输入 `hello_python.py`。第一行写 docstring：

```
"""Phase 01 的第一个可运行 Python 程序。"""
```

Docstring 是给人和工具看的模块说明。接着逐行输入：

```
import platform
import sys
```

`import` 加载模块。`platform` 提供系统/版本信息；`sys` 提供当前解释器路径。然后创建函数：

```
def describe_environment() -> str:
    """返回当前 Python 与操作系统信息。"""
    return f"Python {platform.python_version()} | {platform.system()} | executable={sys.executable}"
```

`def` 定义函数；括号为空代表不接收参数；`-> str` 是返回类型提示；缩进的 `return` 把字符串交给调用者。f-string 用 `{表达式}` 插入值。

最后输入：

```
if __name__ == "__main__":
    print("NER learning environment is ready.")
    print(describe_environment())
```

当直接运行文件时，`__name__` 是 `__main__`；当别的文件 import 它时，这两行不会自动打印。这让函数以后能被测试安全导入。

保存 Ctrl+S，在 Integrated Terminal 项目根目录运行：

```
python learning\hello_python.py
```

本机实际输出结构：

```
NER learning environment is ready.
Python 3.12.15 | Windows | executable=D:\...\Intern-MachineLearning\.venv\Scripts\python.exe
```

路径会因电脑不同而不同；关键是包含 `.venv`。

9. Lesson 01.6：退出与重新进入项目

退出环境：

```
deactivate
```

`(.venv)` 从 prompt 消失，但 `.venv` 文件夹仍存在。第二天重新开发：打开 VS Code -> File/Open Recent -> 项目 -> Terminal/New Terminal -> 执行 `. \.venv\Scripts\Activate.ps1` -> `python --version` -> 运行 hello 文件。无需每天重建环境。

10. 完整代码回顾

阶段结束时 `learning/hello_python.py` 完整代码：

```
"""Phase 01 的第一个可运行 Python 程序。"""

import platform
import sys


def describe_environment() -> str:
    """返回当前 Python 与操作系统信息，帮助学生确认解释器。"""
    return f"Python {platform.python_version()} | {platform.system()} | executable={sys.executable}"


if __name__ == "__main__":
    print("NER learning environment is ready.")
    print(describe_environment())
```

11. 跨文件调用关系

目前没有业务文件依赖它。Phase 01 的价值是验证所有后续文件共享同一解释器。测试未来可以 `from learning.hello_python import describe_environment`，此时 main guard 会阻止额外打印。

`.venv` 根据 `pyproject.toml`/`uv.lock` 安装包；Python extension 选择 `.venv\Scripts\python.exe`；Terminal 调用同一路径；源文件 import 已安装模块。

12. 数据流解释

`platform.python_version()` 返回 `str`；`platform.system()` 返回 `str`；`sys.executable` 是路径字符串。f-string 合并三个字符串，函数返回 `str`，`print` 把它发送到 terminal 标准输出。此阶段没有 Tensor。

13. 常见错误

1. `python` 打开 Store：使用 `where.exe python` 找真实安装并修复 PATH。 2. `Activate.ps1 cannot be loaded`：只对当前 Process 使用 Bypass。 3. `ModuleNotFoundError`：检查 `sys.executable` 是否指向 `.venv`，再安装包。 4. VS Code 有红线但终端能运行：编辑器没有选择 `.venv` interpreter。 5. `IndentationError`：函数体使用四个空格，不混 Tab。 6. 在 `>>>` 输入 PowerShell 命令：输入 `exit()` 回到 PowerShell。 7. 将 `.venv` 提交 Git：确认 `.gitignore` 包含 `.venv/`。

14. Checkpoint

在项目根目录、激活环境后运行：

```
python --version
python -c "import sys; print(sys.executable)"
python learning\hello_python.py
python -c "import torch; print(torch.__version__)"
```

通过条件：Python 3.11/3.12；路径含 `.venv`; hello 打印 ready；torch 可 import。当前仓库真实执行：Python 3.12.15 和 PyTorch 2.9.0+cpu。任一失败先按第 13 节修复。

15. 学生练习

1. 给 `describe_environment` 增加 `platform.machine()`，观察 64-bit 架构文字。 2. 故意在函数缩进中删一个空格，读懂 IndentationError 后恢复。 3. 退出 `.venv`，比较 `sys.executable`，然后重新激活。 4. 用 `python -m pip list` 找出 torch 和 fastapi 版本。 5. 解释为什么不把 `.venv` 上传 GitHub。

16. 面试准备

简单英语："I use a project-specific virtual environment so dependency versions do not conflict with other projects. VS Code and PowerShell both use the same `.venv` interpreter. I verified Python 3.12 and the CPU build of PyTorch before writing model code."

17. 下一阶段预告

Phase 02 会在 `learning/python_foundations.py` 中学习 list、dict、function、class、object、inheritance、`self`、`super()` 和 type hint。这些知识会在 Phase 09 的 `nn.Module` 中逐项复用，而不是第一次看到模型类时才仓促解释。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/hello_python.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-14 行

```
0001 | """Phase 01 的第一个可运行 Python 程序。"""
0002 |
0003 | import platform
0004 | import sys
0005 |
0006 |
0007 | def describe_environment() -> str:
0008 |     """返回当前 Python 与操作系统信息，帮助学生确认解释器。"""
0009 |     return f"Python {platform.python_version()} | {platform.system()} | executable={sys.exec
.... | >utable}"
0010 |
0011 |
0012 | if __name__ == "__main__":
0013 |     print("NER learning environment is ready.")
0014 |     print(describe_environment())
```

立刻运行

```
python -m py_compile learning/hello_python.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：requirements.txt

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 txt 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | # This file was autogenerated by uv via the following command:
0002 | #    uv export --all-groups --format requirements-txt --no-hashes --output-file requirements
.... | >.txt
0003 | annotated-doc==0.0.5
0004 |     # via fastapi
0005 | annotated-types==0.8.0
0006 |     # via pydantic
0007 | anyio==4.15.1
0008 |     # via
0009 |     #   httpx
0010 |     #   starlette
0011 | certifi==2026.10.23
0012 |     # via
0013 |     #   httpcore
0014 |     #   httpx
0015 | cffi==2.1.1 ; platform_python_implementation != 'PyPy'
0016 |     # via cryptography
```

```
0017 | charset-normalizer==3.5.2
0018 |     # via
0019 |     #   pdfminer-six
0020 |     #   reportlab
0021 | click==8.5.0
0022 |     # via uvicorn
0023 | cloudpickle==3.1.2
0024 |     # via joblib
0025 | colorama==0.4.6 ; sys_platform == 'win32'
0026 |     # via pytest
0027 | coverage==7.17.0
0028 |     # via pytest-cov
0029 | cryptography==50.0.2
0030 |     # via pdfminer-six
0031 | fastapi==0.120.1
0032 |     # via personal-information-ner
0033 | filelock==4.2.4
0034 |     # via torch
0035 | fsspec==2026.9.0
0036 |     # via torch
0037 | h11==0.16.0
0038 |     # via
0039 |     #   httpcore
0040 |     #   uvicorn
0041 | httpcore==1.0.9
```

源码第 42-83 行

```
0042 |     # via httpx
0043 | httpx==0.28.1
0044 | idna==3.20
0045 |     # via
0046 |     #   anyio
0047 |     #   httpx
0048 | iniconfig==2.3.1
0049 |     # via pytest
0050 | jinja2==3.1.6
0051 |     # via
0052 |     #   personal-information-ner
0053 |     #   torch
0054 | joblib==1.6.0
0055 |     # via scikit-learn
0056 | markupsafe==3.0.4
0057 |     # via jinja2
0058 | mpmath==1.3.0
0059 |     # via sympy
0060 | networkx==3.6.1 ; python_full_version < '3.12'
0061 |     # via torch
0062 | networkx==3.7 ; python_full_version >= '3.12'
0063 |     # via torch
0064 | numpy==2.3.4
0065 |     # via
0066 |     #   pandas
0067 |     #   personal-information-ner
0068 |     #   scikit-learn
0069 |     #   scipy
0070 | nvidia-cublas-cu12==12.8.4.1 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0071 |     # via
0072 |     #   nvidia-cudnn-cu12
0073 |     #   nvidia-cusolver-cu12
0074 |     #   torch
0075 | nvidia-cuda-cupti-cu12==12.8.90 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0076 |     # via torch
0077 | nvidia-cuda-nvrtc-cu12==12.8.93 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0078 |     # via torch
0079 | nvidia-cuda-runtime-cu12==12.8.90 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0080 |     # via torch
0081 | nvidia-cudnn-cu12==9.10.2.21 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0082 |     # via torch
0083 | nvidia-cufft-cu12==11.3.3.83 ; platform_machine == 'x86_64' and sys_platform == 'linux'
```

源码第 84-125 行

```
0084 |     # via torch
0085 | nvidia-cufile-cu12==1.13.1.3 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0086 |     # via torch
0087 | nvidia-curand-cu12==10.3.9.90 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0088 |     # via torch
```

```
0089 | nvidia-cusolver-cu12==11.7.3.90 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0090 |     # via torch
0091 | nvidia-cusparse-cu12==12.5.8.93 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0092 |     # via
0093 |     #   nvidia-cusolver-cu12
0094 |     #   torch
0095 | nvidia-cusparselt-cu12==0.7.1 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0096 |     # via torch
0097 | nvidia-nccl-cu12==2.27.5 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0098 |     # via torch
0099 | nvidia-nvjitlink-cu12==12.8.93 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0100 |     # via
0101 |     #   nvidia-cufft-cu12
0102 |     #   nvidia-cusolver-cu12
0103 |     #   nvidia-cusparse-cu12
0104 |     #   torch
0105 | nvidia-nvshmem-cu12==3.3.20 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0106 |     # via torch
0107 | nvidia-nvtx-cu12==12.8.90 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0108 |     # via torch
0109 | packaging==26.3
0110 |     # via pytest
0111 | pandas==2.3.3
0112 |     # via personal-information-ner
0113 | pdfminer-six==20250506
0114 |     # via pdfplumber
0115 | pdfplumber==0.11.7
0116 | pillow==13.0.0
0117 |     # via
0118 |     #   pdfplumber
0119 |     #   reportlab
0120 | pluggy==1.7.0
0121 |     # via
0122 |     #   pytest
0123 |     #   pytest-cov
0124 | pycparser==3.12 ; implementation_name != 'PyPy' and platform_python_implementation != 'PyPy'
0125 |     # via cffi
```

源码第 126-167 行

```
0126 | pydantic==2.12.3
0127 |     # via
0128 |     #   fastapi
0129 |     #   personal-information-ner
0130 | pydantic-core==2.41.4
0131 |     # via pydantic
0132 | pygments==2.21.0
0133 |     # via pytest
0134 | pypdf==6.1.3
0135 |     # via personal-information-ner
0136 | pypdfium2==5.14.0
0137 |     # via pdfplumber
0138 | pytest==8.4.2
0139 |     # via pytest-cov
0140 | pytest-cov==7.0.0
0141 | python-dateutil==3.0.0
0142 |     # via pandas
0143 | python-multipart==0.0.20
0144 |     # via personal-information-ner
0145 | pytz==2026.5
0146 |     # via pandas
0147 | reportlab==4.4.4
0148 | scikit-learn==1.7.2
0149 |     # via personal-information-ner
0150 | scipy==1.17.1 ; python_full_version < '3.12'
0151 |     # via scikit-learn
0152 | scipy==1.18.1 ; python_full_version >= '3.12'
0153 |     # via scikit-learn
0154 | setuptools==84.2.1 ; python_full_version >= '3.12'
0155 |     # via torch
0156 | six==1.17.0
0157 |     # via python-dateutil
0158 | starlette==0.49.3
0159 |     # via fastapi
0160 | sympy==1.14.0
0161 |     # via torch
0162 | threadpoolctl==3.7.0
0163 |     # via scikit-learn
```

```
0164 | torch==2.9.0
0165 |     # via personal-information-ner
0166 | triton==3.5.0 ; platform_machine == 'x86_64' and sys_platform == 'linux'
0167 |     # via torch
```

源码第 168-184 行

```
0168 | typing-extensions==4.16.0
0169 |     # via
0170 |     #   anyio
0171 |     #   fastapi
0172 |     #   pydantic
0173 |     #   pydantic-core
0174 |     #   starlette
0175 |     #   torch
0176 |     #   typing-inspection
0177 | typing-inspection==0.4.4
0178 |     # via pydantic
0179 | tzdata==2026.5
0180 |     # via
0181 |     #   pandas
0182 |     #   python-dateutil
0183 | uvicorn==0.38.0
0184 |     # via personal-information-ner
```

立刻运行

```
Get-Content requirements.txt | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 02：为机器学习补齐 Python 基础

1. 本阶段学习目标

本阶段创建 `learning/python_foundations.py`。你会用 NER 真实数据形式学习 list、dict、loop、function、parameter、return、class、object、inheritance、`self`、`__init__()`、`super()`、dataclass 与 type hint。结束时程序应打印 BIO 计数和实体字典。

这些概念现在学，是为了 Phase 09 看到 `class BiLSTMNER(nn.Module)` 时不同时承受 Python 和神经网络两层新知识。

2. 本阶段需要掌握的知识

- `list[str]`：有顺序、可重复的字符串集合。 - `dict[str, int]`：键到值的映射。 - function、parameter、argument、return value。 - class 是对象蓝图；object 是按蓝图创建的实例。 - inheritance 让子类获得父类行为。 - `self` 指当前对象；`super()` 访问父类。 - type hint 帮助阅读和静态检查，不会自动替你验证所有运行时数据。

3. 从零解释知识点

句子的 token 和 BIO label 都需要保持顺序，因此用 list。例如 `labels[0]` 是第一项。实体需要同时保存 text/type/start/end，使用 dict 可以按名字取值。函数把可重复逻辑命名；调用者传 argument，函数的 parameter 接收它，`return` 将结果交回调用者。

Class 像“实体说明书”，object 是一条具体实体。`self.name` 是每个 object 自己的数据。`__init__` 在创建 object 时运行。Inheritance 的语法是 `class Child(Parent)`。`super().__init__()` 调用父类初始化；在 PyTorch 中不调用它，`nn.Module` 可能无法登记层和参数。

Dataclass 自动生成保存字段所需的 `__init__` 等方法。因为 dataclass 与自定义父类同时使用，我们在 `__post_init__` 再调用父类初始化。

4. 本阶段开始前的项目状态

```
learning/
  hello_python.py          [上一阶段已有]
  python_foundations.py    [本阶段新增]
tutorial/chapters/
  phase_00.md              [不变]
  phase_01.md              [不变]
  phase_02.md              [本阶段新增]
```

正式 `src/` 尚未开始；本阶段代码故意放 `learning/`，避免练习代码混进模型核心。

5. 本阶段需要使用的工具

打开 VS Code Explorer -> `learning` -> New File -> `python_foundations.py`。使用项目根目录 PowerShell，确认 (`.venv`) 和 `python -c "import sys; print(sys.executable)"` 指向项目环境。

6. Lesson 02.1：list、loop 与 dict

先输入：

```
labels = ["B-PER", "O", "O", "B-ORG", "I-ORG"]
counts = {}
for label in labels:
    counts[label] = counts.get(label, 0) + 1
print(counts)
```

第一行创建五项 list。`for` 依次让变量 `label` 指向每项。`counts.get(label, 0)` 在 key 不存在时给 0，然后加 1。运行：

```
python learning\python_foundations.py
```

临时版本实际应打印 `{'B-PER': 1, 'O': 2, 'B-ORG': 1, 'I-ORG': 1}`。若 `KeyError`，你可能写成 `counts[label] += 1`，但第一次还没有 key。

7. Lesson 02.2：把逻辑变成 function

把上面代码改为：

```
def count_labels(labels: list[str]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    return counts
```

Function name 是 `count_labels`。Parameter name 是 `labels`，类型提示 `list[str]`。Return type 是 `dict[str,int]`。内部局部变量 `counts` 在调用结束后不应由外部直接修改。

调用示例：

```
result = count_labels(["O", "B-PER", "O"])
assert result == {"O": 2, "B-PER": 1}
```

`assert` 在条件为 False 时抛出 AssertionError，很适合小型阶段测试。函数未来可由测试文件 import。

8. Lesson 02.3：class、object、self 与 `__init__`

继续输入：

```
class NamedItem:
    def __init__(self, name: str) -> None:
        self.name = name
```

`NamedItem` 是 class name。调用 `NamedItem("Sarah")` 创建 object。Python 自动把 object 作为第一个 argument 传入 `self`，把 "Sarah" 传入 `name`。`self.name = name` 将参数保存为对象属性。

局部测试：

```
person = NamedItem("Sarah")
assert person.name == "Sarah"
```

为什么不是普通函数？因为实体有持续状态，多个 method 要共享 name/type/span；对象把这些数据和行为放在一起。简单项目也可用 dict，但我们现在学习 class 是为了理解 PyTorch Module。

9. Lesson 02.4：inheritance 与 `super()`

先写最小子类：

```
class Entity(NamedItem):
    def __init__(self, name: str, entity_type: str) -> None:
        super().__init__(name)
        self.entity_type = entity_type
```

括号里的 `NamedItem` 是父类。`super()` 返回一个代理，让当前子类调用父类实现。`super().__init__(name)` 负责建立 `self.name`；下一行建立新属性。若删掉 super 调用，`entity.name` 会不存在。

测试：

```
entity = Entity("Sarah", "PER")
assert entity.name == "Sarah"
assert entity.entity_type == "PER"
```

与未来 PyTorch 的关系：`BiLSTMNER(nn.Module)` 继承 PyTorch 父类；`super().__init__()` 让父类准备参数、子模块、train/eval 状态等内部容器。

9. Lesson 02.5：dataclass 与半开区间

在 imports 加：

```
from dataclasses import dataclass
```

将 Entity 改为：

```
@dataclass
class Entity(NamedItem):
    name: str
    entity_type: str
    start: int
    end: int

    def __post_init__(self) -> None:
        super().__init__(self.name)
```

Decorator `@dataclass` 在 class 创建后增强它，自动生成接收四个字段的 constructor。半开区间 `[start,end)` 包含 start、不包含 end，所以原文实体可直接取 `text[start:end]`。

`__post_init__` 在 dataclass 自动初始化字段后运行，再调用父类。这里父类赋同一个 name，主要用于演示；正式模型不会使用这个 Entity class。

9. Lesson 02.6：method、union type 与 main guard

在 Entity 内加入：

```
def as_dict(self) -> dict[str, str | int]:
    return {
        "text": self.name,
        "type": self.entity_type,
        "start": self.start,
        "end": self.end,
    }
```

这是 instance method，通过 entity.as_dict() 调用。返回值的 key 是 str；value 可能是 str 或 int，因此用 str | int。最后创建 main() 并用 main guard 调用，这与 Phase 01 原理相同。

运行实际最终版本：

```
python learning\python_foundations.py
```

真实输出：

```
{'B-PER': 1, 'O': 2, 'B-ORG': 1, 'I-ORG': 1}
{'text': 'Universiti Malaya', 'type': 'ORG', 'start': 15, 'end': 33}
```

10. 完整代码回顾

阶段结束时完整代码位于 learning/python_foundations.py，包括 import、count_labels、NamedItem、dataclass Entity、as_dict、main 和 main guard。请打开真实文件逐行比对，不要只复制片段。运行 python -m py_compile learning\python_foundations.py 可先检查语法。

11. 跨文件调用关系

main() 调用 count_labels() 和 Entity(...)；Entity 自动调用 dataclass constructor -> __post_init__() -> NamedItem.__init__()；随后 main 调用 entity.as_dict()。本阶段不 import src/，因此不会影响正式模型。

12. 数据流解释

labels 的类型是 list[str] -> count_labels 的 loop -> dict[str,int] -> print。构造 Entity 时四个 argument -> object fields -> as_dict() -> dict[str,str|int]。没有 Tensor；Python 类型是后续 Tensor 的上游原始数据。

13. 常见错误

1. NameError: dataclass：忘记 import。 2. TypeError: missing argument：构造 Entity 时字段数量不对。 3. AttributeError: name：普通继承版本忘记 super().__init__()。 4. IndentationError：method 必须缩进在 class 内。 5. Function 打印但不 return：调用者得到 None；应使用 return。 6. 把 self 当用户参数传入：正常调用 entity.as_dict() 不写 self。

14. Checkpoint

执行：

```
python -m py_compile learning\python_foundations.py
python learning\python_foundations.py
python -c "from learning.python_foundations import count_labels; assert count_labels(['O','O']) == {'O':
2}; print('PASS')"
```

通过条件：无语法错误、输出两个 dict、最后打印 PASS。本阶段真实执行结果将写入 snapshot manifest。

15. 学生练习

1. 给 Entity 添加 length()，返回 end-start。 2. 输入重复 PER 标签验证 count。 3. 暂时删除 super 调用，观察什么测试失败，然后恢复。 4. 把错误的 end=32 改成 33，解释半开区间。 5. 用自己的名字创建 PER object，但不要提交真实敏感数据。

16. 面试准备

"A class defines state and behavior, while an object is one instance. self refers to the current object. Inheritance lets a child reuse a parent. super().__init__() initializes the parent part. I learned these

concepts before using PyTorch `nn.Module`."

17. 下一阶段预告

Phase 03 创建 `learning/bio_demo.py`，把 Python list/dict/function 用在真实 BIO 序列：验证标签、把 tags 转换成实体 span，并用具体英文句子解释 token 与 entity 的区别。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/python_foundations.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """Phase 02: 为 NER 项目准备 Python 数据结构、函数与类。"""
0002 |
0003 | from dataclasses import dataclass
0004 |
0005 |
0006 | def count_labels(labels: list[str]) -> dict[str, int]:
0007 |     """统计每个 BIO 标签出现次数。"""
0008 |     counts: dict[str, int] = {}
0009 |     for label in labels:
0010 |         counts[label] = counts.get(label, 0) + 1
0011 |     return counts
0012 |
0013 |
0014 | class NamedItem:
0015 |     """保存一个名称，演示普通父类。"""
0016 |
0017 |     def __init__(self, name: str) -> None:
0018 |         self.name = name
0019 |
0020 |
0021 | @dataclass
0022 | class Entity(NamedItem):
0023 |     """一个带类型和半开区间的实体。"""
0024 |
0025 |     name: str
0026 |     entity_type: str
0027 |     start: int
0028 |     end: int
0029 |
0030 |     def __post_init__(self) -> None:
0031 |         super().__init__(self.name)
0032 |
0033 |     def as_dict(self) -> dict[str, str | int]:
0034 |         return {
0035 |             "text": self.name,
0036 |             "type": self.entity_type,
0037 |             "start": self.start,
0038 |             "end": self.end,
0039 |         }
0040 |
0041 |
0042 | def main() -> None:
```

源码第 43-50 行

```
    labels = ["B-PER", "O", "O", "B-ORG", "I-ORG"]
    print(count_labels(labels))
    entity = Entity("Universiti Malaya", "ORG", 15, 33)
    print(entity.as_dict())


if __name__ == "__main__":
    main()
```

立刻运行

```
python -m py_compile learning/python_foundations.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 03：从零认识 NLP、NER 与 BIO

1. 本阶段学习目标

你会创建 `learning/bio_demo.py`，手工写出 token 与 BIO label，验证标注合法性，并把连续标签合并为实体。这里没有神经网络：先完全理解“模型未来要预测的答案长什么样”。

完成后你必须能解释 token、sequence、label、entity、span、B/I/O，并能指出 `I-ORG` 为什么不能无缘无故出现在句首。

2. 本阶段需要掌握的知识

- Natural Language Processing (NLP) 与 Named Entity Recognition (NER)。 - Sequence labeling：为序列中每个 token 预测一个 label。 - Token 不等于单纯按空格分词；标点也可成为 token。 - BIO：Begin、Inside、Outside。 - Entity span 使用 `[start_token,end_token)` 半开区间。 - Sentinel：循环末尾临时添加 `O`，统一关闭最后实体。

3. 从零解释知识点

NLP 是让计算机处理人类语言的领域。NER 是其中一个任务：找出有名称的实体并分类。输入句子是 sequence，模型输出等长 label sequence。

句子 `Ali studies at Universiti Malaya.` 可拆成 `Ali`, `studies`, `at`, `Universiti`, `Malaya`, `.`。`Ali` 是单 token PER；机构有两个 token。BIO 标注为 `B-PER O O B-ORG I-ORG O`。B 表示新实体开始，I 表示同类型实体继续，O 表示不属于目标实体。

如果只有 `ORG` 而没有 B/I，就难以区分相邻的两个实体。规范 BIO 中 `I-LOC` 必须紧跟 B-LOC 或 I-LOC；gold data 出现非法 I 时应拒绝，而不是悄悄训练。

4. 本阶段开始前的项目状态

```
learning/
  hello_python.py             [已有]
  python_foundations.py       [已有]
  bio_demo.py                 [本阶段新增]
tutorial/chapters/phase_03.md [本阶段新增]
```

`src/preprocessing.py` 还没有创建；Phase 06 会把本阶段概念升级为正式实现。

5. 本阶段需要使用的工具

在 VS Code Explorer 的 `learning` 下新建 `bio_demo.py`。Integrated Terminal 必须位于项目根目录并激活 `.venv`。本阶段只使用 Python 标准库，不需要 torch。

6. Lesson 03.1：手工建立 token/label 对齐

先写：

```
tokens = ["Ali", "studies", "at", "Universiti", "Malaya", "."]
labels = ["B-PER", "O", "O", "B-ORG", "I-ORG", "O"]
print(list(zip(tokens, labels, strict=True)))
```

`zip` 把相同位置配对。`strict=True` 在两个 list 长度不同时抛错，避免静默丢数据。运行临时版本，预期看到六个 tuple。若你的 Python 低于 3.10 不支持 strict，这也是为什么环境要求 3.11/3.12。

逐项读：index 0 `Ali/B-PER`；index 3 `Universiti/B-ORG`；index 4 `Malaya/I-ORG`。标点是 O。

7. Lesson 03.2：实现 `validate_bio`

首先定义允许类型：

```
ALLOWED_TYPES = {"PER", "LOC", "ORG"}
```

Set 适合 membership 检查。然后创建函数，先检查长度，再用 `previous_type` 记录前一个实体类型。遇到 O 时清空；遇到 B/I 时用 `split('-',1)` 分开 prefix/type。

关键判断：

```
if prefix == "I" and previous_type != entity_type:
    raise ValueError(f"illegal I-tag at {index}: {label}")
```

Function parameters 都是 `list[str]`；正常返回 `None`，代表“验证通过无需产生新数据”；异常输入 raise ValueError。调用者是 `bio_to_entities`，后者必须先验证再处理。

局部测试：

```
python -c "from learning.bio_demo import validate_bio; validate_bio(['Ali'], ['B-PER']); print('PASS')"
```

错误测试：

```
python -c "from learning.bio_demo import validate_bio; validate_bio(['Malaysia'], ['I-LOC'])"
```

第二条实际应以 `ValueError: illegal I-tag at 0: I-LOC` 失败。失败是正确行为。

8. Lesson 03.3：把 BIO 合并为 entity

函数签名：

```
def bio_to_entities(tokens: list[str], labels: list[str]) -> list[dict[str, str | int]]:
```

输入来自手工列表；正式项目中 labels 来自模型。返回 list，每个 dict 包含 text/type/start_token/end_token。内部 `start` 初始 None，代表没有打开实体。

循环使用：

```
for index, label in enumerate(labels + ["O"]):
```

最后额外 O 是 sentinel。如果最后一个 token 属于实体，sentinel 会触发保存；它不写回原 labels。遇到 B 或 O 时先关闭旧实体，再决定是否打开新实体。

文字由 `" ".join(tokens[start:index])` 合并。end_token 是 exclusive，所以 tokens[start:end] 正好得到实体。

9. Lesson 03.4：实际运行与检查输出

创建 main，使用上一节例子并打印。执行：

```
python learning\bio_demo.py
```

真实输出结构：

```
[('Ali', 'B-PER'), ('studies', 'O'), ('at', 'O'), ('Universiti', 'B-ORG'), ('Malaya', 'I-ORG'), ('.',
'O')]
[{'text': 'Ali', 'type': 'PER', 'start_token': 0, 'end_token': 1}, {'text': 'Universiti Malaya', 'type':
'ORG', 'start_token': 3, 'end_token': 5}]
```

检查 `tokens[3:5]` 是两个机构 token。若 entity 没有保存，检查 sentinel 与循环缩进。

10. 完整代码回顾

打开 `learning/bio_demo.py`，确认顺序是 module docstring -> allowed types -> validate -> bio_to_entities -> main -> main guard。不要把 validation 删除后只保留 happy path；错误数据必须尽早失败。

11. 跨文件调用关系

`main()` 准备 tokens/labels -> 调用 `bio_to_entities()` -> 该函数先调用 `validate_bio()` -> 验证通过后合并 -> entities 返回 main -> print。Phase 06 的 `src/preprocessing.py` 会采用同样约束；Phase 15 的 metrics 会把 BIO 转 exact spans。

12. 数据流解释

输入原句概念上是 str，但本阶段手工提供 `list[str]`。labels 是等长 list[str]。validate 不改变数据。bio_to_entities 用 integer index 跟踪边界，返回 `list[dict]`。此时 span 是 token 位置，不是原始字符位置；字符 span 在 Phase 13 实现。

13. 常见错误

1. token/label 长度不同：先逐项 zip 检查。 2. `I-ORG` 跟在 `B-PER`：实体类型不连续，应改 B-ORG。 3. 忘记最后 sentinel：句尾实体丢失。 4. end 当 inclusive：Python slice 少/多一个 token；统一使用 exclusive。 5. 将 LOC 与 ORG 混淆：依据标注 guideline，而不是凭大小写。 6. 把 token span 当 character span：它们单位不同。

14. Checkpoint

执行：

```
python -m py_compile learning\bio_demo.py
python learning\bio_demo.py
python -c "from learning.bio_demo import bio_to_entities; print(bio_to_entities(['Kuala','Lumpur'],['B-
LOC','I-LOC']))"
```

通过条件：编译成功；主例得到两个实体；最后命令得到一个 text=`Kuala Lumpur`、type=`LOC`、span `[0,2)` 的实体。再运行非法 I 测试，必须抛 ValueError。

15. 学生练习

1. 标注 `Ahmad studies at Universiti Malaya in Kuala Lumpur.`。 2. 构造包含两个相邻 PER 的合法 BIO。 3. 故意制造长度不等并解释错误。 4. 给 validate 增加未知标签 `B-MISC` 的失败测试。 5. 解释为什么 O 会关闭当前实体。

16. 面试准备

"NER is a sequence-labeling task. Each token receives a BIO tag. B starts an entity, I continues the same entity type, and O means outside. I validate gold BIO sequences before training and use half-open token spans to reconstruct entities."

17. 下一阶段预告

Phase 04 会创建正式项目骨架、`src/constants.py`、`.gitignore` 和测试入口。手工 BIO 概念随后在 Phase 05/06 用于真实 JSONL 数据生成、校验和 vocabulary。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/bio_demo.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """Phase 03: 不使用机器学习，手工理解和验证 BIO 标注。"""
0002 |
0003 | ALLOWED_TYPES = {"PER", "LOC", "ORG"}
0004 |
0005 |
0006 | def validate_bio(tokens: list[str], labels: list[str]) -> None:
0007 |     """验证 token/label 长度、标签类型及 I-tag 的前置关系。"""
0008 |     if len(tokens) != len(labels):
0009 |         raise ValueError("tokens and labels must have the same length")
0010 |     previous_type: str | None = None
0011 |     for index, label in enumerate(labels):
0012 |         if label == "O":
0013 |             previous_type = None
0014 |             continue
0015 |         if "-" not in label:
0016 |             raise ValueError(f"invalid label at {index}: {label}")
0017 |         prefix, entity_type = label.split("-", 1)
0018 |         if prefix not in {"B", "I"} or entity_type not in ALLOWED_TYPES:
0019 |             raise ValueError(f"invalid label at {index}: {label}")
0020 |         if prefix == "I" and previous_type != entity_type:
0021 |             raise ValueError(f"illegal I-tag at {index}: {label}")
0022 |         previous_type = entity_type
0023 |
0024 |
0025 | def bio_to_entities(tokens: list[str], labels: list[str]) -> list[dict[str, str | int]]:
0026 |     """把合法 BIO 序列合并成 token 半开区间实体。"""
0027 |     validate_bio(tokens, labels)
0028 |     entities: list[dict[str, str | int]] = []
0029 |     start: int | None = None
0030 |     entity_type: str | None = None
0031 |     for index, label in enumerate(labels + ["O"]):
0032 |         if label.startswith("B-") or label == "O":
0033 |             if start is not None and entity_type is not None:
0034 |                 entities.append(
0035 |                     {
0036 |                         "text": " ".join(tokens[start:index]),
0037 |                         "type": entity_type,
0038 |                         "start_token": start,
0039 |                         "end_token": index,
0040 |                     }
0041 |                 )
0042 |             if label.startswith("B-"):
```

源码第 43-57 行

```
0043 |                 start, entity_type = index, label[2:]
0044 |             else:
```

```
                start, entity_type = None, None
    return entities


def main() -> None:
    tokens = ["Ali", "studies", "at", "Universiti", "Malaya", "."]
    labels = ["B-PER", "O", "O", "B-ORG", "I-ORG", "O"]
    print(list(zip(tokens, labels, strict=True)))
    print(bio_to_entities(tokens, labels))


if __name__ == "__main__":
    main()
```

立刻运行

```
python -m py_compile learning/bio_demo.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 04：创建项目骨架、配置与 Git

1. 本阶段学习目标

把前三阶段的练习目录升级为正式 Python 项目：理解 package、module、relative import、path、configuration、random seed，并创建 `src/constants.py`、`src/__init__.py`、`tests/`、`data/`、`models/`、`reports/` 与 `.gitignore`。结束时 constants 可从任意工作目录 import，路径仍指向项目根目录。

2. 本阶段需要掌握的知识

文件夹与 Python package 的差别；`__init__.py`；绝对/相对路径；`Path(__file__)`；constant；seed；Git repository、working tree、ignore；为什么 `.venv`、cache 与临时 QA 文件不应提交。

3. 从零解释知识点

Module 是一个 `.py` 文件；package 是可 import 的模块目录。`src/__init__.py` 即使为空，也明确告诉读者这是 package。`__file__` 是当前文件实际路径；`Path(__file__).resolve().parents[1]` 从 `src/constants.py` 向上得到项目根目录，因此不依赖 PowerShell 当前在哪。

Constant 是约定不在运行中修改的配置，使用大写命名。LABELS 顺序会写入 checkpoint；一旦训练后改变顺序，旧权重输出 ID 会被错误解释。SEED 让随机步骤尽量可复现，但不同硬件/库仍可能有轻微差异。

Git 记录源代码演化；`.gitignore` 只排除不该跟踪的路径，不会删除文件。当前仓库没有 commit，所以教材快照明确是本轮重建历史。

4. 本阶段开始前的项目状态

```
learning/                       [已有]
tutorial/chapters/phase_00-03   [已有]
src/                            [本阶段正式启用]
  __init__.py                   [新增]
  constants.py                  [新增]
tests/                          [新增]
data/raw/                       [新增]
models/ reports/                [新增]
.gitignore pyproject.toml       [创建或检查]
```

5. 本阶段需要使用的工具

VS Code Explorer 创建目录/文件；Integrated PowerShell 位于根目录；Git 用只读 `status` 检查。先执行 `Get-Location`，再执行 `git status -sb`。若 Git 报 dubious ownership，只在本仓库命令上使用 `git -c safe.directory='D:/AIROST/Intern-MachineLearning' status -sb`，不要全局信任陌生路径。

6. Lesson 04.1：创建 package 与目录

在 Explorer 新建 `src`，其中新建空 `__init__.py`。再创建 `tests`、`data/raw`、`models`、`reports`。PowerShell 等价命令：

```
New-Item -ItemType Directory -Force src,tests,data\raw,models,reports
New-Item -ItemType File -Force src\__init__.py
```

`-Force` 允许目录已存在，不代表删除内容。完成后运行 `python -c "import src; print(src.__name__)"`，预期 `src`。

7. Lesson 04.2：逐行创建 constants

打开 `src/constants.py`，第一行：

```
from pathlib import Path
```

接着输入：

```
ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
PROCESSED_DIR = DATA_DIR / "processed"
MALAYSIAN_DIR = DATA_DIR / "malaysian"
MODELS_DIR = ROOT / "models"
REPORTS_DIR = ROOT / "reports"
```

Path 的 / 是路径拼接，不是数学除法。返回值仍是 Path。然后定义：

```
PAD_TOKEN = "<PAD>"
UNK_TOKEN = "<UNK>"
LABELS = ["<PAD>", "O", "B-PER", "I-PER", "B-LOC", "I-LOC", "B-ORG", "I-ORG"]
SEED = 42
```

PAD label 只用于 batch 补齐，不是数据集真实标签。UNK 代表词表未见 token。

8. Lesson 04.3：建立 `.gitignore` 与依赖入口

`.gitignore` 至少包含：

```
.venv/
__pycache__/
.pytest_cache/
.pytest-tmp/
*.pyc
```

不要忽略 `src/`、`tests/`、教程章节或小型 checkpoint。`pyproject.toml` 声明 Python 3.11-3.12 与依赖；`uv.lock` 锁具体解析结果。学生此时不需要手写全部 lock file，执行 `uv sync --all-groups` 由工具处理。

9. Lesson 04.4：从不同目录验证路径

根目录运行：

```
python -c "from src.constants import ROOT,LABELS; print(ROOT); print(len(LABELS))"
```

预期根路径与数字 8。再 `Set-Location tests`，使用 `python -c "import sys; sys.path.insert(0,'..'); from src.constants import ROOT; print(ROOT)"`，路径仍相同，随后 `Set-Location ..` 返回。实际正式测试通过 import constants。

10. 完整代码回顾

阶段结束的 `src/constants.py` 应只包含路径、特殊 token、labels 和 seed，不 import torch，也不执行业务逻辑。打开真实文件逐行核对；`python -m py_compile src\constants.py` 应无输出并退出 0。

11. 跨文件调用关系

未来 `generate_data` 读取数据目录；`train/evaluate/predict` 读取 models/reports；Dataset/metrics 使用 LABELS。所有文件 import constants，constants 不反向 import 它们，避免 circular import。

12. 数据流解释

本阶段没有训练数据流。配置流是 `constants.py` Path/list/int -> 其他 modules import -> 创建/读取统一位置。LABELS list 的 index 以后成为 label ID。

13. 常见错误

`parents[0]` 得到 src 而非 root；在字符串中手写反斜线可能出现转义；把 PAD 当 gold label；训练后重排 LABELS；误提交 `.venv`；在错误目录运行 Git。逐项用 print、status 和测试定位。

14. Checkpoint

```
python -m py_compile src\constants.py
python -c "from src.constants import ROOT,LABELS,PAD_TOKEN; assert ROOT.name=='Intern-MachineLearning';
assert len(LABELS)==8; assert PAD_TOKEN=='<PAD>'; print('PASS')"
git -c safe.directory='D:/AIROST/Intern-MachineLearning' status -sb
```

真实验收：constants import 与三项断言 PASS。Git 当前显示未提交文件是事实，不宣称已有公开仓库。

15. 学生练习

打印每个 label 与 index；解释为什么 PAD 是 0；新增 `DATA_DIR` 后验证类型是 Path；用错误 parent 创建临时变量观察差异；检查 `.gitignore` 是否忽略 `.venv`。

16. 面试准备

"I centralize stable paths, labels and the random seed in one dependency-free module. Paths are derived from `__file__`, so commands do not depend on the current directory. Label order is checkpoint metadata and must remain stable."

17. 下一阶段预告

Phase 05 使用 constants 中的数据目录创建一般与 Malaysian JSONL，并在写文件前验证 BIO、实体池隔离和跨 split 完整句泄漏。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：pyproject.toml

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 toml 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-34 行

```
0001 | [project]
0002 | name = "personal-information-ner"
0003 | version = "0.1.0"
0004 | description = "A from-scratch PyTorch BiLSTM NER system for student personal information det
.... | >ection and redaction."
0005 | requires-python = ">=3.11,<3.13"
0006 | dependencies = [
0007 |   "fastapi==0.120.1",
0008 |   "jinja2==3.1.6",
0009 |   "numpy==2.3.4",
0010 |   "pandas==2.3.3",
0011 |   "pydantic==2.12.3",
0012 |   "pypdf==6.1.3",
0013 |   "python-multipart==0.0.20",
0014 |   "scikit-learn==1.7.2",
0015 |   "torch==2.9.0",
0016 |   "uvicorn==0.38.0",
0017 | ]
0018 |
0019 | [dependency-groups]
0020 | dev = [
0021 |   "httpx==0.28.1",
0022 |   "pdfplumber==0.11.7",
0023 |   "pytest==8.4.2",
0024 |   "pytest-cov==7.0.0",
0025 |   "reportlab==4.4.4",
0026 | ]
0027 |
0028 | [tool.pytest.ini_options]
0029 | testpaths = ["tests"]
0030 | pythonpath = ["."]
0031 | addopts = "-ra --basetemp=.pytest-tmp"
0032 |
0033 | [tool.uv]
0034 | package = false
```

立刻运行

```
Get-Content pyproject.toml | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：.gitignore

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 text 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-17 行

```
0001 | .venv/
0002 | __pycache__/
0003 | *.py[cod]
0004 | .pytest_cache/
0005 | .pytest-tmp/
0006 | .coverage
0007 | htmlcov/
0008 | tmp/
0009 | data/raw/*
0010 | !data/raw/.gitkeep
0011 | models/*.pt
0012 | models/*.pth
0013 | !models/best_model.pt
```

```
0014 | output/pdf/*.pdf
0015 | !output/pdf/NER_From_Zero_To_Complete_Project_CN.pdf
0016 | output/pdf/qa-*/
0017 | output/pdf/rendered/
```

立刻运行

```
Get-Content .gitignore | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/constants.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-17 行

```
0001 | from pathlib import Path
0002 |
0003 | # 所有路径都由当前文件位置推导，避免依赖用户从哪个目录启动 Python。
0004 | ROOT = Path(__file__).resolve().parents[1]
0005 | DATA_DIR = ROOT / "data"
0006 | PROCESSED_DIR = DATA_DIR / "processed"
0007 | MALAYSIAN_DIR = DATA_DIR / "malaysian"
0008 | MODELS_DIR = ROOT / "models"
0009 | REPORTS_DIR = ROOT / "reports"
0010 |
0011 | # `<PAD>` 用于补齐 Batch；`<UNK>` 代表训练词表未见词。
0012 | PAD_TOKEN = "<PAD>"
0013 | UNK_TOKEN = "<UNK>"
0014 | # Label ID 的顺序会写入 checkpoint；训练和推理必须完全一致。
0015 | LABELS = ["<PAD>", "O", "B-PER", "I-PER", "B-LOC", "I-LOC", "B-ORG", "I-ORG"]
0016 | PAD_LABEL = "<PAD>"
0017 | SEED = 42
```

立刻运行

```
python -m py_compile src/constants.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 05：建立并验证一般与 Malaysian 数据集

1. 本阶段学习目标

理解 supervised dataset、sample、split、JSONL、synthetic data、data leakage；使用 `src/generate_data.py` 真实生成 2400/300/300 一般数据和 160/40/60 Malaysian 数据。任何完整句跨 split 重复时命令必须失败。

2. 本阶段需要掌握的知识

Training/validation/test 的不同职责；template 与 entity pool；随机数 seed；JSON 与 JSONL；人工规则标注；为什么合成数据不等同真实用户数据；为什么实体不重复仍不能消除 template distribution bias。

3. 从零解释知识点

每个 sample 是 dict：`text:str`、`tokens:list[str]`、`labels:list[str]`。JSONL 每行一个独立 JSON object，便于流式读取和定位坏行。Training 更新 weights；validation 选 epoch；test 只在冻结后评估。

合成数据通过 template 插入已知实体，因此可准确生成 label，但语言多样性有限。我们分开 train/validation/test entity pools，并检查规范化完整句交集；仍共享 grammar templates，所以 1.0 F1 会高估真实世界。Malaysia 名称池包含公开机构/地点和虚构姓名，不包含私人资料。

4. 本阶段开始前的项目状态

```
src/constants.py       [已有]
src/generate_data.py   [本阶段新增]
data/processed/        [运行后生成]
data/malaysian/        [运行后生成]
tests/                 [本阶段增加数据检查]
```

5. 本阶段需要使用的工具

VS Code 打开 generator；PowerShell 根目录运行 module；Explorer 可抽查 JSONL，但不要用 Excel 自动改编码。使用 `python -m src.generate_data` 而非双击文件，保证 package import 正确。

6. Lesson 05.1：设计 sample 与 `_annotate`

先手写一个 sample：

```
{"tokens": ["Ali", "works", "at", "Maybank", "."],
 "labels": ["B-PER", "O", "O", "B-ORG", "O"]}
```

`_annotate(rendered, values)` 先 `split()` 得 token，再建立等长 O list，寻找实体 token slice，首项写 B，后续写 I，最后调用 `validate_bio`。Parameters 是 rendered str 与 type->value dict；return 是含 tokens/labels/text 的 dict；调用者是 split generator。

7. Lesson 05.2：独立 entity pools 与 deterministic random

定义 `ENTITY_POOLS['train'|'validation'|'test']`。同一个人名不跨 pool。`random.Random(seed)` 创建局部 RNG，不污染全局。每条样本选择 template 与 pool value；相同 seed 应产生相同文件，便于复现。

运行小测试：

```
python -c "from src.generate_data import generate_split; a=generate_split('test',3,42); b=generate_split(
'test',3,42); assert a==b; print('PASS')"
```

8. Lesson 05.3：Malaysian 独立 split

MALAYSIAN_POOLS 同样分三组。generate_malaysian_split 使用 local templates，输出 160 train、40 validation、60 test。测试集绝不在 train.py 的 training rows 中。规则标注仍需人工抽样；真实扩展应双人标注与解决分歧。

不能声称“人工真实数据”：这里是规则生成合成句。README、报告和 PDF 都必须用相同措辞。

9. Lesson 05.4：写 JSONL 与 leakage fail-fast

_write_jsonl 使用 UTF-8，每个 row json.dumps(...,ensure_ascii=False) 后换行。main 对每 split 建 lowercase text set，与先前 sets 求交；只要 overlap 非空就 raise RuntimeError，不静默删除。

执行：

```
python -m src.generate_data
Get-Content data\processed\summary.json
```

真实输出 rows：2400/300/300 与 Malaysian 160/40/60；unique sentences 少于 rows 是 split 内模板组合重复，不是跨 split 泄漏。跨 split 交集已检查为零。

10. 完整代码回顾

打开 src/generate_data.py：imports -> pools/templates -> annotate -> general/local generators -> writer -> main。确认 main 只写数据，不 import model/train。最终文件较长，应按函数阅读而非一次记忆。

11. 跨文件调用关系

generator import constants paths 和 preprocessing.validate_bio；main 调 generate -> annotate -> validate -> write。后续 dataset.load_jsonl 读取结果；train 只读 train/validation；evaluate 只读 test。

12. 数据流解释

template str + entity values -> rendered str -> tokens list -> labels list -> sample dict -> JSON string -> .jsonl bytes。Summary 将 rows/unique counts 写 JSON。没有 Tensor，数据仍是 Python/磁盘层。

13. 常见错误

实体 substring 重叠导致错误边界；PER2 未单独标注；同一 neutral sentence 跨 split；test pool 被 training import；默认 ASCII 使中文转义；忘记换行；看到高分却忽略模板共享。使用断言、交集和人工抽查修复。

14. Checkpoint

```
python -m src.generate_data
python -c "from src.dataset import load_jsonl; from src.constants import PROCESSED_DIR,MALAYSIAN_DIR;
assert len(load_jsonl(PROCESSED_DIR/'train.jsonl'))==2400; assert len(load_jsonl(MALAYSIAN_DIR/'test.
jsonl'))==60; print('PASS')"
```

再检查每 row tokens/labels 等长、标签合法、train/test text set disjoint。实际生成命令已退出 0。

15. 学生练习

新增一个不含实体 template；人工抽查 20 行；统计三类实体数量；故意复制一句到 test 验证 fail-fast；解释 unique<rows 为什么不必然是跨 split leakage。

16. 面试准备

"I generated transparent synthetic BIO data with separate entity pools and exact-sentence leakage checks. Malaysian train, validation and test are separate. The splits still share templates, so I explicitly

limit claims about real-world generalization."

17. 下一阶段预告

Phase 06 正式实现 tokenizer、字符 span、BIO validator、vocabulary、UNK 与 casing features，并用独立测试处理异常输入。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/preprocessing.py

为什么需要它：提供 tokenizer、字符位置、BIO 校验和 Vocabulary。dataset.py 与 predict.py 都依赖它，因此必须先完成并单测。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """文本分词、词表构建、BIO 标签校验与 Token ID 转换。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 | import re
0007 | from collections import Counter
0008 | from pathlib import Path
0009 | from typing import Iterable
0010 |
0011 | from .constants import LABELS, PAD_TOKEN, UNK_TOKEN
0012 |
0013 | TOKEN_PATTERN = re.compile(r"[A-Za-z]+(?:[-'][A-Za-z]+)*|\d+(?:[.,]\d+)*|[^\w\s]", re.UNICOD
.... | >E)
0014 |
0015 |
0016 | def tokenize(text: str) -> list[str]:
0017 |     """按原顺序拆分英文单词、数字和标点，返回 Token 字符串列表。"""
0018 |     return TOKEN_PATTERN.findall(text)
0019 |
0020 |
0021 | def tokenize_with_spans(text: str) -> tuple[list[str], list[tuple[int, int]]]:
0022 |     """分词并返回每个 Token 在原始字符串中的字符区间 ``[start, end)``。"""
0023 |     matches = list(TOKEN_PATTERN.finditer(text))
0024 |     return [match.group(0) for match in matches], [match.span() for match in matches]
0025 |
0026 |
0027 | def normalize_token(token: str) -> str:
0028 |     """仅为查词表统一成小写；显示与大小写特征仍使用原始 Token。"""
0029 |     return token.lower()
0030 |
0031 |
0032 | def validate_bio(tokens: list[str], labels: list[str]) -> None:
0033 |     """检查 Token/Label 等长、标签合法，以及 I 标签必须跟在同类型实体后。"""
0034 |     if len(tokens) != len(labels):
0035 |         raise ValueError(f"Token/label length mismatch: {len(tokens)} != {len(labels)}")
0036 |     allowed = set(LABELS) - {"<PAD>"}
0037 |     previous_type: str | None = None
0038 |     for index, label in enumerate(labels):
0039 |         if label not in allowed:
0040 |             raise ValueError(f"Unknown label at {index}: {label}")
0041 |         if label == "O":
```

源码第 42-80 行

```
0042 |             previous_type = None
0043 |             continue
0044 |         prefix, entity_type = label.split("-", 1)
0045 |         if prefix == "I" and previous_type != entity_type:
0046 |             raise ValueError(f"Illegal I-tag at {index}: {label}")
0047 |         previous_type = entity_type
0048 |
0049 |
0050 | def build_vocabulary(sentences: Iterable[list[str]], min_frequency: int = 1) -> dict[str, in
.... | >t]:
0051 |     """从训练句子建立 token -> integer ID 的映射。"""
0052 |     counts = Counter(normalize_token(token) for sentence in sentences for token in sentence)
0053 |     vocabulary = {PAD_TOKEN: 0, UNK_TOKEN: 1}
0054 |     for token in sorted(counts):
0055 |         if counts[token] >= min_frequency:
0056 |             vocabulary[token] = len(vocabulary)
0057 |     return vocabulary
0058 |
0059 |
0060 | def tokens_to_ids(tokens: list[str], vocabulary: dict[str, int]) -> list[int]:
0061 |     """把 Token 转为 ID；未见过的词统一使用 <UNK> 的 ID。"""
0062 |     unknown_id = vocabulary[UNK_TOKEN]
0063 |     return [vocabulary.get(normalize_token(token), unknown_id) for token in tokens]
0064 |
0065 |
0066 | def casing_features(tokens: list[str]) -> list[list[float]]:
0067 |     """为每个 Token 返回 [首字母大写, 全大写, 全小写, 含数字] 四个特征。"""
0068 |     return [
0069 |         [float(t.istitle()), float(t.isupper()), float(t.islower()), float(any(c.isdigit() f
.... | >or c in t))]
0070 |         for t in tokens
0071 |     ]
0072 |
0073 |
0074 | def save_json(data: object, path: Path) -> None:
0075 |     path.parent.mkdir(parents=True, exist_ok=True)
0076 |     path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
0077 |
0078 |
0079 | def load_json(path: Path) -> object:
0080 |     return json.loads(path.read_text(encoding="utf-8"))
```

立刻运行

```
python -m py_compile src/preprocessing.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/generate_data.py

为什么需要它：生成一般与 Malaysian 三分数据，验证 BIO 边界、重复句和模板泄漏。main() 由训练前的数据准备命令调用，输出 JSONL 给 dataset.py。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-32 行

```
0001 | """生成来源透明的合成 NER 语料，并检查 Train/Validation/Test 句子泄漏。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
```

```
0006 | import random
0007 | from pathlib import Path
0008 |
0009 | from .constants import MALAYSIAN_DIR, PROCESSED_DIR, SEED
0010 | from .preprocessing import tokenize, validate_bio
0011 |
0012 | ENTITY_POOLS = {
0013 |     "train": {
0014 |         "PER": ["Aisha Rahman", "Daniel Wong", "Nur Izzati", "Michael Chen", "Priya Nair", "
.... | >James Brown", "Emma Wilson", "Hafiz Omar", "Mei Lin", "Ravi Kumar", "Fatimah Zahra", "Lucas
.... | >Martin", "Aminah", "Ravi", "Chloe", "Wei"],
0015 |         "LOC": ["Penang", "Paris", "Tokyo", "Selangor", "Berlin", "Singapore", "Perak", "Syd
.... | >ney", "Sabah", "Dublin", "Manchester", "Putrajaya", "Kota Kinabalu", "Los Angeles", "Hong Ko
.... | >ng", "George Town", "New Delhi", "Bandar Seri Begawan"],
0016 |         "ORG": ["University of Manchester", "Universiti Malaya", "National University of Sin
.... | >gapore", "Monash University", "University of Sydney", "University of Tokyo", "University of
.... | >Berlin", "Universiti Putra Malaysia", "Dublin City University", "University of Delhi", "Manc
.... | >hester College", "Singapore Student Council"],
0017 |     },
0018 |     "validation": {
0019 |         "PER": ["Siti Aminah", "Oliver Smith", "Tan Wei Ming", "Sophia Garcia"],
0020 |         "LOC": ["Kuala Lumpur", "London", "Johor Bahru", "Madrid"],
0021 |         "ORG": ["Universiti Kebangsaan Malaysia", "University of Oxford", "Madrid Internatio
.... | >nal School", "London Student Union"],
0022 |     },
0023 |     "test": {
0024 |         "PER": ["John Carter", "Ali Hassan", "Sarah Lee", "Ahmad Zaki", "Siti Hajar", "Grace
.... | > Taylor"],
0025 |         "LOC": ["Edinburgh", "Bangkok", "Melbourne", "Malacca", "New York", "Kuching"],
0026 |         "ORG": ["Universiti Teknologi Malaysia", "University of Edinburgh", "Bangkok Interna
.... | >tional School", "Sunway University", "New York University", "Kuching High School"],
0027 |     },
0028 | }
0029 |
0030 | TEMPLATES = [
0031 |     "{PER} is a student at {ORG} .",
0032 |     "{PER} studies at {ORG} in {LOC} .",
```

源码第 33-66 行

```
0033 |     "{PER} reported an incident at {ORG} in {LOC} .",
0034 |     "{PER} submitted a complaint to {ORG} .",
0035 |     "The student support team contacted {PER} at {ORG} .",
0036 |     "{PER} attended a student activity in {LOC} .",
0037 |     "{ORG} organised a student event in {LOC} .",
0038 |     "{PER} travelled from {LOC} to study at {ORG} .",
0039 |     "The university report says {PER} lives in {LOC} .",
0040 |     "{PER} and {PER2} joined the club at {ORG} .",
0041 |     "{PER} met {PER2} at {ORG} in {LOC} .",
0042 |     "{PER} requested academic support from {ORG} .",
0043 |     "{PER} submitted feedback about a course at {ORG} .",
0044 |     "A student named {PER} visited {LOC} .",
0045 |     "The incident involving {PER} happened near {ORG} in {LOC} .",
0046 |     "The administrator at {ORG} emailed {PER} .",
0047 |     "{PER} arrived early for class .",
0048 |     "The student conference took place in {LOC} .",
0049 | ]
0050 | NO_ENTITY = {
0051 |     "train": [
0052 |         "Please review the attached student report .", "The support request was completed su
.... | >ccessfully .",
0053 |         "The lecture starts at nine tomorrow .", "The assignment deadline is next week .",
0054 |         "This feedback contains general course information .", "A student activity will happ
.... | >en tomorrow .",
0055 |         "The portal processed the request correctly .", "Students submitted the assignment o
.... | >n time .",
0056 |         "The classroom is quiet and clean .", "No private details appear in this message .",
0057 |     ],
0058 |     "validation": ["A meeting will begin tomorrow morning .", "No private details appear her
.... | >e ."],
0059 |     "test": ["The weather is beautiful today .", "This ordinary sentence contains no entity
.... | >."],
0060 | }
0061 |
0062 | MALAYSIAN_POOLS = {
0063 |     "train": {
0064 |         "PER": ["Ahmad Firdaus", "Nurul Aisyah", "Lim Wei Jian", "Arjun Nair", "Siti Khadija
.... | >h", "Tan Jia Hui", "Muhammad Haziq", "Devi Krishnan"],
```

```
        "LOC": ["Shah Alam", "Ipoh", "Kuantan", "Seremban", "Alor Setar", "Kota Bharu", "Mir
>i", "Cyberjaya"],
        "ORG": ["Universiti Malaya", "Universiti Putra Malaysia", "Universiti Teknologi MARA
>", "Sunway College", "Taman Melawati Secondary School", "Aminuddin Baki Secondary School", "
```

源码第 66-106 行

```
>Malaysian Student Council", "Ministry of Higher Education"],
    },
    "validation": {
        "PER": ["Farah Nabila", "Lee Jun Hao"],
        "LOC": ["Labuan", "Petaling Jaya"],
        "ORG": ["Universiti Kebangsaan Malaysia", "Victoria Secondary School"],
    },
    "test": {
        "PER": ["Aina Sofea", "Kumar Raj", "Jason Teoh"],
        "LOC": ["Sandakan", "Batu Pahat", "Kuching"],
        "ORG": ["Universiti Sains Malaysia", "Universiti Teknologi Malaysia", "International
> Islamic University Malaysia"],
    },
}


def _annotate(rendered: str, values: dict[str, str]) -> dict[str, list[str]]:
    """根据已知合成实体位置创建 BIO 标签；仅用于生成训练数据。"""
    tokens = rendered.split()
    labels = ["O"] * len(tokens)
    for entity_type, value in values.items():
        entity_tokens = value.split()
        for start in range(len(tokens) - len(entity_tokens) + 1):
            if tokens[start : start + len(entity_tokens)] == entity_tokens:
                labels[start] = f"B-{entity_type}"
                for offset in range(1, len(entity_tokens)):
                    labels[start + offset] = f"I-{entity_type}"
                break
    validate_bio(tokens, labels)
    return {"tokens": tokens, "labels": labels, "text": " ".join(tokens)}


def generate_split(split: str, size: int, seed: int) -> list[dict[str, list[str]]]:
    """使用指定 split 的独立实体池生成固定数量样本。"""
    rng = random.Random(seed)
    pool = ENTITY_POOLS[split]
    rows: list[dict[str, list[str]]] = []
    for index in range(size):
        if index % 6 == 0:
            text = rng.choice(NO_ENTITY[split])
            rows.append(_annotate(text, {}))
            continue
```

源码第 107-143 行

```
        template = rng.choice(TEMPLATES)
        values = {kind: rng.choice(items) for kind, items in pool.items() if "{" + kind + "}
>" in template}
        annotations = dict(values)
        if "{PER2}" in template:
            values["PER2"] = rng.choice([name for name in pool["PER"] if name != values.get(
>"PER")])
            annotations["PER_SECOND"] = values["PER2"]
        rendered = template.format(**values)
        annotation_types = {key: ("PER" if key == "PER_SECOND" else key) for key in annotati
>ons}
        typed_values = {annotation_types[key]: value for key, value in annotations.items() i
>f key != "PER_SECOND"}
        row = _annotate(rendered, typed_values)
        if "PER_SECOND" in annotations:
            second = annotations["PER_SECOND"].split()
            for start in range(len(row["tokens"]) - len(second) + 1):
                if row["tokens"][start:start + len(second)] == second:
                    row["labels"][start] = "B-PER"
                    for offset in range(1, len(second)):
                        row["labels"][start + offset] = "I-PER"
                    break
            validate_bio(row["tokens"], row["labels"])
        rows.append(row)
    rng.shuffle(rows)
    return rows
```

```
0129 |
0130 |
0131 | def generate_malaysian_split(split: str, size: int, seed: int) -> list[dict[str, list[str]]]
.... | >:
0132 |     """生成实体池互斥的马来西亚合成语料；规则标注后仍需人工抽查。"""
0133 |     rng = random.Random(seed)
0134 |     pool = MALAYSIAN_POOLS[split]
0135 |     local_templates = [template for template in TEMPLATES if "{PER2}" not in template]
0136 |     rows = []
0137 |     for index in range(size):
0138 |         if index % 8 == 0:
0139 |             neutral = {
0140 |                 "train": "The submitted form contains no named entity .",
0141 |                 "validation": "This local validation sentence has no entity .",
0142 |                 "test": "This local test sentence has no entity .",
0143 |             }[split]
```

源码第 144-178 行

```
0144 |             rows.append(_annotate(neutral, {}))
0145 |             continue
0146 |         template = rng.choice(local_templates)
0147 |         values = {kind: rng.choice(items) for kind, items in pool.items() if "{" + kind + "}
.... | >" in template}
0148 |         rows.append(_annotate(template.format(**values), values))
0149 |     rng.shuffle(rows)
0150 |     return rows
0151 |
0152 |
0153 | def _write_jsonl(rows: list[dict], path: Path) -> None:
0154 |     """将样本保存为一行一个 JSON 对象的 UTF-8 JSONL。"""
0155 |     path.parent.mkdir(parents=True, exist_ok=True)
0156 |     with path.open("w", encoding="utf-8") as stream:
0157 |         for row in rows:
0158 |             stream.write(json.dumps(row, ensure_ascii=False) + "\n")
0159 |
0160 |
0161 | def main() -> None:
0162 |     """生成三个 Split；发现规范化句子跨 Split 重复时立即失败。"""
0163 |     # 完整句子和实体字符串都必须跨 split 隔离，避免验证/测试指标因记忆
0164 |     # 训练集专有名词而虚高。
0165 |     for entity_type in ("PER", "LOC", "ORG"):
0166 |         split_sets = {split: set(values[entity_type]) for split, values in ENTITY_POOLS.item
.... | >s()}
0167 |         for left, right in (("train", "validation"), ("train", "test"), ("validation", "test
.... | >")):
0168 |             overlap = split_sets[left] & split_sets[right]
0169 |             if overlap:
0170 |                 raise RuntimeError(f"{entity_type} entity leakage between {left}/{right}: {s
.... | >orted(overlap)}")
0171 |         combined = {
0172 |             split: set(ENTITY_POOLS[split][entity_type]) | set(MALAYSIAN_POOLS[split][entity
.... | >_type])
0173 |             for split in ("train", "validation", "test")
0174 |         }
0175 |         for left, right in (("train", "validation"), ("train", "test"), ("validation", "test
.... | >")):
0176 |             overlap = combined[left] & combined[right]
0177 |             if overlap:
0178 |                 raise RuntimeError(f"Combined {entity_type} leakage between {left}/{right}:
.... | >{sorted(overlap)}")
```

源码第 179-217 行

```
0179 |
0180 |     sizes = {"train": 2400, "validation": 300, "test": 300}
0181 |     all_texts: set[str] = set()
0182 |     summary = {}
0183 |     for offset, (split, size) in enumerate(sizes.items()):
0184 |         rows = generate_split(split, size, SEED + offset)
0185 |         texts = {row["text"].lower() for row in rows}
0186 |         overlap = all_texts & texts
0187 |         if overlap:
0188 |             # 绝不自动忽略泄漏；失败能迫使开发者先修复数据再训练。
0189 |             raise RuntimeError(f"Sentence leakage into {split}: {sorted(overlap)[:3]}")
0190 |         all_texts |= texts
0191 |         _write_jsonl(rows, PROCESSED_DIR / f"{split}.jsonl")
0192 |         summary[split] = {"rows": len(rows), "unique_sentences": len(texts)}
```

```
    challenge_texts = [
        "Ahmad Ali is a student at Universiti Teknologi Malaysia in Johor Bahru.",
        "Kumar submitted a complaint to Universiti Sains Malaysia in George Town.",
        "Aina and Jason joined a student activity at International Islamic University Malaysia.",
        "The assignment deadline is next week.",
    ]
    MALAYSIAN_DIR.mkdir(parents=True, exist_ok=True)
    (MALAYSIAN_DIR / "challenge.txt").write_text("\n".join(challenge_texts) + "\n", encoding="utf-8")
    local_sizes = {"train": 160, "validation": 40, "test": 60}
    local_texts: set[str] = set()
    for offset, (split, size) in enumerate(local_sizes.items()):
        rows = generate_malaysian_split(split, size, SEED + 100 + offset)
        texts = {row["text"].lower() for row in rows}
        if local_texts & texts:
            raise RuntimeError(f"Malaysian sentence leakage into {split}")
        local_texts |= texts
        _write_jsonl(rows, MALAYSIAN_DIR / f"{split}.jsonl")
        summary[f"malaysian_{split}"] = {"rows": len(rows), "unique_sentences": len(texts)}
    (PROCESSED_DIR / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

立刻运行

```
python -m py_compile src/generate_data.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 06：Tokenization、BIO 校验与 Vocabulary

1. Learning Objectives / 学习目标

把原始英文句子稳定转换成 token、字符位置、BIO 标签和整数 ID。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

正则分词、半开区间 [start,end)、PAD/UNK、训练集建词表、标签合法性。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

generate_data.py -> preprocessing.py -> dataset.py；predict.py 复用同一 tokenizer，避免训练/推理不一致。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/preprocessing.py    [本阶段创建或完整核对]
tests/test_preprocessing.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/preprocessing.py、tests/test_preprocessing.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/preprocessing.py

为什么需要它：提供 tokenizer、字符位置、BIO 校验和 Vocabulary。dataset.py 与 predict.py 都依赖它，因此必须先完成并单测。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """文本分词、词表构建、BIO 标签校验与 Token ID 转换。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 | import re
0007 | from collections import Counter
0008 | from pathlib import Path
0009 | from typing import Iterable
0010 |
0011 | from .constants import LABELS, PAD_TOKEN, UNK_TOKEN
0012 |
0013 | TOKEN_PATTERN = re.compile(r"[A-Za-z]+(?:[-'][A-Za-z]+)*|\d+(?:[.,]\d+)*|[^\w\s]", re.UNICOD
.... | >E)
0014 |
0015 |
0016 | def tokenize(text: str) -> list[str]:
0017 |     """按原顺序拆分英文单词、数字和标点，返回 Token 字符串列表。"""
0018 |     return TOKEN_PATTERN.findall(text)
0019 |
0020 |
0021 | def tokenize_with_spans(text: str) -> tuple[list[str], list[tuple[int, int]]]:
0022 |     """分词并返回每个 Token 在原始字符串中的字符区间 ``[start, end)``。"""
0023 |     matches = list(TOKEN_PATTERN.finditer(text))
0024 |     return [match.group(0) for match in matches], [match.span() for match in matches]
0025 |
0026 |
0027 | def normalize_token(token: str) -> str:
0028 |     """仅为查词表统一成小写；显示与大小写特征仍使用原始 Token。"""
0029 |     return token.lower()
0030 |
0031 |
0032 | def validate_bio(tokens: list[str], labels: list[str]) -> None:
0033 |     """检查 Token/Label 等长、标签合法，以及 I 标签必须跟在同类型实体后。"""
0034 |     if len(tokens) != len(labels):
0035 |         raise ValueError(f"Token/label length mismatch: {len(tokens)} != {len(labels)}")
0036 |     allowed = set(LABELS) - {"<PAD>"}
0037 |     previous_type: str | None = None
0038 |     for index, label in enumerate(labels):
0039 |         if label not in allowed:
```

```
0040 |             raise ValueError(f"Unknown label at {index}: {label}")
0041 |         if label == "O":
```

源码第 42-80 行

```
0042 |             previous_type = None
0043 |             continue
0044 |         prefix, entity_type = label.split("-", 1)
0045 |         if prefix == "I" and previous_type != entity_type:
0046 |             raise ValueError(f"Illegal I-tag at {index}: {label}")
0047 |         previous_type = entity_type
0048 |
0049 |
0050 | def build_vocabulary(sentences: Iterable[list[str]], min_frequency: int = 1) -> dict[str, in
.... | >t]:
0051 |     """从训练句子建立 token -> integer ID 的映射。"""
0052 |     counts = Counter(normalize_token(token) for sentence in sentences for token in sentence)
0053 |     vocabulary = {PAD_TOKEN: 0, UNK_TOKEN: 1}
0054 |     for token in sorted(counts):
0055 |         if counts[token] >= min_frequency:
0056 |             vocabulary[token] = len(vocabulary)
0057 |     return vocabulary
0058 |
0059 |
0060 | def tokens_to_ids(tokens: list[str], vocabulary: dict[str, int]) -> list[int]:
0061 |     """把 Token 转为 ID；未见过的词统一使用 <UNK> 的 ID。"""
0062 |     unknown_id = vocabulary[UNK_TOKEN]
0063 |     return [vocabulary.get(normalize_token(token), unknown_id) for token in tokens]
0064 |
0065 |
0066 | def casing_features(tokens: list[str]) -> list[list[float]]:
0067 |     """为每个 Token 返回 [首字母大写, 全大写, 全小写, 含数字] 四个特征。"""
0068 |     return [
0069 |         [float(t.istitle()), float(t.isupper()), float(t.islower()), float(any(c.isdigit() f
.... | >or c in t))]
0070 |         for t in tokens
0071 |     ]
0072 |
0073 |
0074 | def save_json(data: object, path: Path) -> None:
0075 |     path.parent.mkdir(parents=True, exist_ok=True)
0076 |     path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
0077 |
0078 |
0079 | def load_json(path: Path) -> object:
0080 |     return json.loads(path.read_text(encoding="utf-8"))
```

立刻运行

```
python -m py_compile src/preprocessing.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_preprocessing.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | import pytest
0002 |
```

```
0003 | from src.generate_data import ENTITY_POOLS, MALAYSIAN_POOLS
0004 | from src.preprocessing import build_vocabulary, casing_features, tokenize, tokens_to_ids, va
.... | >lidate_bio
0005 |
0006 |
0007 | def test_tokenization_preserves_punctuation_and_names():
0008 |     assert tokenize("Ali works at UTM, Johor.") == ["Ali", "works", "at", "UTM", ",", "Johor
.... | >", "."]
0009 |
0010 |
0011 | def test_vocabulary_and_unknown_word():
0012 |     vocabulary = build_vocabulary([["Ali", "works"]])
0013 |     assert tokens_to_ids(["ALI", "unseen"], vocabulary) == [vocabulary["ali"], vocabulary["<
.... | >UNK>"]]
0014 |
0015 |
0016 | def test_casing_feature_shape():
0017 |     features = casing_features(["Ali", "UTM", "works", "2026"])
0018 |     assert len(features) == 4
0019 |     assert all(len(row) == 4 for row in features)
0020 |
0021 |
0022 | def test_invalid_bio_is_rejected():
0023 |     with pytest.raises(ValueError, match="Illegal I-tag"):
0024 |         validate_bio(["Ali"], ["I-PER"])
0025 |
0026 |
0027 | @pytest.mark.parametrize("pools", [ENTITY_POOLS, MALAYSIAN_POOLS])
0028 | def test_entity_pools_are_disjoint_across_splits(pools):
0029 |     for entity_type in ("PER", "LOC", "ORG"):
0030 |         split_sets = [set(pools[split][entity_type]) for split in ("train", "validation", "t
.... | >est")]
0031 |         assert split_sets[0].isdisjoint(split_sets[1])
0032 |         assert split_sets[0].isdisjoint(split_sets[2])
0033 |         assert split_sets[1].isdisjoint(split_sets[2])
0034 |
0035 |
0036 | def test_combined_general_and_malaysian_entities_do_not_cross_splits():
0037 |     for entity_type in ("PER", "LOC", "ORG"):
0038 |         combined = {
```

源码第 39-44 行

```
0039 |             split: set(ENTITY_POOLS[split][entity_type]) | set(MALAYSIAN_POOLS[split][entity
.... | >_type])
0040 |             for split in ("train", "validation", "test")
0041 |         }
0042 |         assert combined["train"].isdisjoint(combined["validation"])
0043 |         assert combined["train"].isdisjoint(combined["test"])
0044 |         assert combined["validation"].isdisjoint(combined["test"])
```

立刻运行

```
python -m py_compile tests/test_preprocessing.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

generate_data.py -> preprocessing.py -> dataset.py；predict.py 复用同一 tokenizer，避免训练/推理不一致。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\preprocessing.py
python -m pytest tests\test_preprocessing.py -q
```

10. Expected Results / 预期结果

3 项 preprocessing 测试通过；非法 I-tag 会抛出 ValueError。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

把验证/测试词加入词表造成泄漏；训练和预测使用不同分词；忘记 UNK；span 采用闭区间。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

I build the vocabulary from training data only and reuse exactly the same tokenizer during inference.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 07：Tensor、Token ID、特征与 Padding

1. Learning Objectives / 学习目标

把不同长度的 token 序列变成模型可接收的 LongTensor、FloatTensor、label tensor 与 mask。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

Tensor、dtype、shape、padding、mask、casing features。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

preprocessing.py 产生 ID/特征；dataset.py 的 __getitem__ 返回单句；collate_batch 拼成 batch。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
learning/tensor_demo.py    [本阶段创建或完整核对]
src/preprocessing.py    [本阶段创建或完整核对]
src/dataset.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：learning/tensor_demo.py、src/preprocessing.py、src/dataset.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/tensor_demo.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-18 行

```
0001 | """Phase 07: 用三个 token 独立观察 Tensor dtype 与 shape。"""
0002 |
0003 | import torch
0004 |
```

```
0005 |
0006 | def main() -> None:
0007 |     input_ids = torch.tensor([2, 5, 1], dtype=torch.long)
0008 |     features = torch.zeros((3, 4), dtype=torch.float32)
0009 |     labels = torch.tensor([2, 3, 0], dtype=torch.long)
0010 |     print("input_ids:", input_ids.dtype, tuple(input_ids.shape))
0011 |     print("features:", features.dtype, tuple(features.shape))
0012 |     print("labels:", labels.dtype, tuple(labels.shape))
0013 |     assert input_ids.dtype == labels.dtype == torch.long
0014 |     assert features.shape == (3, 4)
0015 |
0016 |
0017 | if __name__ == "__main__":
0018 |     main()
```

立刻运行

```
python -m py_compile learning/tensor_demo.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/preprocessing.py

为什么需要它：提供 tokenizer、字符位置、BIO 校验和 Vocabulary。dataset.py 与 predict.py 都依赖它，因此必须先完成并单测。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """文本分词、词表构建、BIO 标签校验与 Token ID 转换。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 | import re
0007 | from collections import Counter
0008 | from pathlib import Path
0009 | from typing import Iterable
0010 |
0011 | from .constants import LABELS, PAD_TOKEN, UNK_TOKEN
0012 |
0013 | TOKEN_PATTERN = re.compile(r"[A-Za-z]+(?:[-'][A-Za-z]+)*|\d+(?:[.,]\d+)*|[^\w\s]", re.UNICOD
.... | >E)
0014 |
0015 |
0016 | def tokenize(text: str) -> list[str]:
0017 |     """按原顺序拆分英文单词、数字和标点，返回 Token 字符串列表。"""
0018 |     return TOKEN_PATTERN.findall(text)
0019 |
0020 |
0021 | def tokenize_with_spans(text: str) -> tuple[list[str], list[tuple[int, int]]]:
0022 |     """分词并返回每个 Token 在原始字符串中的字符区间 ``[start, end)``。"""
0023 |     matches = list(TOKEN_PATTERN.finditer(text))
0024 |     return [match.group(0) for match in matches], [match.span() for match in matches]
0025 |
0026 |
0027 | def normalize_token(token: str) -> str:
0028 |     """仅为查词表统一成小写；显示与大小写特征仍使用原始 Token。"""
0029 |     return token.lower()
0030 |
0031 |
0032 | def validate_bio(tokens: list[str], labels: list[str]) -> None:
0033 |     """检查 Token/Label 等长、标签合法，以及 I 标签必须跟在同类型实体后。"""
```

```
0034 |     if len(tokens) != len(labels):
0035 |         raise ValueError(f"Token/label length mismatch: {len(tokens)} != {len(labels)}")
0036 |     allowed = set(LABELS) - {"<PAD>"}
0037 |     previous_type: str | None = None
0038 |     for index, label in enumerate(labels):
0039 |         if label not in allowed:
0040 |             raise ValueError(f"Unknown label at {index}: {label}")
0041 |         if label == "O":
```

源码第 42-80 行

```
0042 |             previous_type = None
0043 |             continue
0044 |         prefix, entity_type = label.split("-", 1)
0045 |         if prefix == "I" and previous_type != entity_type:
0046 |             raise ValueError(f"Illegal I-tag at {index}: {label}")
0047 |         previous_type = entity_type
0048 |
0049 |
0050 | def build_vocabulary(sentences: Iterable[list[str]], min_frequency: int = 1) -> dict[str, in
.... | >t]:
0051 |     """从训练句子建立 token -> integer ID 的映射。"""
0052 |     counts = Counter(normalize_token(token) for sentence in sentences for token in sentence)
0053 |     vocabulary = {PAD_TOKEN: 0, UNK_TOKEN: 1}
0054 |     for token in sorted(counts):
0055 |         if counts[token] >= min_frequency:
0056 |             vocabulary[token] = len(vocabulary)
0057 |     return vocabulary
0058 |
0059 |
0060 | def tokens_to_ids(tokens: list[str], vocabulary: dict[str, int]) -> list[int]:
0061 |     """把 Token 转为 ID；未见过的词统一使用 <UNK> 的 ID。"""
0062 |     unknown_id = vocabulary[UNK_TOKEN]
0063 |     return [vocabulary.get(normalize_token(token), unknown_id) for token in tokens]
0064 |
0065 |
0066 | def casing_features(tokens: list[str]) -> list[list[float]]:
0067 |     """为每个 Token 返回 [首字母大写, 全大写, 全小写, 含数字] 四个特征。"""
0068 |     return [
0069 |         [float(t.istitle()), float(t.isupper()), float(t.islower()), float(any(c.isdigit() f
.... | >or c in t))]
0070 |         for t in tokens
0071 |     ]
0072 |
0073 |
0074 | def save_json(data: object, path: Path) -> None:
0075 |     path.parent.mkdir(parents=True, exist_ok=True)
0076 |     path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
0077 |
0078 |
0079 | def load_json(path: Path) -> object:
0080 |     return json.loads(path.read_text(encoding="utf-8"))
```

立刻运行

```
python -m py_compile src/preprocessing.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/dataset.py

为什么需要它：把 token/label 转为 Tensor，并在 collate_fn 中 padding、生成 mask。train.py 的 DataLoader 直接消费它的返回值。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保

存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
"""把 JSONL 样本转换为 PyTorch Tensor，并把变长句子补齐为 Batch。"""

from __future__ import annotations

import json
from pathlib import Path

import torch
from torch.utils.data import Dataset

from .constants import LABELS
from .preprocessing import casing_features, tokens_to_ids


def load_jsonl(path: Path) -> list[dict]:
    """逐行读取 JSONL；每行代表一个已标注句子。"""
    with path.open(encoding="utf-8") as stream:
        return [json.loads(line) for line in stream if line.strip()]


class NERDataset(Dataset):
    """让 DataLoader 可以按索引取得单条 NER 样本。"""
    def __init__(self, rows: list[dict], vocabulary: dict[str, int], label_to_id: dict[str,
>int], entity_word_dropout: float = 0.0, general_word_dropout: float = 0.0):
        self.rows = rows
        self.vocabulary = vocabulary
        self.label_to_id = label_to_id
        self.entity_word_dropout = entity_word_dropout
        self.general_word_dropout = general_word_dropout

    def __len__(self) -> int:
        return len(self.rows)

    def __getitem__(self, index: int) -> dict:
        # input_ids Shape：(当前句子长度,)。训练时的 word dropout
        # 会把部分词替换成 <UNK>，迫使模型学习上下文而不是背记实体名称。
        row = self.rows[index]
        input_ids = torch.tensor(tokens_to_ids(row["tokens"], self.vocabulary), dtype=torch.
>long)
        if self.entity_word_dropout:
            for position, label in enumerate(row["labels"]):
                probability = self.entity_word_dropout if label != "O" else self.general_wor
```

源码第 40-69 行

```
>d_dropout
                if row["tokens"][position].isalnum() and torch.rand(()) < probability:
                    input_ids[position] = self.vocabulary["<UNK>"]
        return {
            "tokens": row["tokens"],
            "input_ids": input_ids,
            "features": torch.tensor(casing_features(row["tokens"]), dtype=torch.float32),
            "labels": torch.tensor([self.label_to_id[label] for label in row["labels"]], dty
>pe=torch.long),
        }


def collate_batch(batch: list[dict]) -> dict:
    """把不同长度句子 Padding 到本 Batch 的最大长度。"""
    # 最终 input_ids/labels/mask Shape：(batch_size, max_length)
    # features Shape：(batch_size, max_length, 4)
    batch_size = len(batch)
    max_length = max(len(item["input_ids"]) for item in batch)
    input_ids = torch.zeros((batch_size, max_length), dtype=torch.long)
    features = torch.zeros((batch_size, max_length, 4), dtype=torch.float32)
    labels = torch.zeros((batch_size, max_length), dtype=torch.long)
    mask = torch.zeros((batch_size, max_length), dtype=torch.bool)
    tokens: list[list[str]] = []
    for row_index, item in enumerate(batch):
        length = len(item["input_ids"])
        input_ids[row_index, :length] = item["input_ids"]
        features[row_index, :length] = item["features"]
        labels[row_index, :length] = item["labels"]
```

```
0067 |             mask[row_index, :length] = True
0068 |             tokens.append(item["tokens"])
0069 |     return {"tokens": tokens, "input_ids": input_ids, "features": features, "labels"
.... | >, "mask": mask}
```

立刻运行

```
python -m py_compile src/dataset.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

preprocessing.py 产生 ID/特征；dataset.py 的 __getitem__ 返回单句；collate_batch 拼成 batch。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\dataset.py
python -m pytest tests\test_acceptance.py -q -k vocabulary
```

10. Expected Results / 预期结果

词表与 Tensor 路径测试通过；input_ids/labels 为 long，features 为 float32。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

把 token ID 做成 float；label padding 没设 -100；mask 与句长不一致；把 batch 与 sequence 维写反。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

Padding makes a rectangular batch, while the mask tells later code which positions are real tokens.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 08：PyTorch Dataset、DataLoader 与 Batch

1. Learning Objectives / 学习目标

实现可重复迭代的数据集、训练期 word dropout 和可变长度 batch。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

Dataset、DataLoader、__len__、__getitem__、collate_fn、shuffle。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

train.py 创建 NERDataset，再把 collate_batch 交给 DataLoader；model.py 接收 batch 中的 input_ids/features。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
learning/dataloader_demo.py    [本阶段创建或完整核对]
src/dataset.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：learning/dataloader_demo.py、src/dataset.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/dataloader_demo.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-31 行

```
0001 | """Phase 08: 不依赖正式模型，验证变长句子如何 padding 成 batch。"""
0002 |
0003 | import torch
0004 |
0005 | from src.dataset import collate_batch
0006 |
0007 |
0008 | def main() -> None:
0009 |     batch = [
0010 |         {
0011 |             "tokens": ["Ahmad", "studies"],
0012 |             "input_ids": torch.tensor([2, 3]),
0013 |             "features": torch.zeros((2, 4)),
0014 |             "labels": torch.tensor([2, 0]),
0015 |         },
0016 |         {
0017 |             "tokens": ["Hello"],
0018 |             "input_ids": torch.tensor([4]),
0019 |             "features": torch.zeros((1, 4)),
0020 |             "labels": torch.tensor([0]),
0021 |         },
0022 |     ]
0023 |     result = collate_batch(batch)
0024 |     for name in ("input_ids", "features", "labels", "mask"):
0025 |         print(name, tuple(result[name].shape), result[name].dtype)
0026 |     assert result["input_ids"].tolist() == [[2, 3], [4, 0]]
0027 |     assert result["mask"].tolist() == [[True, True], [True, False]]
0028 |
0029 |
0030 | if __name__ == "__main__":
0031 |     main()
```

立刻运行

```
python -m py_compile learning/dataloader_demo.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/dataset.py

为什么需要它：把 token/label 转为 Tensor，并在 collate_fn 中 padding、生成 mask。train.py 的 DataLoader 直接消费它的返回值。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | """把 JSONL 样本转换为 PyTorch Tensor，并把变长句子补齐为 Batch。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 | from pathlib import Path
0007 |
0008 | import torch
0009 | from torch.utils.data import Dataset
```

```
0010 |
0011 | from .constants import LABELS
0012 | from .preprocessing import casing_features, tokens_to_ids
0013 |
0014 |
0015 | def load_jsonl(path: Path) -> list[dict]:
0016 |     """逐行读取 JSONL；每行代表一个已标注句子。"""
0017 |     with path.open(encoding="utf-8") as stream:
0018 |         return [json.loads(line) for line in stream if line.strip()]
0019 |
0020 |
0021 | class NERDataset(Dataset):
0022 |     """让 DataLoader 可以按索引取得单条 NER 样本。"""
0023 |     def __init__(self, rows: list[dict], vocabulary: dict[str, int], label_to_id: dict[str,
.... | >int], entity_word_dropout: float = 0.0, general_word_dropout: float = 0.0):
0024 |         self.rows = rows
0025 |         self.vocabulary = vocabulary
0026 |         self.label_to_id = label_to_id
0027 |         self.entity_word_dropout = entity_word_dropout
0028 |         self.general_word_dropout = general_word_dropout
0029 |
0030 |     def __len__(self) -> int:
0031 |         return len(self.rows)
0032 |
0033 |     def __getitem__(self, index: int) -> dict:
0034 |         # input_ids Shape: (当前句子长度,)。训练时的 word dropout
0035 |         # 会把部分词替换成 <UNK>，迫使模型学习上下文而不是背记实体名称。
0036 |         row = self.rows[index]
0037 |         input_ids = torch.tensor(tokens_to_ids(row["tokens"], self.vocabulary), dtype=torch.
.... | >long)
0038 |         if self.entity_word_dropout:
0039 |             for position, label in enumerate(row["labels"]):
0040 |                 probability = self.entity_word_dropout if label != "O" else self.general_wor
```

源码第 40-69 行

```
.... | >d_dropout
0041 |                 if row["tokens"][position].isalnum() and torch.rand(()) < probability:
0042 |                     input_ids[position] = self.vocabulary["<UNK>"]
0043 |         return {
0044 |             "tokens": row["tokens"],
0045 |             "input_ids": input_ids,
0046 |             "features": torch.tensor(casing_features(row["tokens"]), dtype=torch.float32),
0047 |             "labels": torch.tensor([self.label_to_id[label] for label in row["labels"]], dty
.... | >pe=torch.long),
0048 |         }
0049 |
0050 |
0051 | def collate_batch(batch: list[dict]) -> dict:
0052 |     """把不同长度句子 Padding 到本 Batch 的最大长度。"""
0053 |     # 最终 input_ids/labels/mask Shape: (batch_size, max_length)
0054 |     # features Shape: (batch_size, max_length, 4)
0055 |     batch_size = len(batch)
0056 |     max_length = max(len(item["input_ids"]) for item in batch)
0057 |     input_ids = torch.zeros((batch_size, max_length), dtype=torch.long)
0058 |     features = torch.zeros((batch_size, max_length, 4), dtype=torch.float32)
0059 |     labels = torch.zeros((batch_size, max_length), dtype=torch.long)
0060 |     mask = torch.zeros((batch_size, max_length), dtype=torch.bool)
0061 |     tokens: list[list[str]] = []
0062 |     for row_index, item in enumerate(batch):
0063 |         length = len(item["input_ids"])
0064 |         input_ids[row_index, :length] = item["input_ids"]
0065 |         features[row_index, :length] = item["features"]
0066 |         labels[row_index, :length] = item["labels"]
0067 |         mask[row_index, :length] = True
0068 |         tokens.append(item["tokens"])
0069 |     return {"tokens": tokens, "input_ids": input_ids, "features": features, "labels": labels
.... | >, "mask": mask}
```

立刻运行

```
python -m py_compile src/dataset.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

train.py 创建 NERDataset，再把 collate_batch 交给 DataLoader；model.py 接收 batch 中的 input_ids/features。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\dataset.py
python -m pytest tests\test_acceptance.py -q -k dataset
```

10. Expected Results / 预期结果

dataset loading 与 padding 测试通过；一个 batch 的四个 Tensor 第一维相同。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

在 validation/test 使用 dropout；collate 返回 key 与 train.py 不一致；空 batch；在 Dataset 内移动到 GPU。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

Dataset prepares one example, and collate_fn pads several examples into one batch.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 09：Neural Network、Embedding、LSTM 与完整 BiLSTM

1. 本阶段学习目标

这一阶段从一个没有层、甚至不能 forward 的 Python class 开始，逐步完成六个可运行版本。结束时你要能独立创建模型、解释每一层、追踪每个 Tensor shape、执行 forward、计算参数数量，并说明为什么 `model(input)` 会调用 `forward()`。

我们不会直接展示最终 `src/model.py`。可运行中间文件位于 `tutorial/model_versions/model_v1.py` 到 `model_v6.py`；对应测试在 `tests/test_model_versions.py`。每次只增加一个概念，前一版本通过后才能继续。

2. 本阶段需要掌握的知识

Python：class、object、inheritance、`self`、constructor、`super()`、method。数学：输入、weight、bias、矩阵映射。PyTorch：Tensor、dtype、shape、`nn.Module`、parameter、Embedding、LSTM、bidirectional、Linear、Dropout、ReLU、concatenate、forward、train/eval、gradient。

具体 shape 统一用示例：batch=2、sequence=5、embedding=64、feature projection=8、hidden=96、num_labels=8。最终变化为 `(2,5) -> (2,5,64)`；features `(2,5,4) -> (2,5,8)`；拼接 `(2,5,72)`；BiLSTM `(2,5,192)`；Linear `(2,5,8)`。

3. 从零解释知识点

Neural Network 是带可训练数字的函数组合。输入经过计算得到输出；Loss 衡量输出与答案差距；backpropagation 计算每个 parameter 对 loss 的影响；optimizer 后续才更新 parameter。本阶段只搭建 forward，不训练，避免一次引入过多概念。

Layer 是一类可复用变换。Linear 做 `y=xW+b`；Embedding 按整数 ID 查一行向量；LSTM 逐时间步处理序列并维护状态。Parameter 是训练会改变的 Tensor，普通局部变量不是 parameter。PyTorch Module 会自动收集子 Module 的 parameter，支持 `.to(device)`、`.train()`、`.eval()`、保存与加载。

4. 本阶段开始前的项目状态

```
tutorial/model_versions/
  model_v1.py  [本阶段创建]
  model_v2.py  [本阶段创建]
  model_v3.py  [本阶段创建]
  model_v4.py  [本阶段创建]
  model_v5.py  [本阶段创建]
  model_v6.py  [本阶段创建，等价于最终架构]
tests/test_model_versions.py [本阶段创建]
src/model.py                 [最终正式文件，阶段结束后同步]
```

Phase 08 已提供 `input_ids` shape `(batch,length)`、features `(batch,length,4)`、mask `(batch,length)`。本阶段只消费 input_ids/features；mask 在 loss/metrics 阶段使用。

5. 本阶段需要使用的工具

VS Code 同时打开版本文件与测试。PowerShell 位于项目根目录，激活 `.venv`。每个版本可单独执行 pytest，例如：

```
python -m pytest tests\test_model_versions.py::test_version_2_embedding_shape -q
```

失败时在 VS Code 测试函数左侧设置 breakpoint，查看 tensor 的 `.shape` 和 `.dtype`。

6. Lesson 1：什么是 Neural Network?

先不用 PyTorch。设输入 x=3，weight=2，bias=1，则输出 3*2+1=7。如果正确答案是 10，模型需要调整 weight/bias。神经网络不是“会思考的黑盒”，而是许多可微计算和 parameter 的组合。

操作练习：在 Python terminal 输入 x=3; weight=2; bias=1; print(x*weight+bias)，应输出 7。这段代码不是训练，只帮助区分 input 与 parameter。NER 的 input 是 token IDs；parameter 包括 embedding 表、LSTM weights 与 classifier weights；output 是每 token 的 label logits。

7. Lesson 2：什么是 Layer?

Layer 接收 Tensor 并返回 Tensor。多个 layer 像流水线，但每步 shape 可能变化。Linear layer 对最后一维做矩阵映射。例如输入最后一维 192，输出类别 8，则 nn.Linear(192,8) 的 weight shape 是 (8,192)、bias shape (8,)。

执行：

```
python -c "import torch; from torch import nn; layer=nn.Linear(3,2); x=torch.ones(4,3); print(layer(x).
shape)"
```

实际预期 torch.Size([4, 2])。第一维 4 保留，最后维 3 变 2。

8. Lesson 3：什么是 Parameter?

Parameter 是 requires_grad=True 且被 Module 注册的 Tensor。执行：

```
python -c "from torch import nn; m=nn.Linear(3,2); [(print(n,p.shape,p.requires_grad)) for n,p in m.
named_parameters()]"
```

会看到 weight/bias。普通 self.number=3 不会出现在 named_parameters。未来 optimizer 只更新注册 parameter。参数量计算是每个 shape 维度相乘后求和；不是源代码行数。

9. Lesson 4：什么是 PyTorch Module?

nn.Module 是所有 PyTorch layer/model 的基类。它实现参数登记、子模块树、设备移动、train/eval mode、state_dict、hook 与 __call__。我们组合已有 layer 时，自己的模型也应继承 Module。

不要直接写一个普通函数持有全局 weights；那会让保存、加载、optimizer 和 device 管理困难。Module 不会自动决定网络结构，结构仍由你在 constructor 和 forward 中明确编写。

10. Lesson 5：Python Class、Inheritance、self 与 super

语法 class BiLSTMNER(nn.Module)：定义子类。BiLSTMNER 是我们命名的模型蓝图；括号里的 nn.Module 是父类。创建 model=BiLSTMNER() 得到 object。self 指这个 object。

__init__ 是 constructor。第一行必须 super().__init__()，让父类建立 _parameters、_modules 等内部容器。若先赋 self.embedding=nn.Embedding(...) 而不初始化父类，PyTorch 会报错或无法登记层。super() 不是创建第二个模型，而是从当前继承链访问父类实现。

11. Lesson 6：Version 1 - 创建第一个模型类

打开 tutorial/model_versions/model_v1.py，逐行输入：

```
from torch import nn

class BiLSTMNER(nn.Module):
    def __init__(self) -> None:
        super().__init__()
```

此版本能实例化，但没有 forward。运行：

```
python -m pytest tests\test_model_versions.py::test_version_1_instantiates_module -q
```

真实结果 1 passed。局部测试只验证 inheritance/constructor。若调用 model(torch.tensor(...))，应得到缺少 forward 的 NotImplementedError；这不是 bug，而是 Version 1 的已知边界。

12. Lesson 7：理解 Embedding

Vocabulary 把词映射整数，例如 <PAD>=0，<UNK>=1，sarah=2。整数大小没有语义；ID 2 不代表比 ID 1 更大。Embedding 是 shape (vocab_size,embedding_dim) 的可训练查找表。

输入 IDs (2,5) 表示两句、每句五个位置。查表后 (2,5,64)：每个整数变成 64 个浮点数。padding_idx=0 让 padding row 在训练中不更新。Embedding 随机初始化，训练才让向量适合 NER；本项目没有加载预训练词向量。

13. Lesson 8：Version 2 - 添加 Embedding

复制 V1 为 model_v2.py，修改 constructor 接收 vocab_size 和 embedding_dim，加入：

```
self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
```

为局部测试暂时加入 embed()：

```
def embed(self, input_ids: torch.Tensor) -> torch.Tensor:
    return self.embedding(input_ids)
```

执行 Version 2 测试，真实结果通过：输入 [[2,3,0]] shape (1,3)，输出 (1,3,16)。若 dtype 是 float，Embedding 报 Expected Long；ID 必须 torch.long。若 ID>=vocab_size，会 IndexError。

14. Lesson 9：理解 RNN 与 LSTM

普通 feed-forward layer 独立处理 token，无法知道左侧词。RNN 逐位置更新 hidden state：当前位置结合当前输入和上一状态。长序列中梯度可能消失/爆炸。

LSTM 用 input/forget/output gates 控制 cell state。门是 0 到 1 的可学习比例。无需手算所有公式，但必须知道输入 shape (batch,length,input_size)；输出 encoded 保存每个位置 hidden representation；另有最后 hidden/cell state。本项目需要每 token 预测，所以用完整 encoded，不只最后状态。

15. Lesson 10：理解 Bidirectional LSTM

单向 LSTM 只看左侧。NER 中 Bank 后面的词可能决定它是机构名称。Bidirectional 运行一条左到右 LSTM 和一条右到左 LSTM，再拼接同一位置结果。

若 hidden_dim=96 表示每个方向 96，则输出最后维是 192，不是 96。这个乘 2 是常见 classifier shape mismatch 根源。它不是把句长乘 2；batch 和 length 不变。

16. Lesson 11：Version 3 - 添加 BiLSTM

V3 constructor 在 Embedding 后加入：

```
self.lstm = nn.LSTM(
    embedding_dim,
    hidden_dim,
    batch_first=True,
    bidirectional=True,
)
```

batch_first=True 让 shape 是 (batch,length,features)，与 Dataset 一致。局部 encode 先 embedding，再 encoded,_=self.lstm(embedded)。下划线表示我们暂不使用最终 states。

测试用 embedding=16、hidden=12，输入 (1,3)，输出应 (1,3,24)。实际 Version 3 测试通过。若忘记 bidirectional，最后维是 12，测试会立即失败。

17. Lesson 12：理解 Linear Layer

LSTM 输出每 token 192 个上下文特征，但标签空间只有 8：PAD、O、B/I-PER、B/I-LOC、B/I-ORG。`nn.Linear(192,8)` 对每个 token 独立做相同映射，输出 logits。

Logit 是未归一化分数，可以负数且和不为 1。训练 CrossEntropyLoss 内部做 log-softmax，因此 forward 不应先 softmax。推理 argmax 选择最大 logit 的 label ID。

18. Lesson 13：Version 4 - 添加 Linear

由于双向输出是 `hidden_dim*2`，加入：

```
self.classifier = nn.Linear(hidden_dim * 2, num_labels)
```

暂时用 `classify()` 串起 embedding -> LSTM -> classifier。num_labels 来自实际 label list 长度，不能硬编码 7；本项目 checkpoint 包含 PAD 共 8。

Version 4 测试输入 `(1,3)`，输出 `(1,3,8)`。若 Linear 写 hidden_dim 而非乘 2，会报 mat1/mat2 shapes cannot be multiplied；打印 encoded.shape 即可定位。

19. Lesson 14：理解 Dropout

Dropout 在训练时随机把部分特征设 0，减少模型依赖单一神经元；推理时必须关闭随机屏蔽。`model.train()` 开启，`model.eval()` 关闭。eval 不等于 no_grad：eval 控制层行为，no_grad 控制是否记录梯度，两者职责不同。

V4 在 classifier 前加入 `self.dropout=nn.Dropout(0.25)`。测试调用 `model.eval()` 后 `model.dropout.training` 为 False。不要在 validation 忘记 eval，否则指标会随机波动。

20. Lesson 15：Version 5 - 实现 forward()

把临时 `classify` 改成标准：

```
def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
    embedded = self.embedding(input_ids)
    encoded, _ = self.lstm(embedded)
    return self.classifier(self.dropout(encoded))
```

通常调用 `model(input_ids)`，而不是 `model.forward(input_ids)`。Module 的 `__call__` 会执行 hooks、autocast 等机制后调用 forward。参数 input_ids 来自 DataLoader，dtype long，shape `(batch,length)`；返回 float logits `(batch,length,num_labels)`。

Version 5 测试还执行 `logits.sum().backward()`，确认 classifier weight 得到 gradient。真实结果通过。Backward 只计算 gradient，本阶段没有 optimizer 更新。

21. Lesson 16：完整 Tensor Shape Tracking

使用实际配置：IDs `(2,5)` long。Embedding `(2,5,64)` float。额外 casing features 原始 `(2,5,4)` float，projection 后 `(2,5,8)`。`torch.cat(...,dim=-1)` 后 `(2,5,72)`。

LSTM input_size 必须 72，输出 `(2,5,192)`。Dropout shape 不变。Linear 只改最后维，logits `(2,5,8)`。Argmax(dim=-1) 后 `(2,5)` long。Gold labels 同为 `(2,5)`，mask 也是 `(2,5)` bool。

具体元素数：input IDs 10 个整数；embedding 输出 640 个 float；combined 720；encoded 1920；logits 80。Shape 不是任意格式，每一层 constructor 必须与上一层最后维匹配。

22. Lesson 17：Version 6 - 连接真实 casing features

最终模型利用 `[is_title,is_upper,is_lower,has_digit]` 四特征，帮助未见词。加入：

```
self.feature_projection = nn.Linear(4, 8)
```

LSTM input_size 改 embedding_dim+8。forward 新参数 features；先 ReLU projection，再 cat。dim=-1 表示最后维，不应沿 batch 或 sequence 拼接。

Version 6 测试传 IDs (1,3)、features (1,3,4)，得到 logits (1,3,8)。这与正式 src/model.py 架构一致。若 features length 与 IDs 不同，cat 报 size mismatch，应回到 collate 检查。

23. Lesson 18：连接真实 Dataset 并完整 Forward

真实 batch 是 dict：batch['input_ids']、batch['features']、batch['labels']、batch['mask']。调用：

```
logits = model(batch["input_ids"], batch["features"])
assert logits.shape[:2] == batch["input_ids"].shape
assert logits.shape[-1] == len(LABELS)
```

labels 不传给 model；它们给 loss。mask 也不改变 forward，本项目用它排除 padding metrics。完整 forward 测试不需要训练整个 epoch，因此几秒即可定位接口错误。

24. Lesson 19：常见错误与 Debugging

Shape mismatch：逐层 print shape，检查 LSTM input_size 与 bidirectional*2。Wrong dtype：Embedding IDs 必须 long，features float。Invalid vocabulary index：确保所有 ID 小于 vocab_size，UNK 处理未知词。Device mismatch：model 与所有 tensors 都 .to(device)。

Missing forward：V1 故意会发生，V5 后必须解决。Incorrect num_labels：用 checkpoint labels 长度。Features shape 错：最后维必须 4。忘记 super()：层登记失败。训练/推理 mode 错：validation 调 eval。不要用 try/except 隐藏 shape 错误；让测试失败并读取 traceback 最后一行。

25. Lesson 20：阶段验收与参数数量

运行：

```
$env:OMP_NUM_THREADS="1"
$env:MKL_NUM_THREADS="1"
python -m pytest tests\test_model_versions.py -q
python -m pytest tests\test_model.py -q
python -c "from src.model import BiLSTMNER; m=BiLSTMNER(100,8); print(sum(p.numel() for p in m.
parameters()))"
```

本轮 Version 测试真实结果 6 passed in 1.67s。参数数随 vocab_size 变化；最终 checkpoint 使用实际 vocabulary，总计 143,536。学生必须能解释为什么 embedding 参数与 vocab_size 成正比、BiLSTM 输出为何乘 2、Linear 输出为何等于标签数。

10. 完整代码回顾

不要把六版混成一个文件。依次打开 tutorial/model_versions/model_v1.py 到 model_v6.py，使用 VS Code Compare Selected 对比。V1 只有 Module；V2 embedding；V3 LSTM；V4 dropout/classifier；V5 standard forward；V6 features。

阶段最终正式文件是 src/model.py。它与 V6 层结构一致，但包含更完整中文 docstring/shape 注释。任何正式代码修改都必须同步 V6、测试、Phase 09 与快照 hash。

11. 跨文件调用关系

src/dataset.py 产生 batch -> src/train.py 调 model(input_ids,features) -> Module __call__ 调 forward -> logits 传 CrossEntropyLoss。src/predict.py 重建同一 model、load_state_dict、eval/no_grad、argmax。配置来自 checkpoint，标签来自 src/constants.py。

12. 数据流解释

tokens list -> vocabulary integer IDs -> Dataset long Tensor (sentence_length,) -> collate padded (batch,max_length) -> Embedding/context/features -> logits (batch,max_length,8)。模型不会返回实体文字；BIO decoding 在 prediction 层完成。分层职责让训练与 API 共用同一个 model。

13. 常见错误

除 Lesson 19 外，常见教学错误是一次复制 V6 后无法解释每行。解决方法是删除自己的练习副本，从 V1 重建，每版只看一个测试。另一个错误是把 hidden_dim=96 理解为双向总维；本项目定义为每方向，因此总输出 192。

14. Checkpoint

必须满足：六个版本测试全部通过；能画出具体 shapes；能解释 class/object/inheritance/self/super；能说明 parameter 与普通 variable 差异；能运行真实 batch forward；能计算并解释参数量。任何一项不能解释都回到对应 Lesson。

15. 学生练习

1. 将 V2 embedding_dim 改 32，预测输出 shape 后运行验证。 2. 将 V3 bidirectional=False，记录 shape 变化，再恢复。 3. 故意把 V4 classifier 输入写 hidden_dim，阅读错误。 4. 比较 train/eval 下 dropout 两次输出。 5. 用 named_parameters() 列出层名和 shape。

16. 面试准备

"My model maps token IDs to trainable embeddings and concatenates four casing features. A bidirectional LSTM encodes left and right context. Its output has twice the per-direction hidden size. Dropout regularizes training, and a linear layer produces one logit for each BIO label at every token. I built and tested the architecture in six incremental versions."

17. 下一阶段预告

Phase 10 使用 V6/正式 model 的 logits 学习 CrossEntropyLoss、gradient、backpropagation、zero_grad 与 optimizer.step。模型结构不再改变；重点从“如何计算输出”转到“参数如何从标注数据学习”。

补充 A：`BiLSTMNER.__init__` 完整函数教学卡

Function name 严格说是特殊方法 __init__。Purpose 是接收超参数并创建需要长期保存在 object 中的 layer。Parameters：self 是当前 model；vocab_size:int 是训练 vocabulary 项目数；num_labels:int 是标签数；embedding/hidden 是维度；dropout 是 0 到 1 的比例。Return type 标注 None，因为 constructor 初始化 object，不把新值 return 给调用者。

Calling code 在 src/train.py 与 src/predict.py。它内部调用父类 Module constructor，再调用 nn.Embedding、nn.Linear、nn.LSTM、nn.Dropout constructors。Example input 是 BiLSTMNER(vocab_size=100,num_labels=8,embedding_dim=64,hidden_dim=96)；example output 是可调用 Module object，不是 logits。

为什么 layers 放 __init__ 而不是 forward 内？如果每次 forward 新建 layer，每次都会随机生成新 parameter，optimizer 与 checkpoint 无法追踪。把 layer 赋给 self 后，Module 的 __setattr__ 会登记它。检查：

```
python -c "from src.model import BiLSTMNER; m=BiLSTMNER(100,8); print(dict(m.named_children()).keys())"
```

预期包含 embedding、feature_projection、lstm、dropout、classifier。Common errors：vocab_size=0；dropout 超范围；hidden_dim 与 checkpoint 不一致；先创建 layer 后调用 super。

补充 B：`forward` 完整函数教学卡

Function name `forward`。Purpose 是描述一个 batch 从输入到 logits 的确定性计算图。Parameters：`self`；`input_ids:torch.Tensor`，long，shape (B,L)；`features:torch.Tensor`，float32，shape (B,L,4)。Return value 是 float logits；return type Tensor；shape (B,L,C)。

Calling function 通常不是手写 `forward`，而是 Module 的 `__call__`，由 `model(input_ids,features)` 触发。Forward 内调用 embedding、feature projection、ReLU、cat、LSTM、dropout、classifier。它不调用 loss、不执行 backward、不更新 parameter。这种职责分离允许同一 forward 在 training、validation、prediction 中复用。

Example input：

```
input_ids = torch.tensor([[2, 3, 0], [4, 5, 6]], dtype=torch.long)
features = torch.zeros((2, 3, 4), dtype=torch.float32)
logits = model(input_ids, features)
```

B=2、L=3、C=8，output shape (2,3,8)。Logits 的 48 个 float 还不是 BIO string；后续 argmax -> label IDs -> label mapping。Common errors：把 labels 当 features；features 缺最后维；IDs 用 float；batch/length 不同；model 在 GPU 但 tensors 在 CPU。

补充 C：一次完整数值 Shape 演算

假设两个句子 padding 后长度 5：

```
input_ids = [[2,3,4,5,0], [6,7,8,9,10]]
shape=(2,5), dtype=int64
```

Embedding table 假设 vocab=100、dim=64，参数 shape (100,64)，共 6,400 parameter。查表不改变前两维，输出 (2,5,64)，共 640 float。注意 padding 位置仍有向量，只是 padding row 固定/不更新；loss 还需 ignore padding。

四特征 (2,5,4) 共 40 float。Linear(4,8) 对十个 token 分别映射，输出 (2,5,8)。Weight (8,4)=32，bias 8，共 40 parameter。ReLU 将负数置 0，shape 不变。

Cat 沿最后维连接 64+8=72，输出 (2,5,72)。若 `dim=1` 会试图拼 sequence，得到错误语义 (2,10,64/8) 且最后维不等；正确是 `dim=-1`。

BiLSTM 输入 (2,5,72)。每方向 hidden=96；两个方向同位置拼接，输出 (2,5,192)。PyTorch LSTM 还返回 h_n/c_n，shape (2,2,96)：第一维 2 是方向数，第二维 2 是 batch，不要与 sequence length 混淆。本任务不使用最后 states。

Dropout (2,5,192) -> (2,5,192)。Linear(192,8) weight (8,192) 共 1,536，bias 8。输出 (2,5,8) 共 80 logits。argmax(-1) 变 (2,5)。每个位置得到一个 label ID；mask 决定哪些位置是真 token。

补充 D：LSTM 参数为什么比想象多

LSTM 不只有一个 weight。每方向有 input/forget/cell/output 四组门。粗略单层单方向参数包含 `4*hidden*input_size`、`4*hidden*hidden` 和 biases。双向再乘 2。可用 PyTorch 而非手抄公式核对：

```
python -c "from src.model import BiLSTMNER; m=BiLSTMNER(100,8); [(print(n,tuple(p.shape),p.numel())) for
n,p in m.named_parameters()]"
```

你会看到 `weight_ih_l0`、`weight_hh_l0`、bias，以及 `_reverse` 版本。`ih` 连接 input-to-hidden，`hh` 连接 hidden-to-hidden。总参数量依赖 vocab_size，因此教学小模型与最终 checkpoint 数量不同。最终

143,536 来自实际 vocabulary/config，不应硬编码在代码。

补充 E：Version 1 到 Version 6 的逐行变更清单

V1 新增 import nn、class header、constructor、super；可实例化。V2 新增 torch import、两个 constructor parameters、Embedding attribute、embed method；可验证查表。V3 新增 hidden_dim、LSTM attribute、encode method；可验证上下文 shape。V4 新增 num_labels/dropout、Dropout/Linear、classify method；可验证分类维度和 mode。V5 只将临时 classify 变标准 forward；让 Module 调用机制和 gradient 测试成立。V6 新增 feature projection、forward 第二参数、ReLU 与 cat，并修改 LSTM input_size。

为什么不在 V2 就写最终 forward？因为当时输出只是 embedding，不是模型最终语义；命名为 embed 能诚实表达局部功能。为什么 V3 用 encode？它输出 contextual representation。V4 classify 已能返回 logits，但 V5 才教授 PyTorch 标准接口。这种临时 method 在最终版本删除，是正常 incremental development，不是遗留 API。

对比命令：

```
git diff --no-index tutorial\model_versions\model_v3.py tutorial\model_versions\model_v4.py
```

`--no-index` 比较任意两个文件，不要求 Git commit。退出码 1 表示有差异，不等于命令损坏。你应看到 Dropout、Linear 和 num_labels，而不是无关数据代码。

补充 F：五个可复现 Debug Lab

Lab 1 Shape mismatch：在自己的临时副本把 classifier 改 `nn.Linear(hidden_dim, num_labels)`，运行 V4 test，读取 `mat1 and mat2 shapes`。写下实际 encoded 最后维 24 与错误期望 12，再恢复。不要修改正式文件。

Lab 2 Wrong dtype：运行 `model.embed(torch.tensor([[2.0]]))`，观察 Embedding 对 float ID 的错误。修复是 dtype long，不是把 embedding weight 改整数。

Lab 3 Invalid index：vocab_size=20 时输入 ID 20（合法范围 0-19），观察 index out of range。正式修复是 unknown token 映射 `<UNK>=1`，不能扩大 vocab 掩盖预处理错误。

Lab 4 Device mismatch：有 GPU 才执行。把 model `.to('cuda')` 但 input 留 CPU，会报设备不一致。统一 device。CPU 学生只需解释，不伪造 GPU 输出。

Lab 5 Missing forward：实例化 V1 后调用 model，观察 NotImplementedError；对比 V5 成功。它证明 Module class 并不会自动猜测 layer 顺序。

每个 Lab 的目标是阅读 traceback、定位自己文件行号、打印 shape/dtype、修复根因。不得用宽泛 try/except 把错误吞掉，否则训练可能产生错误而不被发现。

补充 G：为什么当前选择 BiLSTM Token Classifier 而不是 CRF

理想架构可在 Linear 后加 CRF，让整个标签序列联合打分并学习转移，例如 I-PER 不应直接跟 B-LOC。当前比赛规范允许在完整 CRF 超出教学/时间范围时使用自行定义 BiLSTM token classifier。我们选择 fallback 的原因是 CPU 训练、零基础复现和调试更直接；核心 NER 仍是自己训练。

代价必须明确：每 token argmax 可能产生非法初始 I-tag；prediction 会把不合法 I 安全视为新实体开始，但这不是 CRF 联合解码。后续改进可以在完全理解当前 loss/mask 后实现 CRF，并增加转移约束测试。不能在没实现 CRF 时把标题写成 BiLSTM-CRF。

补充 H：本阶段掌握证明

学生不看代码，画出五个 box：IDs、Embedding+features、BiLSTM、Dropout、Linear；在每条箭头写 shape。然后打开空文件，在 20 分钟内写 V1/V2，并让两项测试通过。再口头解释 super、parameter、bidirectional*2、logit、train/eval。

最后提交一份错误日志：选择一个 Debug Lab，保留错误信息、原因、修复 diff 和修复后 PASS。只有“能从失败恢复”才能证明不是机械复制。教师检查六项测试和学生解释，而不是只看最终文件存在。

补充 I：逐行阅读最终 forward 的口述练习

第一行 `embedded = self.embedding(input_ids)`：调用已经登记的 Embedding object，输入 long IDs，返回 float vectors。第二行 feature projection：四个手工特征通过可训练 Linear，再用 ReLU 引入非线性。第三行 cat：不改变 batch/sequence，只扩展每个 token 的描述。

第四行 `encoded, _ = self.lstm(combined)`：LSTM 返回两个对象，我们保留所有位置的 contextual states，暂时丢弃最终 hidden/cell。第五行先 dropout 再 classifier：训练时随机正则化，最后为每个 token 产生八个 logit。Return 把 Tensor 交给调用者，不在函数内 print，也不把 argmax 固定进模型，以便 CrossEntropyLoss 使用完整 logits。

学生应分别回答：如果删除 ReLU，shape 是否改变？不会，但数值变换能力不同。如果 cat 改成相加，为什么失败？64 与 8 维无法直接相加，而且会丢失独立特征。如果 forward 返回 argmax，为什么不能训练？离散 argmax 不适合 CrossEntropyLoss 的 logits 输入，也切断有用梯度信息。

补充 J：代码审查清单

检查所有 layer 只在 constructor 创建；super 位于 layer 前；padding_idx 等于 vocabulary PAD；LSTM input_size=embedding_dim+8；bidirectional=True；classifier 输入 hidden_dim*2；forward 参数和 DataLoader keys 一致；没有 softmax；没有外部 NER pipeline；docstring 与实际 shape 一致。

运行 `python -m pytest tests\test_model_versions.py tests\test_model.py -q` 后再进入训练。若只通过 V6 而早期版本失败，教学演化仍不完整；六个版本都是交付的一部分。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：tutorial/model_versions/model_v1.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-8 行

```
0001 | """Version 1: 只有最小 nn.Module class，可以实例化但不能 forward。"""
0002 |
0003 | from torch import nn
0004 |
0005 |
0006 | class BiLSTMNER(nn.Module):
0007 |     def __init__(self) -> None:
0008 |         super().__init__()
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v1.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tutorial/model_versions/model_v2.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-13 行

```
0001 | """Version 2: 在最小 Module 中加入 Embedding。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 |
0006 |
0007 | class BiLSTMNER(nn.Module):
0008 |     def __init__(self, vocab_size: int, embedding_dim: int = 64) -> None:
0009 |         super().__init__()
0010 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0011 |
0012 |     def embed(self, input_ids: torch.Tensor) -> torch.Tensor:
0013 |         return self.embedding(input_ids)
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v2.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tutorial/model_versions/model_v3.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-21 行

```
0001 | """Version 3: Embedding 后加入双向 LSTM。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 |
0006 |
```

```
0007 | class BiLSTMNER(nn.Module):
0008 |     def __init__(self, vocab_size: int, embedding_dim: int = 64, hidden_dim: int = 96) -> No
.... | >ne:
0009 |         super().__init__()
0010 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0011 |         self.lstm = nn.LSTM(
0012 |             embedding_dim,
0013 |             hidden_dim,
0014 |             batch_first=True,
0015 |             bidirectional=True,
0016 |         )
0017 |
0018 |     def encode(self, input_ids: torch.Tensor) -> torch.Tensor:
0019 |         embedded = self.embedding(input_ids)
0020 |         encoded, _ = self.lstm(embedded)
0021 |         return encoded
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v3.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tutorial/model_versions/model_v4.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-30 行

```
0001 | """Version 4: 加入 Dropout 与 Linear classifier。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 |
0006 |
0007 | class BiLSTMNER(nn.Module):
0008 |     def __init__(
0009 |         self,
0010 |         vocab_size: int,
0011 |         num_labels: int,
0012 |         embedding_dim: int = 64,
0013 |         hidden_dim: int = 96,
0014 |         dropout: float = 0.25,
0015 |     ) -> None:
0016 |         super().__init__()
0017 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0018 |         self.lstm = nn.LSTM(
0019 |             embedding_dim,
0020 |             hidden_dim,
0021 |             batch_first=True,
0022 |             bidirectional=True,
0023 |         )
0024 |         self.dropout = nn.Dropout(dropout)
0025 |         self.classifier = nn.Linear(hidden_dim * 2, num_labels)
0026 |
0027 |     def classify(self, input_ids: torch.Tensor) -> torch.Tensor:
0028 |         embedded = self.embedding(input_ids)
0029 |         encoded, _ = self.lstm(embedded)
0030 |         return self.classifier(self.dropout(encoded))
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v4.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tutorial/model_versions/model_v5.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-30 行

```
0001 | """Version 5: 实现标准 forward，支持 model(input_ids)。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 |
0006 |
0007 | class BiLSTMNER(nn.Module):
0008 |     def __init__(
0009 |         self,
0010 |         vocab_size: int,
0011 |         num_labels: int,
0012 |         embedding_dim: int = 64,
0013 |         hidden_dim: int = 96,
0014 |         dropout: float = 0.25,
0015 |     ) -> None:
0016 |         super().__init__()
0017 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0018 |         self.lstm = nn.LSTM(
0019 |             embedding_dim,
0020 |             hidden_dim,
0021 |             batch_first=True,
0022 |             bidirectional=True,
0023 |         )
0024 |         self.dropout = nn.Dropout(dropout)
0025 |         self.classifier = nn.Linear(hidden_dim * 2, num_labels)
0026 |
0027 |     def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
0028 |         embedded = self.embedding(input_ids)
0029 |         encoded, _ = self.lstm(embedded)
0030 |         return self.classifier(self.dropout(encoded))
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v5.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tutorial/model_versions/model_v6.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-33 行

```
0001 | """Version 6: 与最终模型一致，加入四维大小写/数字特征。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 |
0006 |
0007 | class BiLSTMNER(nn.Module):
0008 |     def __init__(
0009 |         self,
0010 |         vocab_size: int,
0011 |         num_labels: int,
0012 |         embedding_dim: int = 64,
0013 |         hidden_dim: int = 96,
0014 |         dropout: float = 0.25,
0015 |     ) -> None:
0016 |         super().__init__()
0017 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0018 |         self.feature_projection = nn.Linear(4, 8)
0019 |         self.lstm = nn.LSTM(
0020 |             embedding_dim + 8,
0021 |             hidden_dim,
0022 |             batch_first=True,
0023 |             bidirectional=True,
0024 |         )
0025 |         self.dropout = nn.Dropout(dropout)
0026 |         self.classifier = nn.Linear(hidden_dim * 2, num_labels)
0027 |
0028 |     def forward(self, input_ids: torch.Tensor, features: torch.Tensor) -> torch.Tensor:
0029 |         embedded = self.embedding(input_ids)
0030 |         projected_features = torch.relu(self.feature_projection(features))
0031 |         combined = torch.cat([embedded, projected_features], dim=-1)
0032 |         encoded, _ = self.lstm(combined)
0033 |         return self.classifier(self.dropout(encoded))
```

立刻运行

```
python -m py_compile tutorial/model_versions/model_v6.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_model_versions.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | import importlib
0002 |
0003 | import torch
0004 |
0005 |
0006 | def load(version: int):
0007 |     return importlib.import_module(f"tutorial.model_versions.model_v{version}")
```

```
0008 |
0009 |
0010 | def test_version_1_instantiates_module():
0011 |     assert isinstance(load(1).BiLSTMNER(), torch.nn.Module)
0012 |
0013 |
0014 | def test_version_2_embedding_shape():
0015 |     model = load(2).BiLSTMNER(vocab_size=20, embedding_dim=16)
0016 |     assert model.embed(torch.tensor([[2, 3, 0]])).shape == (1, 3, 16)
0017 |
0018 |
0019 | def test_version_3_bilstm_shape():
0020 |     model = load(3).BiLSTMNER(vocab_size=20, embedding_dim=16, hidden_dim=12)
0021 |     assert model.encode(torch.tensor([[2, 3, 0]])).shape == (1, 3, 24)
0022 |
0023 |
0024 | def test_version_4_classifier_shape_and_train_eval_dropout():
0025 |     model = load(4).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0026 |     assert model.classify(torch.tensor([[2, 3, 0]])).shape == (1, 3, 8)
0027 |     model.eval()
0028 |     assert not model.dropout.training
0029 |
0030 |
0031 | def test_version_5_standard_forward_and_gradient():
0032 |     model = load(5).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0033 |     logits = model(torch.tensor([[2, 3, 0]]))
0034 |     logits.sum().backward()
0035 |     assert logits.shape == (1, 3, 8)
0036 |     assert model.classifier.weight.grad is not None
0037 |
0038 |
0039 | def test_version_6_features_match_final_shape():
0040 |     model = load(6).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0041 |     logits = model(torch.tensor([[2, 3, 0]]), torch.zeros((1, 3, 4)))
0042 |     assert logits.shape == (1, 3, 8)
```

立刻运行

```
python -m py_compile tests/test_model_versions.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/model.py

为什么需要它：定义正式 BiLSTM Token Classifier。forward 接收 input_ids、features，返回形状 (batch, length, num_labels) 的 logits。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """从零实现 Embedding + BiLSTM + Linear 的 Token 分类模型。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import torch
0006 | from torch import nn
0007 |
0008 |
0009 | class BiLSTMNER(nn.Module):
0010 |     """根据每个 Token 的 ID、大小写特征，预测对应的 BIO 标签分数。"""
0011 |
0012 |     def __init__(self, vocab_size: int, num_labels: int, embedding_dim: int = 64, hidden_dim
.... | >: int = 96, dropout: float = 0.25):
```

```
0013 |         # 必须先初始化 nn.Module，PyTorch 才能登记下面各层的可训练参数。
0014 |         super().__init__()
0015 |         # 输入 Shape：(batch_size, sequence_length)
0016 |         # 输出 Shape：(batch_size, sequence_length, embedding_dim)
0017 |         # padding_idx=0 保证补齐位置的向量不参与正常词语学习。
0018 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0019 |         # 每个 Token 有 4 个大小写/数字特征，先投影到 8 维。
0020 |         self.feature_projection = nn.Linear(4, 8)
0021 |         # BiLSTM 同时读取左到右和右到左的上下文。
0022 |         # 输出最后一维是 hidden_dim * 2，因为两个方向会拼接。
0023 |         self.lstm = nn.LSTM(
0024 |             embedding_dim + 8,
0025 |             hidden_dim,
0026 |             batch_first=True,
0027 |             bidirectional=True,
0028 |             num_layers=1,
0029 |         )
0030 |         # Dropout 只在训练模式随机屏蔽部分特征，降低过拟合。
0031 |         self.dropout = nn.Dropout(dropout)
0032 |         # 把每个 Token 的 192 维上下文表示映射为 num_labels 个 logits。
0033 |         self.classifier = nn.Linear(hidden_dim * 2, num_labels)
0034 |
0035 |     def forward(self, input_ids: torch.Tensor, features: torch.Tensor) -> torch.Tensor:
0036 |         """执行前向传播，返回 Shape 为 (batch, length, num_labels) 的 logits。"""
0037 |         # 将整数 Token ID 查表转换为可训练的稠密向量。
0038 |         embedded = self.embedding(input_ids)
0039 |         # ReLU 把大小写特征变换为非线性表示。
0040 |         projected_features = torch.relu(self.feature_projection(features))
0041 |         # 在最后一维拼接词向量与大小写特征，不改变 batch 和 sequence 维度。
```

源码第 42-46 行

```
0042 |         combined = torch.cat([embedded, projected_features], dim=-1)
0043 |         # encoded 保存每个 Token 已融合双向上下文的 Hidden State。
0044 |         encoded, _ = self.lstm(combined)
0045 |         # 返回未经过 Softmax 的 logits；CrossEntropyLoss 会在内部处理 Softmax。
0046 |         return self.classifier(self.dropout(encoded))
```

立刻运行

```
python -m py_compile src/model.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

Phase 10：Loss、Gradient、Backpropagation 与 Training Step

1. Learning Objectives / 学习目标

让随机初始化模型完成一次真实的 forward、loss、backward 和 optimizer update。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

logits、CrossEntropyLoss、gradient、zero_grad、backward、optimizer.step、gradient clipping。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

dataset batch -> model logits -> masked labels -> loss -> gradients -> Adam parameters。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
learning/training_step_demo.py    [本阶段创建或完整核对]
src/model.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：learning/training_step_demo.py、src/model.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：learning/training_step_demo.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-35 行

```
0001 | """Phase 10: 在四个 token 上完成一次真正的参数更新。"""
0002 |
0003 | import torch
0004 | from torch import nn
0005 | from torch.optim import Adam
0006 |
0007 | from src.model import BiLSTMNER
0008 |
0009 |
0010 | def main() -> None:
0011 |     torch.manual_seed(42)
0012 |     model = BiLSTMNER(vocab_size=12, num_labels=8, embedding_dim=8, hidden_dim=6)
0013 |     input_ids = torch.tensor([[2, 3, 4, 5]], dtype=torch.long)
0014 |     features = torch.zeros((1, 4, 4), dtype=torch.float32)
0015 |     labels = torch.tensor([[2, 3, 0, 0]], dtype=torch.long)
0016 |     optimizer = Adam(model.parameters(), lr=0.01)
0017 |     criterion = nn.CrossEntropyLoss()
0018 |     before = model.classifier.weight.detach().clone()
0019 |
0020 |     optimizer.zero_grad()  # 清除上一轮梯度；本例是第一轮，但正式循环每轮都需要。
0021 |     logits = model(input_ids, features)
0022 |     loss = criterion(logits.reshape(-1, 8), labels.reshape(-1))
0023 |     loss.backward()  # 计算梯度，不在这一行更新参数。
0024 |     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
0025 |     optimizer.step()  # Adam 根据梯度修改参数。
0026 |
0027 |     print("logits shape:", tuple(logits.shape))
0028 |     print("loss:", round(loss.item(), 6))
0029 |     print("classifier changed:", not torch.equal(before, model.classifier.weight))
0030 |     assert logits.shape == (1, 4, 8)
```

```
0031 |     assert not torch.equal(before, model.classifier.weight)
0032 |
0033 |
0034 | if __name__ == "__main__":
0035 |     main()
```

立刻运行

```
python -m py_compile learning/training_step_demo.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/model.py

为什么需要它：定义正式 BiLSTM Token Classifier。forward 接收 input_ids、features，返回形状 (batch, length, num_labels) 的 logits。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """从零实现 Embedding + BiLSTM + Linear 的 Token 分类模型。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import torch
0006 | from torch import nn
0007 |
0008 |
0009 | class BiLSTMNER(nn.Module):
0010 |     """根据每个 Token 的 ID、大小写特征，预测对应的 BIO 标签分数。"""
0011 |
0012 |     def __init__(self, vocab_size: int, num_labels: int, embedding_dim: int = 64, hidden_dim
.... | >: int = 96, dropout: float = 0.25):
0013 |         # 必须先初始化 nn.Module，PyTorch 才能登记下面各层的可训练参数。
0014 |         super().__init__()
0015 |         # 输入 Shape: (batch_size, sequence_length)
0016 |         # 输出 Shape: (batch_size, sequence_length, embedding_dim)
0017 |         # padding_idx=0 保证补齐位置的向量不参与正常词语学习。
0018 |         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
0019 |         # 每个 Token 有 4 个大小写/数字特征，先投影到 8 维。
0020 |         self.feature_projection = nn.Linear(4, 8)
0021 |         # BiLSTM 同时读取左到右和右到左的上下文。
0022 |         # 输出最后一维是 hidden_dim * 2，因为两个方向会拼接。
0023 |         self.lstm = nn.LSTM(
0024 |             embedding_dim + 8,
0025 |             hidden_dim,
0026 |             batch_first=True,
0027 |             bidirectional=True,
0028 |             num_layers=1,
0029 |         )
0030 |         # Dropout 只在训练模式随机屏蔽部分特征，降低过拟合。
0031 |         self.dropout = nn.Dropout(dropout)
0032 |         # 把每个 Token 的 192 维上下文表示映射为 num_labels 个 logits。
0033 |         self.classifier = nn.Linear(hidden_dim * 2, num_labels)
0034 |
0035 |     def forward(self, input_ids: torch.Tensor, features: torch.Tensor) -> torch.Tensor:
0036 |         """执行前向传播，返回 Shape 为 (batch, length, num_labels) 的 logits。"""
0037 |         # 将整数 Token ID 查表转换为可训练的稠密向量。
0038 |         embedded = self.embedding(input_ids)
0039 |         # ReLU 把大小写特征变换为非线性表示。
0040 |         projected_features = torch.relu(self.feature_projection(features))
0041 |         # 在最后一维拼接词向量与大小写特征，不改变 batch 和 sequence 维度。
```

源码第 42-46 行

```
0042 |         combined = torch.cat([embedded, projected_features], dim=-1)
0043 |         # encoded 保存每个 Token 已融合双向上下文的 Hidden State。
0044 |         encoded, _ = self.lstm(combined)
0045 |         # 返回未经过 Softmax 的 logits；CrossEntropyLoss 会在内部处理 Softmax。
0046 |         return self.classifier(self.dropout(encoded))
```

立刻运行

```
python -m py_compile src/model.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

dataset batch -> model logits -> masked labels -> loss -> gradients -> Adam parameters。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\model.py src\train.py
python -m pytest tests\test_acceptance.py -q -k training_step
```

10. Expected Results / 预期结果

training step 测试通过；loss 是有限正数，至少一个参数在 step 后变化。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

忘记 zero_grad 导致梯度累积；对 argmax 结果算 loss；padding 未 ignore；只 backward 不 step。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

Backward computes gradients; optimizer.step uses them to update parameters.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 11：Epoch、Validation、Checkpoint 与可复现训练

1. Learning Objectives / 学习目标

完成 16 个 epoch 的 CPU 训练，按 validation entity F1 保存最佳 checkpoint 与 history。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

epoch、train/eval mode、no_grad、seed、checkpoint、early model selection。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

train.py 读取 general+Malaysian train/validation，调用 model/metrics，输出 models/best_model.pt 与 reports/training_log.json。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/metrics.py    [本阶段创建或完整核对]
src/train.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/metrics.py、src/train.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/metrics.py

为什么需要它：以完整实体 span 而非单 token 统计 precision、recall、F1 和 per-class F1。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | """按实体边界与类型完全匹配计算 Entity-level Precision/Recall/F1。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | from collections import defaultdict
0006 |
0007 |
0008 | def bio_to_spans(labels: list[str]) -> set[tuple[int, int, str]]:
0009 |     """把 BIO 序列转换为 (start, end_exclusive, entity_type) 集合。"""
0010 |     spans: set[tuple[int, int, str]] = set()
0011 |     start: int | None = None
0012 |     current_type: str | None = None
0013 |     for index, label in enumerate(labels + ["O"]):
0014 |         if label.startswith("B-") or label == "O" or (label.startswith("I-") and label[2:] !
.... | >= current_type):
0015 |             if start is not None and current_type is not None:
0016 |                 spans.add((start, index, current_type))
0017 |             if label.startswith("B-"):
0018 |                 start, current_type = index, label[2:]
0019 |             else:
0020 |                 start, current_type = None, None
0021 |     return spans
0022 |
0023 |
0024 | def entity_metrics(gold_sequences: list[list[str]], predicted_sequences: list[list[str]]) ->
.... | > dict:
0025 |     """累计 PER/LOC/ORG 的 TP、FP、FN，并计算各类与总体指标。"""
0026 |     counts = defaultdict(lambda: {"tp": 0, "fp": 0, "fn": 0})
0027 |     for gold, predicted in zip(gold_sequences, predicted_sequences, strict=True):
0028 |         gold_spans = bio_to_spans(gold)
0029 |         predicted_spans = bio_to_spans(predicted)
0030 |         for entity_type in {"PER", "LOC", "ORG"}:
0031 |             gold_type = {span for span in gold_spans if span[2] == entity_type}
0032 |             pred_type = {span for span in predicted_spans if span[2] == entity_type}
0033 |             counts[entity_type]["tp"] += len(gold_type & pred_type)
0034 |             counts[entity_type]["fp"] += len(pred_type - gold_type)
0035 |             counts[entity_type]["fn"] += len(gold_type - pred_type)
0036 |
0037 |     def calculate(item: dict[str, int]) -> dict[str, float | int]:
0038 |         # 分母为 0 时返回 0，避免空实体数据导致 ZeroDivisionError。
0039 |         tp, fp, fn = item["tp"], item["fp"], item["fn"]
0040 |         precision = tp / (tp + fp) if tp + fp else 0.0
```

源码第 41-48 行

```
0041 |         recall = tp / (tp + fn) if tp + fn else 0.0
0042 |         f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
0043 |         return {**item, "precision": precision, "recall": recall, "f1": f1}
0044 |
0045 |     result = {entity_type: calculate(counts[entity_type]) for entity_type in ("PER", "LOC",
.... | >"ORG")}
0046 |     total = {key: sum(counts[kind][key] for kind in counts) for key in ("tp", "fp", "fn")}
0047 |     result["overall"] = calculate(total)
0048 |     return result
```

立刻运行

```
python -m py_compile src/metrics.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/train.py

为什么需要它：实现 seed、loss、backward、optimizer、validation、checkpoint 和 history。它串起数据、模型与 metrics。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """训练 BiLSTM，以 Validation F1 选择最佳模型，并保存完整推理工件。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import argparse
0006 | import json
0007 | import random
0008 | import time
0009 |
0010 | import numpy as np
0011 | import torch
0012 | from torch import nn
0013 | from torch.optim import Adam
0014 | from torch.utils.data import DataLoader
0015 |
0016 | from .constants import LABELS, MODELS_DIR, PROCESSED_DIR, REPORTS_DIR, SEED
0017 | from .dataset import NERDataset, collate_batch, load_jsonl
0018 | from .metrics import entity_metrics
0019 | from .model import BiLSTMNER
0020 | from .preprocessing import build_vocabulary, save_json
0021 |
0022 |
0023 | def set_seed(seed: int) -> None:
0024 |     """固定 Python、NumPy、PyTorch 随机种子，尽可能提高可复现性。"""
0025 |     random.seed(seed)
0026 |     np.random.seed(seed)
0027 |     torch.manual_seed(seed)
0028 |     if torch.cuda.is_available():
0029 |         torch.cuda.manual_seed_all(seed)
0030 |
0031 |
0032 | def run_epoch(model, loader, criterion, device, optimizer=None):
0033 |     """运行一个训练或验证 Epoch；optimizer=None 表示只验证、不更新参数。"""
0034 |     training = optimizer is not None
0035 |     model.train(training)
0036 |     total_loss = 0.0
0037 |     gold_sequences, predicted_sequences = [], []
0038 |     id_to_label = {index: label for index, label in enumerate(LABELS)}
0039 |     for batch in loader:
0040 |         input_ids = batch["input_ids"].to(device)
0041 |         features = batch["features"].to(device)
0042 |         labels = batch["labels"].to(device)
```

源码第 43-81 行

```
0043 |         mask = batch["mask"].to(device)
0044 |         if training:
0045 |             # PyTorch 默认累积梯度，所以每个 Batch 开始前必须清空旧梯度。
0046 |             optimizer.zero_grad()
0047 |         # model(...) 会由 nn.Module 自动调用 BiLSTMNER.forward()。
0048 |         logits = model(input_ids, features)
0049 |         # CrossEntropyLoss 需要二维类别分数，因此把 Batch 和 Length 合并。
0050 |         loss = criterion(logits.reshape(-1, logits.shape[-1]), labels.reshape(-1))
0051 |         if training:
0052 |             # 从 Loss 反向计算每个可训练参数的 Gradient。
0053 |             loss.backward()
0054 |             # 避免 LSTM 梯度过大造成训练不稳定。
0055 |             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
0056 |             # Adam 根据刚才的 Gradient 更新模型参数；顺序不能提前。
0057 |             optimizer.step()
0058 |         total_loss += loss.item()
0059 |         predictions = logits.argmax(dim=-1)
0060 |         for row in range(labels.shape[0]):
0061 |             length = int(mask[row].sum().item())
```

```
            gold_sequences.append([id_to_label[int(value)] for value in labels[row, :length]
>])
            predicted_sequences.append([id_to_label[int(value)] for value in predictions[row
>, :length]])
    return total_loss / max(len(loader), 1), entity_metrics(gold_sequences, predicted_sequen
>ces)


def main() -> None:
    """命令行训练入口：加载数据、建立词表、训练、验证并保存 checkpoint。"""
    parser = argparse.ArgumentParser()
    parser.add_argument("--epochs", type=int, default=12)
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--learning-rate", type=float, default=0.003)
    args = parser.parse_args()
    set_seed(SEED)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    train_rows = load_jsonl(PROCESSED_DIR / "train.jsonl")
    validation_rows = load_jsonl(PROCESSED_DIR / "validation.jsonl")
    local_train_path = PROCESSED_DIR.parent / "malaysian" / "train.jsonl"
    local_validation_path = PROCESSED_DIR.parent / "malaysian" / "validation.jsonl"
    local_train_rows = load_jsonl(local_train_path) if local_train_path.exists() else []
```

源码第 82-117 行

```
    local_validation_rows = load_jsonl(local_validation_path) if local_validation_path.exist
>s() else []
    train_rows.extend(local_train_rows)
    validation_rows.extend(local_validation_rows)
    vocabulary = build_vocabulary(row["tokens"] for row in train_rows)
    label_to_id = {label: index for index, label in enumerate(LABELS)}
    train_loader = DataLoader(NERDataset(train_rows, vocabulary, label_to_id, entity_word_dr
>opout=0.8, general_word_dropout=0.15), batch_size=args.batch_size, shuffle=True, collate_fn=
>collate_batch)
    validation_loader = DataLoader(NERDataset(validation_rows, vocabulary, label_to_id), bat
>ch_size=args.batch_size, shuffle=False, collate_fn=collate_batch)

    config = {"embedding_dim": 64, "hidden_dim": 96, "dropout": 0.25}
    model = BiLSTMNER(len(vocabulary), len(LABELS), **config).to(device)
    # Padding 位置没有真实标签，ignore_index 保证它不贡献 Loss。
    criterion = nn.CrossEntropyLoss(ignore_index=label_to_id["<PAD>"])
    optimizer = Adam(model.parameters(), lr=args.learning_rate)
    MODELS_DIR.mkdir(parents=True, exist_ok=True)
    REPORTS_DIR.mkdir(parents=True, exist_ok=True)
    best_f1 = -1.0
    history = []
    started = time.perf_counter()
    for epoch in range(1, args.epochs + 1):
        epoch_started = time.perf_counter()
        train_loss, train_metrics = run_epoch(model, train_loader, criterion, device, optimi
>zer)
        with torch.no_grad():
            validation_loss, validation_metrics = run_epoch(model, validation_loader, criter
>ion, device)
        record = {
            "epoch": epoch,
            "train_loss": train_loss,
            "validation_loss": validation_loss,
            "validation_precision": validation_metrics["overall"]["precision"],
            "validation_recall": validation_metrics["overall"]["recall"],
            "validation_f1": validation_metrics["overall"]["f1"],
            "duration_seconds": time.perf_counter() - epoch_started,
        }
        history.append(record)
        print(json.dumps(record))
        if record["validation_f1"] > best_f1:
            # 只根据 Validation F1 选模；Test Set 不参与任何训练决策。
```

源码第 118-136 行

```
            best_f1 = record["validation_f1"]
            torch.save({
                "model_state_dict": model.state_dict(),
                "vocabulary": vocabulary,
                "labels": LABELS,
                "model_config": config,
                "preprocessing": {"lowercase_vocabulary": True, "casing_features": ["is_titl
```

```
.... | >e", "is_upper", "is_lower", "has_digit"], "training_entity_word_dropout": 0.8, "training_gen
.... | >eral_word_dropout": 0.15},
0125 |                 "dataset_counts": {"training": len(train_rows), "validation": len(validation
.... | >_rows), "malaysian_training": len(local_train_rows), "malaysian_validation": len(local_valid
.... | >ation_rows)},
0126 |                 "seed": SEED,
0127 |                 "best_epoch": epoch,
0128 |                 "best_validation_f1": best_f1,
0129 |             }, MODELS_DIR / "best_model.pt")
0130 |     output = {"device": str(device), "total_duration_seconds": time.perf_counter() - started
.... | >, "history": history}
0131 |     save_json(output, REPORTS_DIR / "training_log.json")
0132 |     print(f"Saved best model with validation F1={best_f1:.4f}")
0133 |
0134 |
0135 | if __name__ == "__main__":
0136 |     main()
```

立刻运行

```
python -m py_compile src/train.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

train.py 读取 general+Malaysian train/validation，调用 model/metrics，输出 models/best_model.pt 与 reports/training_log.json。

9. Run Instructions / 直接复制的运行命令

```
python -m src.generate_data
python -m src.train --epochs 16 --batch-size 64 --learning-rate 0.003
```

10. Expected Results / 预期结果

16 行 epoch 结果；本次验收的 best epoch 为 16；checkpoint、vocabulary、labels、config、history 均已保存。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

把 test 用于选模型；validation 忘记 eval/no_grad；只存 state_dict 不存词表；随机种子不完整。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

I select the checkpoint by validation entity F1 and never use either test split for training decisions.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 12：Checkpoint Loading、Raw Text Prediction 与 BIO Decoding

1. Learning Objectives / 学习目标

输入任意英文文本，恢复模型并返回实体文字、类型和字符位置。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

checkpoint contract、map_location=cpu、argmax、BIO decoding、character offsets。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

Predictor 加载 train.py 的 checkpoint，调用 preprocessing/model，再把 BIO span 转为 API 可序列化 dict。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/predict.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/predict.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/predict.py

为什么需要它：从 checkpoint 恢复 config/vocabulary/labels/model，在 CPU 上完成 tokenization、argmax、BIO span 和字符 offset。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | """加载自己训练的 checkpoint，对原始文本执行 NER 推理。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | from dataclasses import dataclass
0006 | from pathlib import Path
0007 | import re
0008 |
0009 | import torch
0010 |
0011 | from .constants import MODELS_DIR
0012 | from .model import BiLSTMNER
0013 | from .preprocessing import casing_features, tokenize_with_spans, tokens_to_ids
0014 |
0015 |
0016 | @dataclass
0017 | class Predictor:
0018 |     """封装模型重建、Token 预测和实体抽取，供 CLI/API 复用。"""
0019 |     checkpoint_path: Path = MODELS_DIR / "best_model.pt"
0020 |
0021 |     def __post_init__(self) -> None:
0022 |         # 避免 Windows CPU 推理时 OpenMP 线程过度订阅导致首次 forward 阻塞。
0023 |         torch.set_num_threads(1)
0024 |         # checkpoint 同时保存权重、Vocabulary、Label Mapping 和模型配置。
0025 |         checkpoint = torch.load(self.checkpoint_path, map_location="cpu", weights_only=False
.... | >)
0026 |         self.vocabulary = checkpoint["vocabulary"]
0027 |         self.labels = checkpoint["labels"]
0028 |         self.model = BiLSTMNER(len(self.vocabulary), len(self.labels), **checkpoint["model_c
.... | >onfig"])
0029 |         self.model.load_state_dict(checkpoint["model_state_dict"])
0030 |         self.model.eval()
0031 |
0032 |     def predict_tokens(self, tokens: list[str]) -> list[str]:
0033 |         """接收 Token 列表，返回等长的 BIO 标签列表。"""
0034 |         if not tokens:
0035 |             return []
0036 |         input_ids = torch.tensor([tokens_to_ids(tokens, self.vocabulary)], dtype=torch.long)
0037 |         features = torch.tensor([casing_features(tokens)], dtype=torch.float32)
0038 |         with torch.no_grad():
0039 |             # 推理不需要 Gradient；argmax 选择每个 Token 分数最高的标签。
0040 |             predictions = self.model(input_ids, features).argmax(dim=-1)[0]
```

源码第 41-79 行

```
0041 |         tags = [self.labels[int(index)] for index in predictions]
0042 |         # 纯标点不可能单独构成 PER/LOC/ORG。若模型把 `!` 等预测为 I-LOC，
0043 |         # 直接解码会吞掉标点并破坏实体 text 与字符 offset 的一致性。
0044 |         return [tag if re.search(r"\w", token, flags=re.UNICODE) else "O" for token, tag in
```

```
.... | >zip(tokens, tags, strict=True)]
0045 |
0046 |     def predict(self, text: str) -> dict:
0047 |         """接收原始字符串，返回 Token、BIO 标签和合并后的实体。"""
0048 |         tokens, token_spans = tokenize_with_spans(text)
0049 |         tags = self.predict_tokens(tokens)
0050 |         entities = extract_entities(tokens, tags, token_spans)
0051 |         return {"text": text, "tokens": tokens, "tags": tags, "entities": entities}
0052 |
0053 |
0054 | def extract_entities(
0055 |     tokens: list[str],
0056 |     tags: list[str],
0057 |     token_spans: list[tuple[int, int]] | None = None,
0058 | ) -> list[dict]:
0059 |     """把连续 B/I 标签合并为带起止 Token 位置的实体字典。"""
0060 |     entities = []
0061 |     current = None
0062 |     for index, (token, tag) in enumerate(zip(tokens, tags, strict=True)):
0063 |         if tag.startswith("B-") or (tag.startswith("I-") and (current is None or current["ty
.... | >pe"] != tag[2:])):
0064 |             if current:
0065 |                 entities.append(current)
0066 |             current = {"text": token, "type": tag[2:], "start_token": index, "end_token": in
.... | >dex + 1}
0067 |         elif tag.startswith("I-") and current:
0068 |             current["text"] += " " + token
0069 |             current["end_token"] = index + 1
0070 |         else:
0071 |             if current:
0072 |                 entities.append(current)
0073 |                 current = None
0074 |     if current:
0075 |         entities.append(current)
0076 |     if token_spans is not None:
0077 |         for entity in entities:
0078 |             entity["start"] = token_spans[entity["start_token"]][0]
0079 |             entity["end"] = token_spans[entity["end_token"] - 1][1]
```

源码第 80-80 行

```
0080 |     return entities
```

立刻运行

```
python -m py_compile src/predict.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

Predictor 加载 train.py 的 checkpoint，调用 preprocessing/model，再把 BIO span 转为 API 可序列化 dict。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\predict.py
python -m pytest tests\test_prediction.py -q
```

10. Expected Results / 预期结果

checkpoint loading、CPU inference、prediction schema 测试通过。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

模型维度与 checkpoint config 不同；忘记 model.eval；用 token 索引冒充字符索引；空字符串未拒绝。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

The predictor restores the vocabulary and configuration saved with the weights, then maps BIO spans back to character offsets.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 13：字符 Span 与可靠 Text Redaction

1. Learning Objectives / 学习目标

使用模型返回的 [start,end) 字符区间进行不破坏原文的脱敏。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

half-open span、reverse replacement、overlap validation、placeholder。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

API 先调用 Predictor.predict 得到 entities，再把原文和 entities 交给 redact_text。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/redact.py    [本阶段创建或完整核对]
tests/test_redaction.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/redact.py、tests/test_redaction.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/redact.py

为什么需要它：按字符位置从右向左替换实体，避免前一次替换改变后续 offset。API 和测试都会调用它。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-30 行

```
0001 | """只根据模型预测出的实体区间生成脱敏文本。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 |
0006 | def redact_tokens(tokens: list[str], entities: list[dict]) -> str:
0007 |     """把每个实体区间替换为 [PER]/[LOC]/[ORG]，并恢复标点空格。"""
0008 |     by_start = {entity["start_token"]: entity for entity in entities}
0009 |     output = []
0010 |     index = 0
0011 |     while index < len(tokens):
0012 |         if index in by_start:
0013 |             entity = by_start[index]
0014 |             output.append(f"[{entity['type']}]")
0015 |             index = entity["end_token"]
0016 |         else:
0017 |             output.append(tokens[index])
0018 |             index += 1
0019 |     text = " ".join(output)
0020 |     for punctuation in [".", ",", "!", "?", ":", ";"]:
0021 |         text = text.replace(" " + punctuation, punctuation)
0022 |     return text
0023 |
0024 |
0025 | def redact_text(text: str, entities: list[dict]) -> str:
0026 |     """按字符区间从右向左替换实体，完整保留其余空格、换行和标点。"""
0027 |     redacted = text
0028 |     for entity in sorted(entities, key=lambda item: item["start"], reverse=True):
0029 |         redacted = redacted[: entity["start"]] + f"[{entity['type']}]" + redacted[entity["en
.... | >d"] :]
```

```
0030 |     return redacted
```

立刻运行

```
python -m py_compile src/redact.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_redaction.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-28 行

```
0001 | from src.redact import redact_text, redact_tokens
0002 |
0003 |
0004 | def test_redaction_replaces_multitoken_entities_and_repairs_spacing():
0005 |     tokens = ["Ali", "studies", "at", "Universiti", "Malaya", "."]
0006 |     entities = [
0007 |         {"text": "Ali", "type": "PER", "start_token": 0, "end_token": 1},
0008 |         {"text": "Universiti Malaya", "type": "ORG", "start_token": 3, "end_token": 5},
0009 |     ]
0010 |     assert redact_tokens(tokens, entities) == "[PER] studies at [ORG]."
0011 |
0012 |
0013 | def test_repeated_entities_are_each_redacted():
0014 |     tokens = ["Ali", "met", "Ali", "."]
0015 |     entities = [
0016 |         {"text": "Ali", "type": "PER", "start_token": 0, "end_token": 1},
0017 |         {"text": "Ali", "type": "PER", "start_token": 2, "end_token": 3},
0018 |     ]
0019 |     assert redact_tokens(tokens, entities) == "[PER] met [PER]."
0020 |
0021 |
0022 | def test_character_redaction_preserves_original_whitespace():
0023 |     text = "Alice  works at Microsoft.\n"
0024 |     entities = [
0025 |         {"type": "PER", "start": 0, "end": 5},
0026 |         {"type": "ORG", "start": 16, "end": 25},
0027 |     ]
0028 |     assert redact_text(text, entities) == "[PER]  works at [ORG].\n"
```

立刻运行

```
python -m py_compile tests/test_redaction.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文

注释给出答案。

8. Code Relationships / 跨文件关系

API 先调用 Predictor.predict 得到 entities，再把原文和 entities 交给 redact_text。

9. Run Instructions / 直接复制的运行命令

```
python -m py_compile src\redact.py
python -m pytest tests\test_redaction.py -q
```

10. Expected Results / 预期结果

普通、相邻、空实体和非法重叠测试全部通过。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

从左向右替换导致 offset 漂移；用 str.replace 误替换同名非实体；end 当作包含端点；重叠 span 未拒绝。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

I replace spans from right to left so earlier character offsets remain valid.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 14：Malaysian 学生数据合入训练与独立测试

1. Learning Objectives / 学习目标

生成可人工检查的马来西亚学生姓名、学校/大学、地点和学生语境数据，严格分离 train/validation/test。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

local domain data、synthetic disclosure、split isolation、template leakage。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

generate_data.py 输出三份 Malaysian JSONL；train.py 只合入 train/validation；evaluate.py 最后独立读取 test。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/generate_data.py    [本阶段创建或完整核对]
src/train.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/generate_data.py、src/train.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/generate_data.py

为什么需要它：生成一般与 Malaysian 三分数据，验证 BIO 边界、重复句和模板泄漏。main() 由训练前的数据准备命令调用，输出 JSONL 给 dataset.py。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-32 行

```
0001 | """生成来源透明的合成 NER 语料，并检查 Train/Validation/Test 句子泄漏。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
```

```
import random
from pathlib import Path

from .constants import MALAYSIAN_DIR, PROCESSED_DIR, SEED
from .preprocessing import tokenize, validate_bio

ENTITY_POOLS = {
    "train": {
        "PER": ["Aisha Rahman", "Daniel Wong", "Nur Izzati", "Michael Chen", "Priya Nair", "
>James Brown", "Emma Wilson", "Hafiz Omar", "Mei Lin", "Ravi Kumar", "Fatimah Zahra", "Lucas
>Martin", "Aminah", "Ravi", "Chloe", "Wei"],
        "LOC": ["Penang", "Paris", "Tokyo", "Selangor", "Berlin", "Singapore", "Perak", "Syd
>ney", "Sabah", "Dublin", "Manchester", "Putrajaya", "Kota Kinabalu", "Los Angeles", "Hong Ko
>ng", "George Town", "New Delhi", "Bandar Seri Begawan"],
        "ORG": ["University of Manchester", "Universiti Malaya", "National University of Sin
>gapore", "Monash University", "University of Sydney", "University of Tokyo", "University of
>Berlin", "Universiti Putra Malaysia", "Dublin City University", "University of Delhi", "Manc
>hester College", "Singapore Student Council"],
    },
    "validation": {
        "PER": ["Siti Aminah", "Oliver Smith", "Tan Wei Ming", "Sophia Garcia"],
        "LOC": ["Kuala Lumpur", "London", "Johor Bahru", "Madrid"],
        "ORG": ["Universiti Kebangsaan Malaysia", "University of Oxford", "Madrid Internatio
>nal School", "London Student Union"],
    },
    "test": {
        "PER": ["John Carter", "Ali Hassan", "Sarah Lee", "Ahmad Zaki", "Siti Hajar", "Grace
> Taylor"],
        "LOC": ["Edinburgh", "Bangkok", "Melbourne", "Malacca", "New York", "Kuching"],
        "ORG": ["Universiti Teknologi Malaysia", "University of Edinburgh", "Bangkok Interna
>tional School", "Sunway University", "New York University", "Kuching High School"],
    },
}

TEMPLATES = [
    "{PER} is a student at {ORG} .",
    "{PER} studies at {ORG} in {LOC} .",
```

源码第 33-66 行

```
    "{PER} reported an incident at {ORG} in {LOC} .",
    "{PER} submitted a complaint to {ORG} .",
    "The student support team contacted {PER} at {ORG} .",
    "{PER} attended a student activity in {LOC} .",
    "{ORG} organised a student event in {LOC} .",
    "{PER} travelled from {LOC} to study at {ORG} .",
    "The university report says {PER} lives in {LOC} .",
    "{PER} and {PER2} joined the club at {ORG} .",
    "{PER} met {PER2} at {ORG} in {LOC} .",
    "{PER} requested academic support from {ORG} .",
    "{PER} submitted feedback about a course at {ORG} .",
    "A student named {PER} visited {LOC} .",
    "The incident involving {PER} happened near {ORG} in {LOC} .",
    "The administrator at {ORG} emailed {PER} .",
    "{PER} arrived early for class .",
    "The student conference took place in {LOC} .",
]
NO_ENTITY = {
    "train": [
        "Please review the attached student report .", "The support request was completed su
>ccessfully .",
        "The lecture starts at nine tomorrow .", "The assignment deadline is next week .",
        "This feedback contains general course information .", "A student activity will happ
>en tomorrow .",
        "The portal processed the request correctly .", "Students submitted the assignment o
>n time .",
        "The classroom is quiet and clean .", "No private details appear in this message .",
    ],
    "validation": ["A meeting will begin tomorrow morning .", "No private details appear her
>e ."],
    "test": ["The weather is beautiful today .", "This ordinary sentence contains no entity
>."],
}

MALAYSIAN_POOLS = {
    "train": {
        "PER": ["Ahmad Firdaus", "Nurul Aisyah", "Lim Wei Jian", "Arjun Nair", "Siti Khadija
>h", "Tan Jia Hui", "Muhammad Haziq", "Devi Krishnan"],
```

```
        "LOC": ["Shah Alam", "Ipoh", "Kuantan", "Seremban", "Alor Setar", "Kota Bharu", "Mir
>i", "Cyberjaya"],
        "ORG": ["Universiti Malaya", "Universiti Putra Malaysia", "Universiti Teknologi MARA
>", "Sunway College", "Taman Melawati Secondary School", "Aminuddin Baki Secondary School", "
```

源码第 66-106 行

```
>Malaysian Student Council", "Ministry of Higher Education"],
    },
    "validation": {
        "PER": ["Farah Nabila", "Lee Jun Hao"],
        "LOC": ["Labuan", "Petaling Jaya"],
        "ORG": ["Universiti Kebangsaan Malaysia", "Victoria Secondary School"],
    },
    "test": {
        "PER": ["Aina Sofea", "Kumar Raj", "Jason Teoh"],
        "LOC": ["Sandakan", "Batu Pahat", "Kuching"],
        "ORG": ["Universiti Sains Malaysia", "Universiti Teknologi Malaysia", "International
> Islamic University Malaysia"],
    },
}


def _annotate(rendered: str, values: dict[str, str]) -> dict[str, list[str]]:
    """根据已知合成实体位置创建 BIO 标签；仅用于生成训练数据。"""
    tokens = rendered.split()
    labels = ["O"] * len(tokens)
    for entity_type, value in values.items():
        entity_tokens = value.split()
        for start in range(len(tokens) - len(entity_tokens) + 1):
            if tokens[start : start + len(entity_tokens)] == entity_tokens:
                labels[start] = f"B-{entity_type}"
                for offset in range(1, len(entity_tokens)):
                    labels[start + offset] = f"I-{entity_type}"
                break
    validate_bio(tokens, labels)
    return {"tokens": tokens, "labels": labels, "text": " ".join(tokens)}


def generate_split(split: str, size: int, seed: int) -> list[dict[str, list[str]]]:
    """使用指定 split 的独立实体池生成固定数量样本。"""
    rng = random.Random(seed)
    pool = ENTITY_POOLS[split]
    rows: list[dict[str, list[str]]] = []
    for index in range(size):
        if index % 6 == 0:
            text = rng.choice(NO_ENTITY[split])
            rows.append(_annotate(text, {}))
            continue
```

源码第 107-143 行

```
        template = rng.choice(TEMPLATES)
        values = {kind: rng.choice(items) for kind, items in pool.items() if "{" + kind + "}
>" in template}
        annotations = dict(values)
        if "{PER2}" in template:
            values["PER2"] = rng.choice([name for name in pool["PER"] if name != values.get(
>"PER")])
            annotations["PER_SECOND"] = values["PER2"]
        rendered = template.format(**values)
        annotation_types = {key: ("PER" if key == "PER_SECOND" else key) for key in annotati
>ons}
        typed_values = {annotation_types[key]: value for key, value in annotations.items() i
>f key != "PER_SECOND"}
        row = _annotate(rendered, typed_values)
        if "PER_SECOND" in annotations:
            second = annotations["PER_SECOND"].split()
            for start in range(len(row["tokens"]) - len(second) + 1):
                if row["tokens"][start:start + len(second)] == second:
                    row["labels"][start] = "B-PER"
                    for offset in range(1, len(second)):
                        row["labels"][start + offset] = "I-PER"
                    break
            validate_bio(row["tokens"], row["labels"])
        rows.append(row)
    rng.shuffle(rows)
    return rows
```

```
0129 |
0130 |
0131 | def generate_malaysian_split(split: str, size: int, seed: int) -> list[dict[str, list[str]]]
.... | >:
0132 |     """生成实体池互斥的马来西亚合成语料；规则标注后仍需人工抽查。"""
0133 |     rng = random.Random(seed)
0134 |     pool = MALAYSIAN_POOLS[split]
0135 |     local_templates = [template for template in TEMPLATES if "{PER2}" not in template]
0136 |     rows = []
0137 |     for index in range(size):
0138 |         if index % 8 == 0:
0139 |             neutral = {
0140 |                 "train": "The submitted form contains no named entity .",
0141 |                 "validation": "This local validation sentence has no entity .",
0142 |                 "test": "This local test sentence has no entity .",
0143 |             }[split]
```

源码第 144-178 行

```
0144 |             rows.append(_annotate(neutral, {}))
0145 |             continue
0146 |         template = rng.choice(local_templates)
0147 |         values = {kind: rng.choice(items) for kind, items in pool.items() if "{" + kind + "}
.... | >" in template}
0148 |         rows.append(_annotate(template.format(**values), values))
0149 |     rng.shuffle(rows)
0150 |     return rows
0151 |
0152 |
0153 | def _write_jsonl(rows: list[dict], path: Path) -> None:
0154 |     """将样本保存为一行一个 JSON 对象的 UTF-8 JSONL。"""
0155 |     path.parent.mkdir(parents=True, exist_ok=True)
0156 |     with path.open("w", encoding="utf-8") as stream:
0157 |         for row in rows:
0158 |             stream.write(json.dumps(row, ensure_ascii=False) + "\n")
0159 |
0160 |
0161 | def main() -> None:
0162 |     """生成三个 Split；发现规范化句子跨 Split 重复时立即失败。"""
0163 |     # 完整句子和实体字符串都必须跨 split 隔离，避免验证/测试指标因记忆
0164 |     # 训练集专有名词而虚高。
0165 |     for entity_type in ("PER", "LOC", "ORG"):
0166 |         split_sets = {split: set(values[entity_type]) for split, values in ENTITY_POOLS.item
.... | >s()}
0167 |         for left, right in (("train", "validation"), ("train", "test"), ("validation", "test
.... | >")):
0168 |             overlap = split_sets[left] & split_sets[right]
0169 |             if overlap:
0170 |                 raise RuntimeError(f"{entity_type} entity leakage between {left}/{right}: {s
.... | >orted(overlap)}")
0171 |         combined = {
0172 |             split: set(ENTITY_POOLS[split][entity_type]) | set(MALAYSIAN_POOLS[split][entity
.... | >_type])
0173 |             for split in ("train", "validation", "test")
0174 |         }
0175 |         for left, right in (("train", "validation"), ("train", "test"), ("validation", "test
.... | >")):
0176 |             overlap = combined[left] & combined[right]
0177 |             if overlap:
0178 |                 raise RuntimeError(f"Combined {entity_type} leakage between {left}/{right}:
.... | >{sorted(overlap)}")
```

源码第 179-217 行

```
0179 |
0180 |     sizes = {"train": 2400, "validation": 300, "test": 300}
0181 |     all_texts: set[str] = set()
0182 |     summary = {}
0183 |     for offset, (split, size) in enumerate(sizes.items()):
0184 |         rows = generate_split(split, size, SEED + offset)
0185 |         texts = {row["text"].lower() for row in rows}
0186 |         overlap = all_texts & texts
0187 |         if overlap:
0188 |             # 绝不自动忽略泄漏；失败能迫使开发者先修复数据再训练。
0189 |             raise RuntimeError(f"Sentence leakage into {split}: {sorted(overlap)[:3]}")
0190 |         all_texts |= texts
0191 |         _write_jsonl(rows, PROCESSED_DIR / f"{split}.jsonl")
0192 |         summary[split] = {"rows": len(rows), "unique_sentences": len(texts)}
```

```
0193 |
0194 |     challenge_texts = [
0195 |         "Ahmad Ali is a student at Universiti Teknologi Malaysia in Johor Bahru.",
0196 |         "Kumar submitted a complaint to Universiti Sains Malaysia in George Town.",
0197 |         "Aina and Jason joined a student activity at International Islamic University Malays
.... | >ia.",
0198 |         "The assignment deadline is next week.",
0199 |     ]
0200 |     MALAYSIAN_DIR.mkdir(parents=True, exist_ok=True)
0201 |     (MALAYSIAN_DIR / "challenge.txt").write_text("\n".join(challenge_texts) + "\n", encoding
.... | >="utf-8")
0202 |     local_sizes = {"train": 160, "validation": 40, "test": 60}
0203 |     local_texts: set[str] = set()
0204 |     for offset, (split, size) in enumerate(local_sizes.items()):
0205 |         rows = generate_malaysian_split(split, size, SEED + 100 + offset)
0206 |         texts = {row["text"].lower() for row in rows}
0207 |         if local_texts & texts:
0208 |             raise RuntimeError(f"Malaysian sentence leakage into {split}")
0209 |         local_texts |= texts
0210 |         _write_jsonl(rows, MALAYSIAN_DIR / f"{split}.jsonl")
0211 |         summary[f"malaysian_{split}"] = {"rows": len(rows), "unique_sentences": len(texts)}
0212 |     (PROCESSED_DIR / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf
.... | >-8")
0213 |     print(json.dumps(summary, indent=2))
0214 |
0215 |
0216 | if __name__ == "__main__":
0217 |     main()
```

立刻运行

```
python -m py_compile src/generate_data.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/train.py

为什么需要它：实现 seed、loss、backward、optimizer、validation、checkpoint 和 history。它串起数据、模型与 metrics。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """训练 BiLSTM，以 Validation F1 选择最佳模型，并保存完整推理工件。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import argparse
0006 | import json
0007 | import random
0008 | import time
0009 |
0010 | import numpy as np
0011 | import torch
0012 | from torch import nn
0013 | from torch.optim import Adam
0014 | from torch.utils.data import DataLoader
0015 |
0016 | from .constants import LABELS, MODELS_DIR, PROCESSED_DIR, REPORTS_DIR, SEED
0017 | from .dataset import NERDataset, collate_batch, load_jsonl
0018 | from .metrics import entity_metrics
0019 | from .model import BiLSTMNER
0020 | from .preprocessing import build_vocabulary, save_json
```

```
0021 |
0022 |
0023 | def set_seed(seed: int) -> None:
0024 |     """固定 Python、NumPy、PyTorch 随机种子，尽可能提高可复现性。"""
0025 |     random.seed(seed)
0026 |     np.random.seed(seed)
0027 |     torch.manual_seed(seed)
0028 |     if torch.cuda.is_available():
0029 |         torch.cuda.manual_seed_all(seed)
0030 |
0031 |
0032 | def run_epoch(model, loader, criterion, device, optimizer=None):
0033 |     """运行一个训练或验证 Epoch；optimizer=None 表示只验证、不更新参数。"""
0034 |     training = optimizer is not None
0035 |     model.train(training)
0036 |     total_loss = 0.0
0037 |     gold_sequences, predicted_sequences = [], []
0038 |     id_to_label = {index: label for index, label in enumerate(LABELS)}
0039 |     for batch in loader:
0040 |         input_ids = batch["input_ids"].to(device)
0041 |         features = batch["features"].to(device)
0042 |         labels = batch["labels"].to(device)
```

源码第 43-81 行

```
0043 |         mask = batch["mask"].to(device)
0044 |         if training:
0045 |             # PyTorch 默认累积梯度，所以每个 Batch 开始前必须清空旧梯度。
0046 |             optimizer.zero_grad()
0047 |         # model(...) 会由 nn.Module 自动调用 BiLSTMNER.forward()。
0048 |         logits = model(input_ids, features)
0049 |         # CrossEntropyLoss 需要二维类别分数，因此把 Batch 和 Length 合并。
0050 |         loss = criterion(logits.reshape(-1, logits.shape[-1]), labels.reshape(-1))
0051 |         if training:
0052 |             # 从 Loss 反向计算每个可训练参数的 Gradient。
0053 |             loss.backward()
0054 |             # 避免 LSTM 梯度过大造成训练不稳定。
0055 |             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
0056 |             # Adam 根据刚才的 Gradient 更新模型参数；顺序不能提前。
0057 |             optimizer.step()
0058 |         total_loss += loss.item()
0059 |         predictions = logits.argmax(dim=-1)
0060 |         for row in range(labels.shape[0]):
0061 |             length = int(mask[row].sum().item())
0062 |             gold_sequences.append([id_to_label[int(value)] for value in labels[row, :length]
.... | >])
0063 |             predicted_sequences.append([id_to_label[int(value)] for value in predictions[row
.... | >, :length]])
0064 |     return total_loss / max(len(loader), 1), entity_metrics(gold_sequences, predicted_sequen
.... | >ces)
0065 |
0066 |
0067 | def main() -> None:
0068 |     """命令行训练入口：加载数据、建立词表、训练、验证并保存 checkpoint。"""
0069 |     parser = argparse.ArgumentParser()
0070 |     parser.add_argument("--epochs", type=int, default=12)
0071 |     parser.add_argument("--batch-size", type=int, default=64)
0072 |     parser.add_argument("--learning-rate", type=float, default=0.003)
0073 |     args = parser.parse_args()
0074 |     set_seed(SEED)
0075 |     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
0076 |
0077 |     train_rows = load_jsonl(PROCESSED_DIR / "train.jsonl")
0078 |     validation_rows = load_jsonl(PROCESSED_DIR / "validation.jsonl")
0079 |     local_train_path = PROCESSED_DIR.parent / "malaysian" / "train.jsonl"
0080 |     local_validation_path = PROCESSED_DIR.parent / "malaysian" / "validation.jsonl"
0081 |     local_train_rows = load_jsonl(local_train_path) if local_train_path.exists() else []
```

源码第 82-117 行

```
0082 |     local_validation_rows = load_jsonl(local_validation_path) if local_validation_path.exist
.... | >s() else []
0083 |     train_rows.extend(local_train_rows)
0084 |     validation_rows.extend(local_validation_rows)
0085 |     vocabulary = build_vocabulary(row["tokens"] for row in train_rows)
0086 |     label_to_id = {label: index for index, label in enumerate(LABELS)}
0087 |     train_loader = DataLoader(NERDataset(train_rows, vocabulary, label_to_id, entity_word_dr
.... | >opout=0.8, general_word_dropout=0.15), batch_size=args.batch_size, shuffle=True, collate_fn=
```

```
....  | >collate_batch)
0088  |     validation_loader = DataLoader(NERDataset(validation_rows, vocabulary, label_to_id), bat
....  | >ch_size=args.batch_size, shuffle=False, collate_fn=collate_batch)
0089  |
0090  |     config = {"embedding_dim": 64, "hidden_dim": 96, "dropout": 0.25}
0091  |     model = BiLSTMNER(len(vocabulary), len(LABELS), **config).to(device)
0092  |     # Padding 位置没有真实标签，ignore_index 保证它不贡献 Loss。
0093  |     criterion = nn.CrossEntropyLoss(ignore_index=label_to_id["<PAD>"])
0094  |     optimizer = Adam(model.parameters(), lr=args.learning_rate)
0095  |     MODELS_DIR.mkdir(parents=True, exist_ok=True)
0096  |     REPORTS_DIR.mkdir(parents=True, exist_ok=True)
0097  |     best_f1 = -1.0
0098  |     history = []
0099  |     started = time.perf_counter()
0100  |     for epoch in range(1, args.epochs + 1):
0101  |         epoch_started = time.perf_counter()
0102  |         train_loss, train_metrics = run_epoch(model, train_loader, criterion, device, optimi
....  | >zer)
0103  |         with torch.no_grad():
0104  |             validation_loss, validation_metrics = run_epoch(model, validation_loader, criter
....  | >ion, device)
0105  |         record = {
0106  |             "epoch": epoch,
0107  |             "train_loss": train_loss,
0108  |             "validation_loss": validation_loss,
0109  |             "validation_precision": validation_metrics["overall"]["precision"],
0110  |             "validation_recall": validation_metrics["overall"]["recall"],
0111  |             "validation_f1": validation_metrics["overall"]["f1"],
0112  |             "duration_seconds": time.perf_counter() - epoch_started,
0113  |         }
0114  |         history.append(record)
0115  |         print(json.dumps(record))
0116  |         if record["validation_f1"] > best_f1:
0117  |             # 只根据 Validation F1 选模；Test Set 不参与任何训练决策。
```

源码第 118-136 行

```
0118  |             best_f1 = record["validation_f1"]
0119  |             torch.save({
0120  |                 "model_state_dict": model.state_dict(),
0121  |                 "vocabulary": vocabulary,
0122  |                 "labels": LABELS,
0123  |                 "model_config": config,
0124  |                 "preprocessing": {"lowercase_vocabulary": True, "casing_features": ["is_titl
....  | >e", "is_upper", "is_lower", "has_digit"], "training_entity_word_dropout": 0.8, "training_gen
....  | >eral_word_dropout": 0.15},
0125  |                 "dataset_counts": {"training": len(train_rows), "validation": len(validation
....  | >_rows), "malaysian_training": len(local_train_rows), "malaysian_validation": len(local_valid
....  | >ation_rows)},
0126  |                 "seed": SEED,
0127  |                 "best_epoch": epoch,
0128  |                 "best_validation_f1": best_f1,
0129  |             }, MODELS_DIR / "best_model.pt")
0130  |     output = {"device": str(device), "total_duration_seconds": time.perf_counter() - started
....  | >, "history": history}
0131  |     save_json(output, REPORTS_DIR / "training_log.json")
0132  |     print(f"Saved best model with validation F1={best_f1:.4f}")
0133  |
0134  |
0135  | if __name__ == "__main__":
0136  |     main()
```

立刻运行

```
python -m py_compile src/train.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

generate_data.py 输出三份 Malaysian JSONL；train.py 只合入 train/validation；evaluate.py 最后独立读取 test。

9. Run Instructions / 直接复制的运行命令

```
python -m src.generate_data
python -m src.train --epochs 16 --batch-size 64 --learning-rate 0.003
python -m src.evaluate
```

10. Expected Results / 预期结果

160/40/60 Malaysian 学生句子；训练不读取 local test；报告单独列 Malaysian 指标。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

把 test 合入训练；跨 split 复用完整句子或实体；把合成数据描述成真实用户数据。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

The Malaysian student corpus is synthetic and manually inspectable; its test split is never used for training.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 15：Entity-level Evaluation、失败案例与限制

1. Learning Objectives / 学习目标

按完整实体边界计算 precision/recall/F1/per-class F1，并诚实记录泛化失败。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

TP/FP/FN、micro F1、per-class F1、person recall、token accuracy trap。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

metrics.py 把 BIO 序列变成 span；evaluate.py 加载 checkpoint 并分别评估两个 test split。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
src/metrics.py    [本阶段创建或完整核对]
src/evaluate.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：src/metrics.py、src/evaluate.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：src/metrics.py

为什么需要它：以完整实体 span 而非单 token 统计 precision、recall、F1 和 per-class F1。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | """按实体边界与类型完全匹配计算 Entity-level Precision/Recall/F1。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | from collections import defaultdict
0006 |
0007 |
```

```
0008 | def bio_to_spans(labels: list[str]) -> set[tuple[int, int, str]]:
0009 |     """把 BIO 序列转换为 (start, end_exclusive, entity_type) 集合。"""
0010 |     spans: set[tuple[int, int, str]] = set()
0011 |     start: int | None = None
0012 |     current_type: str | None = None
0013 |     for index, label in enumerate(labels + ["O"]):
0014 |         if label.startswith("B-") or label == "O" or (label.startswith("I-") and label[2:] !
.... | >= current_type):
0015 |             if start is not None and current_type is not None:
0016 |                 spans.add((start, index, current_type))
0017 |             if label.startswith("B-"):
0018 |                 start, current_type = index, label[2:]
0019 |             else:
0020 |                 start, current_type = None, None
0021 |     return spans
0022 |
0023 |
0024 | def entity_metrics(gold_sequences: list[list[str]], predicted_sequences: list[list[str]]) ->
.... | > dict:
0025 |     """累计 PER/LOC/ORG 的 TP、FP、FN，并计算各类与总体指标。"""
0026 |     counts = defaultdict(lambda: {"tp": 0, "fp": 0, "fn": 0})
0027 |     for gold, predicted in zip(gold_sequences, predicted_sequences, strict=True):
0028 |         gold_spans = bio_to_spans(gold)
0029 |         predicted_spans = bio_to_spans(predicted)
0030 |         for entity_type in {"PER", "LOC", "ORG"}:
0031 |             gold_type = {span for span in gold_spans if span[2] == entity_type}
0032 |             pred_type = {span for span in predicted_spans if span[2] == entity_type}
0033 |             counts[entity_type]["tp"] += len(gold_type & pred_type)
0034 |             counts[entity_type]["fp"] += len(pred_type - gold_type)
0035 |             counts[entity_type]["fn"] += len(gold_type - pred_type)
0036 |
0037 |     def calculate(item: dict[str, int]) -> dict[str, float | int]:
0038 |         # 分母为 0 时返回 0，避免空实体数据导致 ZeroDivisionError。
0039 |         tp, fp, fn = item["tp"], item["fp"], item["fn"]
0040 |         precision = tp / (tp + fp) if tp + fp else 0.0
```

源码第 41-48 行

```
0041 |         recall = tp / (tp + fn) if tp + fn else 0.0
0042 |         f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
0043 |         return {**item, "precision": precision, "recall": recall, "f1": f1}
0044 |
0045 |     result = {entity_type: calculate(counts[entity_type]) for entity_type in ("PER", "LOC",
.... | >"ORG")}
0046 |     total = {key: sum(counts[kind][key] for kind in counts) for key in ("tp", "fp", "fn")}
0047 |     result["overall"] = calculate(total)
0048 |     return result
```

立刻运行

```
python -m py_compile src/metrics.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：src/evaluate.py

为什么需要它：加载最终 checkpoint，分别评估 general test 与 Malaysian test，并把真实指标写入报告。打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-35 行

```
0001 | """在独立 Test Set 上评估冻结 checkpoint，并保存真实 Entity-level 指标。"""
```

```
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 |
0007 | from .constants import MALAYSIAN_DIR, MODELS_DIR, PROCESSED_DIR, REPORTS_DIR
0008 | from .dataset import load_jsonl
0009 | from .metrics import entity_metrics
0010 | from .predict import Predictor
0011 | from .preprocessing import save_json
0012 |
0013 |
0014 | def main() -> None:
0015 |     """评估入口；此文件不创建 Optimizer，也不会修改模型权重。"""
0016 |     predictor = Predictor(MODELS_DIR / "best_model.pt")
0017 |     rows = load_jsonl(PROCESSED_DIR / "test.jsonl")
0018 |     gold = [row["labels"] for row in rows]
0019 |     predicted = [predictor.predict_tokens(row["tokens"]) for row in rows]
0020 |     metrics = entity_metrics(gold, predicted)
0021 |     report = {"dataset": "synthetic leakage-safe test split", "sentences": len(rows), "metri
.... | >cs": metrics}
0022 |     save_json(report, REPORTS_DIR / "test_metrics.json")
0023 |     print(json.dumps(report, indent=2))
0024 |     local_rows = load_jsonl(MALAYSIAN_DIR / "test.jsonl")
0025 |     local_metrics = entity_metrics(
0026 |         [row["labels"] for row in local_rows],
0027 |         [predictor.predict_tokens(row["tokens"]) for row in local_rows],
0028 |     )
0029 |     local_report = {"dataset": "synthetic Malaysian local test split", "sentences": len(loca
.... | >l_rows), "metrics": local_metrics}
0030 |     save_json(local_report, REPORTS_DIR / "malaysian_test_metrics.json")
0031 |     print(json.dumps(local_report, indent=2))
0032 |
0033 |
0034 | if __name__ == "__main__":
0035 |     main()
```

立刻运行

```
python -m py_compile src/evaluate.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

metrics.py 把 BIO 序列变成 span；evaluate.py 加载 checkpoint 并分别评估两个 test split。

9. Run Instructions / 直接复制的运行命令

```
python -m src.evaluate
Get-Content reports\evaluation.json
```

10. Expected Results / 预期结果

general 300 与 Malaysian 60 均被评估；报告包含 overall 与 PER/LOC/ORG。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

只报 token accuracy；用测试集调参；把合成模板 1.0 宣传成真实场景表现；隐藏人工失败样例。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

Entity-level F1 requires both the type and the complete span to be correct; token accuracy alone is misleading.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 16：FastAPI Text API 与真实 HTTP 验收

1. Learning Objectives / 学习目标

提供 POST /api/predict，让浏览器或客户端提交原始英文文本并收到 entities 与 redacted_text。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

HTTP、JSON、Pydantic、status code、application lifespan、shared Predictor。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

app/main.py -> Predictor -> model -> redact_text -> JSON response；测试使用 FastAPI TestClient。请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
tutorial/app_versions/app_v1_text_api.py    [本阶段创建或完整核对]
tests/test_text_api.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：tutorial/app_versions/app_v1_text_api.py、tests/test_text_api.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：tutorial/app_versions/app_v1_text_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """Phase 16: 只有文本 JSON API, 没有文件或网页。"""
0002 |
0003 | from fastapi import FastAPI, HTTPException
0004 | from pydantic import BaseModel, Field, field_validator
0005 |
0006 | from src.predict import Predictor
0007 | from src.redact import redact_text
0008 |
0009 | app = FastAPI(title="Student Privacy Text API")
0010 | predictor: Predictor | None = None
0011 |
0012 |
0013 | class TextRequest(BaseModel):
0014 |     text: str = Field(min_length=1, max_length=100_000)
0015 |
0016 |     @field_validator("text")
0017 |     @classmethod
0018 |     def reject_blank_text(cls, value: str) -> str:
0019 |         if not value.strip():
0020 |             raise ValueError("text must contain a visible character")
0021 |         return value
0022 |
0023 |
0024 | class RedactResponse(BaseModel):
0025 |     redacted_text: str
0026 |
0027 |
0028 | def get_predictor() -> Predictor:
0029 |     global predictor
0030 |     if predictor is None:
0031 |         predictor = Predictor()
0032 |     return predictor
0033 |
0034 |
0035 | def analyze(text: str) -> dict:
```

```
0036 |     result = get_predictor().predict(text)
0037 |     result["redacted_text"] = redact_text(text, result["entities"])
0038 |     return result
0039 |
0040 |
0041 | @app.post("/api/predict")
0042 | def predict(request: TextRequest) -> dict:
```

源码第 43-52 行

```
0043 |     return analyze(request.text)
0044 |
0045 |
0046 | @app.post("/api/redact", response_model=RedactResponse)
0047 | def redact(request: TextRequest) -> RedactResponse:
0048 |     try:
0049 |         result = analyze(request.text)
0050 |     except (FileNotFoundError, RuntimeError) as error:
0051 |         raise HTTPException(status_code=503, detail="The trained NER model is not available"
.... | ) from error
0052 |     return RedactResponse(redacted_text=result["redacted_text"])
```

立刻运行

```
python -m py_compile tutorial/app_versions/app_v1_text_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_text_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-25 行

```
0001 | """Phase 16 只测试文本 API；文件上传留到 Phase 17。"""
0002 |
0003 | import pytest
0004 | from pydantic import ValidationError
0005 |
0006 | from tutorial.app_versions.app_v1_text_api import TextRequest, predict, redact
0007 |
0008 |
0009 | def test_text_prediction_and_redaction_use_model():
0010 |     text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
0011 |     full = predict(TextRequest(text=text))
0012 |     short = redact(TextRequest(text=text))
0013 |     assert {entity["type"] for entity in full["entities"]} == {"PER", "ORG", "LOC"}
0014 |     assert short.redacted_text == "[PER] studies at [ORG] in [LOC]."
0015 |
0016 |
0017 | def test_blank_text_is_rejected_before_model():
0018 |     with pytest.raises(ValidationError):
0019 |         TextRequest(text="  \n")
0020 |
0021 |
0022 | def test_predictor_is_cached_after_first_request():
0023 |     from tutorial.app_versions import app_v1_text_api
0024 |
0025 |     assert app_v1_text_api.predictor is not None
```

立刻运行

```
python -m py_compile tests/test_text_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

app/main.py -> Predictor -> model -> redact_text -> JSON response；测试使用 FastAPI TestClient。

9. Run Instructions / 直接复制的运行命令

```
python -m pytest tests\test_api.py -q
python -m uvicorn app.main:app --reload
```

10. Expected Results / 预期结果

API 测试通过；浏览器访问 http://127.0.0.1:8000；空文本返回 422/400 而非假成功。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

每次请求重复加载 checkpoint；吞掉异常仍返回 200；字段名与前端不一致；服务启动目录错误。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

FastAPI validates the request and reuses one trained predictor instead of loading the checkpoint for every call.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人

工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 17：TXT/PDF 文件处理、安全限制与错误响应

1. Learning Objectives / 学习目标

安全接收最多 5 MB 的 UTF-8 TXT 或带文本层 PDF，并把提取文字送入同一自训练模型。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

multipart、MIME/extension、size limit、UTF-8、PDF text layer、OCR boundary。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

upload endpoint -> validate bytes/type -> extract text -> Predictor -> redact -> JSON。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
tutorial/app_versions/app_v2_file_api.py    [本阶段创建或完整核对]
tests/test_file_api.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：tutorial/app_versions/app_v2_file_api.py、tests/test_file_api.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：tutorial/app_versions/app_v2_file_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | """Phase 17: 在文本 API 上增加受限 TXT/PDF 上传。"""
0002 |
0003 | from io import BytesIO
0004 |
0005 | from fastapi import File, HTTPException, UploadFile
0006 | from pypdf import PdfReader
0007 | from starlette.concurrency import run_in_threadpool
0008 |
0009 | from tutorial.app_versions.app_v1_text_api import app, analyze
0010 |
0011 | MAX_UPLOAD_BYTES = 5 * 1024 * 1024
0012 | MAX_TEXT_CHARS = 100_000
0013 |
0014 |
0015 | def analyze_uploaded_content(name: str, content: bytes) -> dict:
0016 |     if len(content) > MAX_UPLOAD_BYTES:
0017 |         raise HTTPException(status_code=413, detail="File exceeds the 5 MB limit")
0018 |     try:
0019 |         if name.endswith(".txt"):
0020 |             text = content.decode("utf-8")
0021 |         elif name.endswith(".pdf"):
0022 |             text = "\n".join(page.extract_text() or "" for page in PdfReader(BytesIO(content
.... | >)).pages)
0023 |         else:
0024 |             raise HTTPException(status_code=415, detail="Only .txt and .pdf files are suppor
.... | >ted")
0025 |     except UnicodeDecodeError as error:
0026 |         raise HTTPException(status_code=400, detail="TXT file must use UTF-8 encoding") from
.... | > error
0027 |     except HTTPException:
0028 |         raise
0029 |     except Exception as error:
0030 |         raise HTTPException(status_code=400, detail="Could not extract file text") from erro
.... | >r
0031 |     if not text.strip():
0032 |         raise HTTPException(status_code=400, detail="The file contains no extractable text")
0033 |     if len(text) > MAX_TEXT_CHARS:
0034 |         raise HTTPException(status_code=413, detail="Extracted text is too long")
0035 |     return analyze(text)
0036 |
0037 |
0038 | @app.post("/api/upload")
```

源码第 39-42 行

```
0039 | async def upload(file: UploadFile = File(...)) -> dict:
0040 |     name = (file.filename or "").lower()
0041 |     content = await file.read(MAX_UPLOAD_BYTES + 1)
0042 |     return await run_in_threadpool(analyze_uploaded_content, name, content)
```

立刻运行

```
python -m py_compile tutorial/app_versions/app_v2_file_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_file_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | """Phase 17: 文件解析与限制，不依赖最终网页。"""
0002 |
0003 | from io import BytesIO
0004 |
0005 | import pytest
0006 | from fastapi import HTTPException
0007 | from reportlab.pdfgen import canvas
0008 |
0009 | from tutorial.app_versions.app_v2_file_api import MAX_UPLOAD_BYTES, analyze_uploaded_content
0010 |
0011 |
0012 | def test_txt_and_pdf_use_the_same_model_path():
0013 |     txt = analyze_uploaded_content(
0014 |         "student.txt", b"Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
0015 |     )
0016 |     buffer = BytesIO()
0017 |     pdf = canvas.Canvas(buffer)
0018 |     pdf.drawString(72, 750, "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bah
.... | >ru.")
0019 |     pdf.save()
0020 |     document = analyze_uploaded_content("student.pdf", buffer.getvalue())
0021 |     assert txt["redacted_text"] == "[PER] studies at [ORG] in [LOC]."
0022 |     assert document["redacted_text"].strip() == "[PER] studies at [ORG] in [LOC]."
0023 |
0024 |
0025 | @pytest.mark.parametrize(
0026 |     ("name", "content", "status"),
0027 |     [("data.csv", b"x", 415), ("blank.txt", b"  ", 400)],
0028 | )
0029 | def test_file_errors_are_explicit(name, content, status):
0030 |     with pytest.raises(HTTPException) as captured:
0031 |         analyze_uploaded_content(name, content)
0032 |     assert captured.value.status_code == status
0033 |
0034 |
0035 | def test_file_size_limit():
0036 |     with pytest.raises(HTTPException) as captured:
0037 |         analyze_uploaded_content("large.txt", b"a" * (MAX_UPLOAD_BYTES + 1))
0038 |     assert captured.value.status_code == 413
```

立刻运行

```
python -m py_compile tests/test_file_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

upload endpoint -> validate bytes/type -> extract text -> Predictor -> redact -> JSON。

9. Run Instructions / 直接复制的运行命令

```
python -m pytest tests\test_acceptance.py -q -k upload
python -m uvicorn app.main:app --reload
```

10. Expected Results / 预期结果

TXT/PDF 上传测试通过；错误类型、过大文件、空/扫描 PDF 返回明确 4xx。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

只信 Content-Type；无限读取上传；把 PDF bytes 当 UTF-8；声称支持 OCR；文件失败返回 500。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

TXT and text-layer PDF are supported; scanned PDFs need OCR and are rejected when no text is extracted.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 18：Web Interface 与端到端数据流

1. Learning Objectives / 学习目标

让用户粘贴文本、点击 Detect、上传文件并清楚看到实体类型、offset 与脱敏文本。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

HTML form、fetch、FormData、DOM、JSON rendering、end-to-end flow。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

browser -> FastAPI -> Predictor -> response -> DOM；网页不执行 NER，只展示服务器模型结果。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

app/main.py　[本阶段创建或完整核对]
tests/test_web_ui.py　[本阶段创建或完整核对]

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：app/main.py、tests/test_web_ui.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：app/main.py

为什么需要它：FastAPI 应用、网页、JSON API、TXT/PDF 上传、大小与类型验证都在这里；NER 始终调用自训练 Predictor。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | from __future__ import annotations
0002 |
0003 | from io import BytesIO
0004 |
0005 | from fastapi import FastAPI, File, HTTPException, UploadFile
0006 | from fastapi.responses import HTMLResponse
0007 | from pydantic import BaseModel, Field, field_validator
0008 | from pypdf import PdfReader
0009 | from starlette.concurrency import run_in_threadpool
0010 |
0011 | from src.predict import Predictor
0012 | from src.redact import redact_text
0013 |
0014 | app = FastAPI(title="Student Privacy Protection NER", version="1.1.0")
0015 | # 模型延迟加载；命令行和 Uvicorn 启动时可先完成模块导入。
0016 | predictor: Predictor | None = None
0017 | MAX_UPLOAD_BYTES = 5 * 1024 * 1024
0018 | MAX_TEXT_CHARS = 100_000
0019 |
0020 |
0021 | class TextRequest(BaseModel):
0022 |     """POST /api/predict 的 JSON 请求结构。"""
0023 |     text: str = Field(min_length=1, max_length=100_000)
0024 |
0025 |     @field_validator("text")
0026 |     @classmethod
0027 |     def reject_blank_text(cls, value: str) -> str:
0028 |         if not value.strip():
0029 |             raise ValueError("text must contain at least one visible character")
0030 |         return value
0031 |
0032 |
0033 | class RedactResponse(BaseModel):
```

```
    """精简网页只需要脱敏文本；结果仍来自同一个模型推理流程。"""

    redacted_text: str


def get_predictor() -> Predictor:
    """首次调用时从 checkpoint 加载模型，之后在同一进程内复用。"""
    global predictor
    if predictor is None:
```

源码第 43-80 行

```
        predictor = Predictor()
    return predictor


def analyze(text: str) -> dict:
    """统一执行实体预测与脱敏，供文本和文件 Endpoint 调用。"""
    result = get_predictor().predict(text)
    result["redacted_text"] = redact_text(text, result["entities"])
    return result


@app.get("/", response_class=HTMLResponse)
def home() -> str:
    """返回零构建步骤的响应式单页前端；NER 只在 FastAPI 后端执行。"""
    return """<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1'>
  <title>Student Privacy Redaction</title>
  <style>
    :root{font-family:Inter,"Segoe UI",sans-serif;color:#172033;background:#f3f6fb}
    *{box-sizing:border-box}body{margin:0;padding:32px 16px}
    main{max-width:920px;margin:auto;background:#fff;border:1px solid #dbe3ef;border-radius:
>18px;padding:clamp(20px,4vw,40px);box-shadow:0 18px 50px #1d35571a}
    h1{margin:0 0 10px;font-size:clamp(1.8rem,5vw,2.6rem);color:#123b6d}
    .intro{margin:0 0 30px;color:#53657d;line-height:1.6}
    .grid{display:grid;grid-template-columns:1fr 1fr;gap:22px}
    label{display:block;font-weight:700;margin-bottom:9px;color:#243b5a}
    textarea{width:100%;min-height:280px;resize:vertical;border:1px solid #b8c6d9;border-rad
>ius:12px;padding:15px;font:inherit;line-height:1.55;color:#172033;background:#fff}
    textarea:focus{outline:3px solid #2d73c733;border-color:#2d73c7}
    textarea[readonly]{background:#f7f9fc}
    .actions{display:flex;gap:10px;align-items:center;margin-top:20px;flex-wrap:wrap}
    button{border:0;border-radius:10px;padding:12px 18px;font:700 1rem inherit;cursor:pointe
>r}
    #redact{background:#1769aa;color:#fff}#redact:hover{background:#10588f}
    #clear{background:#e8eef6;color:#26415f}button:disabled{opacity:.6;cursor:not-allowed}
    #status{min-height:24px;margin:16px 0 0;color:#325f3b;font-weight:600}
    #status.error{color:#a32727}
    @media(max-width:700px){body{padding:16px 10px}.grid{grid-template-columns:1fr}textarea{
>min-height:210px}}
```

源码第 81-110 行

```
  </style>
</head>
<body>
<main>
  <h1>Student Privacy Redaction</h1>
  <p class='intro'>Paste English student-related text. The trained NER model will replace de
>tected PERSON, ORGANISATION and LOCATION entities.</p>
  <div class='grid'>
    <section><label for='input-text'>Input Text</label><textarea id='input-text' maxlength='
>100000' placeholder='Ahmad Ali is studying at Universiti Teknologi Malaysia in Johor Bahru.'
>></textarea></section>
    <section><label for='redacted-text'>Redacted Text</label><textarea id='redacted-text' re
>adonly aria-live='polite' placeholder='The redacted result will appear here.'></textarea></s
>ection>
  </div>
  <div class='actions'><button id='redact' type='button'>Redact Text</button><button id='cle
>ar' type='button'>Clear</button></div>
  <p id='status' role='status' aria-live='polite'></p>
</main>
<script>
  const input=document.getElementById('input-text');
```

```
0096 |   const output=document.getElementById('redacted-text');
0097 |   const statusNode=document.getElementById('status');
0098 |   const redactButton=document.getElementById('redact');
0099 |   const clearButton=document.getElementById('clear');
0100 |   function showStatus(message,isError=false){statusNode.textContent=message;statusNode.class
.... | >Name=isError?'error':'';}
0101 |   async function redactText(){
0102 |     const text=input.value;
0103 |     output.value='';
0104 |     if(!text.trim()){showStatus('Please enter English text before redacting.',true);input.fo
.... | >cus();return;}
0105 |     redactButton.disabled=true;showStatus('Redacting text...');
0106 |     try{
0107 |       const response=await fetch('/api/redact',{method:'POST',headers:{'Content-Type':'appli
.... | >cation/json'},body:JSON.stringify({text})});
0108 |       let data;try{data=await response.json();}catch{throw new Error('The server returned an
.... | > invalid response.');}
0109 |       if(!response.ok){const detail=Array.isArray(data.detail)?data.detail.map(item=>item.ms
.... | >g).join('; '):data.detail;throw new Error(detail||`Request failed (HTTP ${response.status}).
.... | >`);}
0110 |       output.value=data.redacted_text;showStatus('Redaction complete.');
```

源码第 111-149 行

```
0111 |     }catch(error){showStatus(error.message||'Unable to contact the redaction service.',true)
.... | >;}
0112 |     finally{redactButton.disabled=false;}
0113 |   }
0114 |   function clearText(){input.value='';output.value='';showStatus('');input.focus();}
0115 |   redactButton.addEventListener('click',redactText);
0116 |   clearButton.addEventListener('click',clearText);
0117 | </script>
0118 | </body>
0119 | </html>"""
0120 |
0121 |
0122 | @app.post("/api/predict")
0123 | def predict(request: TextRequest) -> dict:
0124 |     """接收原始文本 JSON, 成功时返回 HTTP 200 与模型结果。"""
0125 |     return analyze(request.text)
0126 |
0127 |
0128 | @app.post("/api/redact", response_model=RedactResponse)
0129 | def redact(request: TextRequest) -> RedactResponse:
0130 |     """供简易网页调用; 实体识别与 /api/predict 使用同一训练模型。"""
0131 |     try:
0132 |         result = analyze(request.text)
0133 |     except (FileNotFoundError, RuntimeError) as error:
0134 |         raise HTTPException(status_code=503, detail="The trained NER model is not available"
.... | >) from error
0135 |     except HTTPException:
0136 |         raise
0137 |     except Exception as error:
0138 |         raise HTTPException(status_code=500, detail="The redaction service could not process
.... | > this text") from error
0139 |     return RedactResponse(redacted_text=result["redacted_text"])
0140 |
0141 |
0142 | @app.post("/api/upload")
0143 | async def upload(file: UploadFile = File(...)) -> dict:
0144 |     """读取 UTF-8 TXT 或带文字层 PDF; NER 仍由自己的模型完成。"""
0145 |     name = (file.filename or "").lower()
0146 |     content = await file.read(MAX_UPLOAD_BYTES + 1)
0147 |     return await run_in_threadpool(analyze_uploaded_content, name, content)
0148 |
0149 |
```

源码第 150-171 行

```
0150 | def analyze_uploaded_content(name: str, content: bytes) -> dict:
0151 |     """验证并提取上传内容; 同步函数便于测试且让模型离开事件循环。"""
0152 |     if len(content) > MAX_UPLOAD_BYTES:
0153 |         raise HTTPException(status_code=413, detail="File exceeds the 5 MB limit")
0154 |     try:
0155 |         if name.endswith(".txt"):
0156 |             text = content.decode("utf-8")
0157 |         elif name.endswith(".pdf"):
0158 |             text = "\n".join(page.extract_text() or "" for page in PdfReader(BytesIO(content
```

```
.... | >)).pages)
0159 |         else:
0160 |             raise HTTPException(status_code=415, detail="Only .txt and .pdf files are suppor
.... | >ted")
0161 |     except UnicodeDecodeError as error:
0162 |         raise HTTPException(status_code=400, detail="TXT file must use UTF-8 encoding") from
.... | > error
0163 |     except HTTPException:
0164 |         raise
0165 |     except Exception as error:
0166 |         raise HTTPException(status_code=400, detail="Could not extract file text") from erro
.... | >r
0167 |     if not text.strip():
0168 |         raise HTTPException(status_code=400, detail="The file contains no extractable text")
0169 |     if len(text) > MAX_TEXT_CHARS:
0170 |         raise HTTPException(status_code=413, detail="Extracted text exceeds the 100,000 char
.... | >acter limit")
0171 |     return analyze(text)
```

立刻运行

```
python -m py_compile app/main.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_web_ui.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-17 行

```
0001 | """Phase 18 的前端静态契约；真实浏览器点击另行验收。"""
0002 |
0003 | from app.main import home
0004 |
0005 |
0006 | def test_web_ui_is_responsive_and_has_visible_status():
0007 |     html = home()
0008 |     assert "@media(max-width:700px)" in html
0009 |     assert "role='status'" in html
0010 |     assert "Please enter English text before redacting." in html
0011 |     assert "console.log" not in html
0012 |
0013 |
0014 | def test_javascript_calls_backend_instead_of_matching_names():
0015 |     html = home()
0016 |     assert "fetch('/api/redact'" in html
0017 |     assert "keyword" not in html.lower()
```

立刻运行

```
python -m py_compile tests/test_web_ui.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

browser -> FastAPI -> Predictor -> response -> DOM；网页不执行 NER，只展示服务器模型结果。

9. Run Instructions / 直接复制的运行命令

```
python -m uvicorn app.main:app --reload
python -m pytest tests\test_acceptance.py -q -k end_to_end
```

10. Expected Results / 预期结果

网页能完成文本与文件两条路径；端到端测试通过；关闭服务用 Ctrl+C。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

在 JavaScript 硬编码实体；接口路径错；未展示服务错误；把 HTML 输入直接拼接造成注入。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

The browser only collects input and displays JSON; all entity recognition is performed by the trained PyTorch model on the server.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 19：自动化测试、Windows 调试与错误修复

1. Learning Objectives / 学习目标

运行 56 项测试，理解每个断言覆盖的需求，并用 traceback 修复根因。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

unit/integration/end-to-end test、fixture、assertion、regression、coverage、traceback。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

tests 分别调用 preprocessing/model/predict/redact/API；acceptance 测试串起真实 checkpoint 与应用。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
tests/test_acceptance.py    [本阶段创建或完整核对]
tests/test_api.py    [本阶段创建或完整核对]
tests/test_file_api.py    [本阶段创建或完整核对]
tests/test_model.py    [本阶段创建或完整核对]
tests/test_model_versions.py    [本阶段创建或完整核对]
tests/test_prediction.py    [本阶段创建或完整核对]
tests/test_preprocessing.py    [本阶段创建或完整核对]
tests/test_redaction.py    [本阶段创建或完整核对]
tests/test_text_api.py    [本阶段创建或完整核对]
tests/test_web_ui.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：tests/test_acceptance.py、tests/test_api.py、tests/test_file_api.py、tests/test_model.py、tests/test_model_versions.py、tests/test_prediction.py、tests/test_preprocessing.py、tests/test_redaction.py、tests/test_text_api.py、tests/test_web_ui.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：tests/test_acceptance.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | import torch
0002 | from torch import nn
0003 |
0004 | from src.constants import LABELS, MALAYSIAN_DIR, MODELS_DIR, PROCESSED_DIR
0005 | from src.dataset import load_jsonl
0006 | from src.metrics import entity_metrics
0007 | from src.model import BiLSTMNER
0008 | from src.predict import Predictor
0009 | from src.redact import redact_text
0010 |
0011 |
0012 | def test_dataset_loading_and_malaysian_split_integrity():
0013 |     train = load_jsonl(PROCESSED_DIR / "train.jsonl")
0014 |     local_train = load_jsonl(MALAYSIAN_DIR / "train.jsonl")
0015 |     local_test = load_jsonl(MALAYSIAN_DIR / "test.jsonl")
0016 |     assert len(train) == 2400 and len(local_train) == 160 and len(local_test) == 60
0017 |     assert {row["text"].lower() for row in local_train}.isdisjoint(
0018 |         {row["text"].lower() for row in local_test}
0019 |     )
0020 |
0021 |
0022 | def test_single_training_step_updates_parameters():
0023 |     model = BiLSTMNER(vocab_size=12, num_labels=len(LABELS), embedding_dim=8, hidden_dim=6)
0024 |     optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
0025 |     inputs = torch.tensor([[2, 3, 4]])
0026 |     features = torch.zeros((1, 3, 4))
0027 |     labels = torch.tensor([[2, 1, 4]])
0028 |     before = model.classifier.weight.detach().clone()
0029 |     loss = nn.CrossEntropyLoss()(model(inputs, features).reshape(-1, len(LABELS)), labels.re
.... | >shape(-1))
0030 |     optimizer.zero_grad()
0031 |     loss.backward()
0032 |     optimizer.step()
0033 |     assert not torch.equal(before, model.classifier.weight.detach())
0034 |
0035 |
0036 | def test_checkpoint_can_be_saved_and_loaded(tmp_path):
0037 |     model = BiLSTMNER(vocab_size=10, num_labels=len(LABELS), embedding_dim=8, hidden_dim=6)
0038 |     path = tmp_path / "checkpoint.pt"
0039 |     torch.save(model.state_dict(), path)
0040 |     restored = BiLSTMNER(vocab_size=10, num_labels=len(LABELS), embedding_dim=8, hidden_dim=
.... | >6)
```

源码第 41-60 行

```
0041 |     restored.load_state_dict(torch.load(path, map_location="cpu", weights_only=True))
0042 |     assert torch.equal(model.classifier.weight, restored.classifier.weight)
0043 |
0044 |
0045 | def test_model_evaluation_counts_entity_boundaries():
0046 |     metrics = entity_metrics(
0047 |         [["B-PER", "O", "B-LOC"]],
0048 |         [["B-PER", "O", "B-ORG"]],
0049 |     )
0050 |     assert metrics["PER"]["f1"] == 1.0
0051 |     assert metrics["LOC"]["fn"] == 1
0052 |     assert metrics["ORG"]["fp"] == 1
0053 |
0054 |
0055 | def test_checkpoint_cpu_inference_and_end_to_end_redaction():
0056 |     predictor = Predictor(MODELS_DIR / "best_model.pt")
0057 |     assert next(predictor.model.parameters()).device.type == "cpu"
0058 |     result = predictor.predict("Ahmad Ali studies at Universiti Teknologi Malaysia in Johor
.... | >Bahru.")
0059 |     redacted = redact_text(result["text"], result["entities"])
0060 |     assert redacted == "[PER] studies at [ORG] in [LOC]."
```

立刻运行

```
python -m py_compile tests/test_acceptance.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-39 行

```
0001 | from io import BytesIO
0002 |
0003 | import pytest
0004 | from fastapi import HTTPException
0005 | from pydantic import ValidationError
0006 | from reportlab.pdfgen import canvas
0007 |
0008 | import app.main as web_app
0009 | from app.main import MAX_TEXT_CHARS, TextRequest, analyze_uploaded_content, app, home, predi
.... | >ct, redact
0010 |
0011 |
0012 | def test_predict_endpoint():
0013 |     response = predict(TextRequest(text="Ahmad Ali studies at Universiti Teknologi Malaysia
.... | >in Johor Bahru."))
0014 |     assert response["redacted_text"] == "[PER] studies at [ORG] in [LOC]."
0015 |     assert response["entities"][0]["start"] == 0
0016 |
0017 |
0018 | def test_empty_text_is_rejected():
0019 |     with pytest.raises(ValidationError):
0020 |         TextRequest(text="")
0021 |     with pytest.raises(ValidationError):
0022 |         TextRequest(text="   \n")
0023 |
0024 |
0025 | def test_web_interface_is_simple_responsive_and_calls_real_redact_api():
0026 |     html = home()
0027 |     for expected in ("Student Privacy Redaction", "Input Text", "Redacted Text", "Redact Tex
.... | >t", "Clear", "fetch('/api/redact'", "readonly", "@media(max-width:700px)"):
0028 |         assert expected in html
0029 |     assert "keyword" not in html.lower()
0030 |
0031 |
0032 | def test_redact_endpoint_uses_model_and_preserves_input():
0033 |     text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
0034 |     response = redact(TextRequest(text=text))
0035 |     assert response.model_dump() == {"redacted_text": "[PER] studies at [ORG] in [LOC]."}
0036 |     assert text == "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
0037 |     assert any(route.path == "/api/redact" for route in app.routes)
0038 |
0039 |
```

源码第 40-79 行

```
0040 | @pytest.mark.parametrize("text", ["", "   ", "\n\t"])
0041 | def test_redact_endpoint_rejects_empty_or_blank_text(text):
0042 |     with pytest.raises(ValidationError):
0043 |         TextRequest(text=text)
0044 |
0045 |
0046 | def test_redact_endpoint_rejects_overlong_text():
0047 |     with pytest.raises(ValidationError):
```

```
        TextRequest(text="a" * (MAX_TEXT_CHARS + 1))


def test_redact_endpoint_handles_multiline_punctuation_and_no_entity_text():
    text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru.\nThe submitte
>d form contains no named entity."
    redacted = redact(TextRequest(text=text)).redacted_text
    assert redacted.startswith("[PER] studies at [ORG] in [LOC].\n")
    assert redacted.endswith("The submitted form contains no named entity.")


def test_redact_endpoint_supports_consecutive_requests():
    first = redact(TextRequest(text="Li Wei visited Toronto."))
    second = redact(TextRequest(text="Nur Aisyah studies at Universiti Malaya."))
    assert first.redacted_text == "[PER] visited [LOC]."
    assert second.redacted_text == "[PER] studies at [ORG]."


def test_redact_endpoint_reports_missing_model(monkeypatch):
    def fail(_text):
        raise FileNotFoundError("checkpoint missing")

    monkeypatch.setattr(web_app, "analyze", fail)
    with pytest.raises(HTTPException) as captured:
        redact(TextRequest(text="Student report"))
    assert captured.value.status_code == 503
    assert "model" in captured.value.detail.lower()


def test_txt_upload():
    response = analyze_uploaded_content("sample.txt", b"Ali Hassan studies at Universiti Tek
>nologi Malaysia.")
    assert "ORG" in {entity["type"] for entity in response["entities"]}
```

源码第 80-119 行

```

def test_text_pdf_upload():
    buffer = BytesIO()
    pdf = canvas.Canvas(buffer)
    pdf.drawString(72, 750, "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bah
>ru.")
    pdf.save()
    response = analyze_uploaded_content("sample.pdf", buffer.getvalue())
    assert {entity["type"] for entity in response["entities"]} == {"PER", "ORG", "LOC"}


def test_multi_page_pdf_upload_extracts_every_page():
    buffer = BytesIO()
    pdf = canvas.Canvas(buffer)
    pdf.drawString(72, 750, "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bah
>ru.")
    pdf.showPage()
    pdf.drawString(72, 750, "Li Wei studies at Stanford University in Toronto.")
    pdf.save()
    response = analyze_uploaded_content("two-pages.pdf", buffer.getvalue())
    assert "Ahmad Ali" in response["text"] and "Li Wei" in response["text"]
    assert response["entities"]  # 两页合并后的文本确实进入自己的模型，而非只做文件解析。


@pytest.mark.parametrize(
    ("name", "content"),
    [("empty.txt", b""), ("blank.txt", b" \r\n"), ("broken.pdf", b"not a pdf")],
)
def test_empty_and_corrupt_uploads_are_rejected(name, content):
    with pytest.raises(HTTPException) as captured:
        analyze_uploaded_content(name, content)
    assert captured.value.status_code == 400


def test_unsupported_upload():
    with pytest.raises(HTTPException) as captured:
        analyze_uploaded_content("sample.csv", b"a,b")
    assert captured.value.status_code == 415


def test_upload_size_limit():
```

```
0119 |     with pytest.raises(HTTPException) as captured:
```

源码第 120-127 行

```
0120 |         analyze_uploaded_content("large.txt", b"a" * (5 * 1024 * 1024 + 1))
0121 |     assert captured.value.status_code == 413
0122 |
0123 |
0124 | def test_extracted_text_length_limit():
0125 |     with pytest.raises(HTTPException) as captured:
0126 |         analyze_uploaded_content("long.txt", b"a" * (MAX_TEXT_CHARS + 1))
0127 |     assert captured.value.status_code == 413
```

立刻运行

```
python -m py_compile tests/test_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_file_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | """Phase 17: 文件解析与限制，不依赖最终网页。"""
0002 |
0003 | from io import BytesIO
0004 |
0005 | import pytest
0006 | from fastapi import HTTPException
0007 | from reportlab.pdfgen import canvas
0008 |
0009 | from tutorial.app_versions.app_v2_file_api import MAX_UPLOAD_BYTES, analyze_uploaded_content
0010 |
0011 |
0012 | def test_txt_and_pdf_use_the_same_model_path():
0013 |     txt = analyze_uploaded_content(
0014 |         "student.txt", b"Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
0015 |     )
0016 |     buffer = BytesIO()
0017 |     pdf = canvas.Canvas(buffer)
0018 |     pdf.drawString(72, 750, "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bah
.... | >ru.")
0019 |     pdf.save()
0020 |     document = analyze_uploaded_content("student.pdf", buffer.getvalue())
0021 |     assert txt["redacted_text"] == "[PER] studies at [ORG] in [LOC]."
0022 |     assert document["redacted_text"].strip() == "[PER] studies at [ORG] in [LOC]."
0023 |
0024 |
0025 | @pytest.mark.parametrize(
0026 |     ("name", "content", "status"),
0027 |     [("data.csv", b"x", 415), ("blank.txt", b"  ", 400)],
0028 | )
0029 | def test_file_errors_are_explicit(name, content, status):
0030 |     with pytest.raises(HTTPException) as captured:
0031 |         analyze_uploaded_content(name, content)
0032 |     assert captured.value.status_code == status
0033 |
0034 |
0035 | def test_file_size_limit():
```

```
0036 |     with pytest.raises(HTTPException) as captured:
0037 |         analyze_uploaded_content("large.txt", b"a" * (MAX_UPLOAD_BYTES + 1))
0038 |     assert captured.value.status_code == 413
```

立刻运行

```
python -m py_compile tests/test_file_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_model.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-13 行

```
0001 | import torch
0002 |
0003 | from src.model import BiLSTMNER
0004 |
0005 |
0006 | def test_forward_tensor_shape_and_gradient_flow():
0007 |     model = BiLSTMNER(vocab_size=20, num_labels=8, embedding_dim=16, hidden_dim=12)
0008 |     input_ids = torch.tensor([[2, 3, 0], [4, 5, 6]])
0009 |     features = torch.zeros((2, 3, 4))
0010 |     logits = model(input_ids, features)
0011 |     assert logits.shape == (2, 3, 8)
0012 |     logits.sum().backward()
0013 |     assert model.classifier.weight.grad is not None
```

立刻运行

```
python -m py_compile tests/test_model.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_model_versions.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | import importlib
0002 |
0003 | import torch
```

```
0004 |
0005 |
0006 | def load(version: int):
0007 |     return importlib.import_module(f"tutorial.model_versions.model_v{version}")
0008 |
0009 |
0010 | def test_version_1_instantiates_module():
0011 |     assert isinstance(load(1).BiLSTMNER(), torch.nn.Module)
0012 |
0013 |
0014 | def test_version_2_embedding_shape():
0015 |     model = load(2).BiLSTMNER(vocab_size=20, embedding_dim=16)
0016 |     assert model.embed(torch.tensor([[2, 3, 0]])).shape == (1, 3, 16)
0017 |
0018 |
0019 | def test_version_3_bilstm_shape():
0020 |     model = load(3).BiLSTMNER(vocab_size=20, embedding_dim=16, hidden_dim=12)
0021 |     assert model.encode(torch.tensor([[2, 3, 0]])).shape == (1, 3, 24)
0022 |
0023 |
0024 | def test_version_4_classifier_shape_and_train_eval_dropout():
0025 |     model = load(4).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0026 |     assert model.classify(torch.tensor([[2, 3, 0]])).shape == (1, 3, 8)
0027 |     model.eval()
0028 |     assert not model.dropout.training
0029 |
0030 |
0031 | def test_version_5_standard_forward_and_gradient():
0032 |     model = load(5).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0033 |     logits = model(torch.tensor([[2, 3, 0]]))
0034 |     logits.sum().backward()
0035 |     assert logits.shape == (1, 3, 8)
0036 |     assert model.classifier.weight.grad is not None
0037 |
0038 |
0039 | def test_version_6_features_match_final_shape():
0040 |     model = load(6).BiLSTMNER(20, 8, embedding_dim=16, hidden_dim=12)
0041 |     logits = model(torch.tensor([[2, 3, 0]]), torch.zeros((1, 3, 4)))
0042 |     assert logits.shape == (1, 3, 8)
```

立刻运行

```
python -m py_compile tests/test_model_versions.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_prediction.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | from src.predict import Predictor, extract_entities
0002 | from src.redact import redact_text
0003 |
0004 |
0005 | def test_entity_extraction_handles_malformed_initial_i_tag():
0006 |     entities = extract_entities(["John", "London"], ["I-PER", "B-LOC"])
0007 |     assert entities == [
0008 |         {"text": "John", "type": "PER", "start_token": 0, "end_token": 1},
0009 |         {"text": "London", "type": "LOC", "start_token": 1, "end_token": 2},
```

```
0010 |     ]
0011 |
0012 |
0013 | def test_saved_model_reloads_and_predicts_required_types():
0014 |     predictor = Predictor()
0015 |     result = predictor.predict("Ahmad Ali studies at Universiti Teknologi Malaysia in Johor
.... | >Bahru.")
0016 |     assert {entity["type"] for entity in result["entities"]} == {"PER", "ORG", "LOC"}
0017 |     university = next(entity for entity in result["entities"] if entity["type"] == "ORG")
0018 |     assert result["text"][university["start"]:university["end"]] == "Universiti Teknologi Ma
.... | >laysia"
0019 |
0020 |
0021 | def test_no_entity_text():
0022 |     assert Predictor().predict("The weather is beautiful today.")["entities"] == []
0023 |
0024 |
0025 | def test_punctuation_is_not_absorbed_into_entity_span():
0026 |     text = "Sarah studies at Oxford University in London!"
0027 |     result = Predictor().predict(text)
0028 |     assert all(text[entity["start"]:entity["end"]] == entity["text"] for entity in result["e
.... | >ntities"])
0029 |     assert redact_text(text, result["entities"]).endswith("[LOC]!")
0030 |
0031 |
0032 | def test_empty_long_repeated_and_multisentence_inputs_do_not_crash():
0033 |     predictor = Predictor()
0034 |     assert predictor.predict("")["entities"] == []
0035 |     repeated = predictor.predict("Sarah and Sarah joined a club at Oxford University in Lond
.... | >on.")
0036 |     assert sum(entity["text"] == "Sarah" for entity in repeated["entities"]) == 2
0037 |     long_text = " ".join(["Sarah studies at Oxford University in London."] * 200)
0038 |     result = predictor.predict(long_text)
```

源码第 39-40 行

```
0039 |     assert len(result["tokens"]) > 1_000
0040 |     assert all(long_text[e["start"]:e["end"]] == e["text"] for e in result["entities"])
```

立刻运行

```
python -m py_compile tests/test_prediction.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_preprocessing.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-38 行

```
0001 | import pytest
0002 |
0003 | from src.generate_data import ENTITY_POOLS, MALAYSIAN_POOLS
0004 | from src.preprocessing import build_vocabulary, casing_features, tokenize, tokens_to_ids, va
.... | >lidate_bio
0005 |
0006 |
0007 | def test_tokenization_preserves_punctuation_and_names():
0008 |     assert tokenize("Ali works at UTM, Johor.") == ["Ali", "works", "at", "UTM", ",", "Johor
.... | >", "."]
```

```
0009 |
0010 |
0011 | def test_vocabulary_and_unknown_word():
0012 |     vocabulary = build_vocabulary([["Ali", "works"]])
0013 |     assert tokens_to_ids(["ALI", "unseen"], vocabulary) == [vocabulary["ali"], vocabulary["<
.... | >UNK>"]]
0014 |
0015 |
0016 | def test_casing_feature_shape():
0017 |     features = casing_features(["Ali", "UTM", "works", "2026"])
0018 |     assert len(features) == 4
0019 |     assert all(len(row) == 4 for row in features)
0020 |
0021 |
0022 | def test_invalid_bio_is_rejected():
0023 |     with pytest.raises(ValueError, match="Illegal I-tag"):
0024 |         validate_bio(["Ali"], ["I-PER"])
0025 |
0026 |
0027 | @pytest.mark.parametrize("pools", [ENTITY_POOLS, MALAYSIAN_POOLS])
0028 | def test_entity_pools_are_disjoint_across_splits(pools):
0029 |     for entity_type in ("PER", "LOC", "ORG"):
0030 |         split_sets = [set(pools[split][entity_type]) for split in ("train", "validation", "t
.... | >est")]
0031 |         assert split_sets[0].isdisjoint(split_sets[1])
0032 |         assert split_sets[0].isdisjoint(split_sets[2])
0033 |         assert split_sets[1].isdisjoint(split_sets[2])
0034 |
0035 |
0036 | def test_combined_general_and_malaysian_entities_do_not_cross_splits():
0037 |     for entity_type in ("PER", "LOC", "ORG"):
0038 |         combined = {
```

源码第 39-44 行

```
0039 |             split: set(ENTITY_POOLS[split][entity_type]) | set(MALAYSIAN_POOLS[split][entity
.... | >_type])
0040 |             for split in ("train", "validation", "test")
0041 |         }
0042 |         assert combined["train"].isdisjoint(combined["validation"])
0043 |         assert combined["train"].isdisjoint(combined["test"])
0044 |         assert combined["validation"].isdisjoint(combined["test"])
```

立刻运行

```
python -m py_compile tests/test_preprocessing.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_redaction.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-28 行

```
0001 | from src.redact import redact_text, redact_tokens
0002 |
0003 |
0004 | def test_redaction_replaces_multitoken_entities_and_repairs_spacing():
0005 |     tokens = ["Ali", "studies", "at", "Universiti", "Malaya", "."]
0006 |     entities = [
```

```
        {"text": "Ali", "type": "PER", "start_token": 0, "end_token": 1},
        {"text": "Universiti Malaya", "type": "ORG", "start_token": 3, "end_token": 5},
    ]
    assert redact_tokens(tokens, entities) == "[PER] studies at [ORG]."


def test_repeated_entities_are_each_redacted():
    tokens = ["Ali", "met", "Ali", "."]
    entities = [
        {"text": "Ali", "type": "PER", "start_token": 0, "end_token": 1},
        {"text": "Ali", "type": "PER", "start_token": 2, "end_token": 3},
    ]
    assert redact_tokens(tokens, entities) == "[PER] met [PER]."


def test_character_redaction_preserves_original_whitespace():
    text = "Alice  works at Microsoft.\n"
    entities = [
        {"type": "PER", "start": 0, "end": 5},
        {"type": "ORG", "start": 16, "end": 25},
    ]
    assert redact_text(text, entities) == "[PER]  works at [ORG].\n"
```

立刻运行

```
python -m py_compile tests/test_redaction.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_text_api.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-25 行

```
"""Phase 16 只测试文本 API；文件上传留到 Phase 17。"""

import pytest
from pydantic import ValidationError

from tutorial.app_versions.app_v1_text_api import TextRequest, predict, redact


def test_text_prediction_and_redaction_use_model():
    text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru."
    full = predict(TextRequest(text=text))
    short = redact(TextRequest(text=text))
    assert {entity["type"] for entity in full["entities"]} == {"PER", "ORG", "LOC"}
    assert short.redacted_text == "[PER] studies at [ORG] in [LOC]."


def test_blank_text_is_rejected_before_model():
    with pytest.raises(ValidationError):
        TextRequest(text="  \n")


def test_predictor_is_cached_after_first_request():
    from tutorial.app_versions import app_v1_text_api

    assert app_v1_text_api.predictor is not None
```

立刻运行

```
python -m py_compile tests/test_text_api.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：tests/test_web_ui.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-17 行

```
0001 | """Phase 18 的前端静态契约；真实浏览器点击另行验收。"""
0002 |
0003 | from app.main import home
0004 |
0005 |
0006 | def test_web_ui_is_responsive_and_has_visible_status():
0007 |     html = home()
0008 |     assert "@media(max-width:700px)" in html
0009 |     assert "role='status'" in html
0010 |     assert "Please enter English text before redacting." in html
0011 |     assert "console.log" not in html
0012 |
0013 |
0014 | def test_javascript_calls_backend_instead_of_matching_names():
0015 |     html = home()
0016 |     assert "fetch('/api/redact'" in html
0017 |     assert "keyword" not in html.lower()
```

立刻运行

```
python -m py_compile tests/test_web_ui.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

tests 分别调用 preprocessing/model/predict/redact/API；acceptance 测试串起真实 checkpoint 与应用。

9. Run Instructions / 直接复制的运行命令

```
python -m pytest -q
python -m pytest tests\test_acceptance.py -q -vv
```

10. Expected Results / 预期结果

56 passed；任何失败都保留完整 traceback，修复后先重跑单项再跑全套。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

删除失败测试；用宽泛 except 掩盖异常；只在 IDE 运行导致工作目录不同；测试污染正式数据。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

I use unit tests for small contracts and acceptance tests for the complete model-to-API path.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 20：README、验收报告、GitHub 发布准备与文档同步

1. Learning Objectives / 学习目标

把真实安装、训练、测试、指标、限制和发布状态写成可审核文档。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

reproducibility、requirement traceability、artifact hash、public repository boundary。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

README 给复现者；acceptance report 给评委；presentation 给现场演示；三者必须引用同一真实证据。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
README.md    [本阶段创建或完整核对]
MODEL_ACCEPTANCE_REPORT.md    [本阶段创建或完整核对]
TUTORIAL_AUDIT_REPORT.md    [本阶段创建或完整核对]
PRESENTATION_MATERIALS.md    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：README.md、MODEL_ACCEPTANCE_REPORT.md、TUTORIAL_AUDIT_REPORT.md、PRESENTATION_MATERIALS.md。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：README.md

为什么需要它：公开仓库入口。学生需要逐段填写项目介绍、安装、数据、训练、运行、测试、架构、指标与限制，而不是写一句占位说明。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 md 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-35 行

```
0001 | # Student Privacy Protection — Personal Information Detection and Redaction
0002 |
0003 | ## Project Overview
0004 |
0005 | 这是一个面向英文学生文本的命名实体识别（NER）项目，服务于 Student Incident Reports、Support Chats、Complaints、Feedback、Un
.... | >iversity Administrative Reports 与 Student Activity Reports。模型从随机权重开始训练，由 PyTorch 自定义 `Embedd
.... | >ing + casing features + BiLSTM + Linear` 实现，识别 `PERSON (PER)`、`LOCATION (LOC)`、`ORGANISATION
.... | > (ORG)`，返回字符位置并生成脱敏文本。
0006 |
0007 | 项目不调用外部 AI/NER API，也不加载 spaCy/Hugging Face 现成 NER pipeline。仓库直接包含经过训练的 `models/best_model.pt
.... | >`，Clone 后不需要重新训练即可预测。
0008 |
0009 | ## Competition Requirements
0010 |
0011 | | Requirement | Status |
0012 | |---|---|
0013 | | 自行定义并训练的 Machine Learning Model | PASS |
0014 | | PER / LOC / ORG | PASS |
0015 | | 原始英文文本输入及类型/位置输出 | PASS |
0016 | | Public GitHub repository | 本仓库 |
0017 | | Bonus: Text Redaction | PASS |
0018 | | Bonus: Malaysian local synthetic dataset | PASS |
0019 | | Bonus: TXT/PDF input | PASS |
0020 |
0021 | ## Features
0022 |
```

- CPU 可运行的自定义 BiLSTM token classifier。
- 原始英文文本预测、BIO tags、实体类型、token span、字符 `[start,end)`。
- `[PER]`、`[LOC]`、`[ORG]` 自动脱敏。
- Malaysian 学生姓名、地点、大学、学校、学生组织与教育机构合成标注数据。
- FastAPI JSON API 和响应式单页 Web 界面（Input Text、Redact Text、只读结果、Clear）。
- UTF-8 `.txt` 与带文本层 `.pdf` 上传；5 MB 限制和错误响应。
- 56 项自动化测试（包含阶段专用 Tensor、DataLoader、training step、text API、file API 与 Web UI 检查）、13 项学生场景固定
>挑战矩阵及独立 general/Malaysian test evaluation。

Model Architecture

`token IDs -> trainable Embedding(64)` 与 `4 casing features -> Linear(8)` 拼接，进入一层双向 LSTM（每方向
> 96），经过 Dropout 0.25，最后 Linear 输出 8 个 BIO 标签 logits。参数总数 143,536。

没有使用 CRF：采用比赛允许的 BiLSTM Token Classifier 备选，以保证 CPU 和初学者环境可复现。代价是模型没有联合学习 BIO 转移；抽取器会安全修复非法
>始 I-tag。

源码第 36-77 行

Dataset

本项目使用可重现的学生场景合成英文 BIO 数据，不声称是 CoNLL-2003、真实学生记录或真实用户数据：

- General: 2,400 train / 300 validation / 300 test。
- Malaysian: 160 train / 40 validation / 60 test。
- 标签：`O, B/I-PER, B/I-LOC, B/I-ORG`。
- 语境覆盖学生报告、投诉、反馈、支持聊天、大学行政报告与学生活动。
- 完整句子与 general/Malaysian 合并实体池跨 split 隔离；句法模板仍共享，因此测试指标高估真实泛化。

数据由 `src/generate_data.py` 生成，可公开重新分发；仓库不包含受限制的第三方数据集。

Installation - Windows 11 / PowerShell

推荐使用 **Visual Studio Code**，并安装 Microsoft 发布的 Python 扩展。运行项目还需要：

- 64-bit Python 3.11 或 3.12（安装时勾选 `Add Python to PATH`）
- Git for Windows
- PowerShell（Windows 11 已包含）
- Chrome、Edge 或 Firefox 浏览器

不需要 Node.js、npm、React、Vue、数据库、NVIDIA GPU 或重新训练模型。以下命令已经按本项目结构验证：

```powershell
git clone https://github.com/sleeplessLh/NER-Personal-Information-Detection.git
cd NER-Personal-Information-Detection
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Quick Start

不需要重新训练。仓库中的 checkpoint 会在第一次请求时自动加载到 CPU：

```powershell
$env:OMP_NUM_THREADS="1"
$env:MKL_NUM_THREADS="1"
python -m uvicorn app.main:app --reload
```

源码第 78-114 行

```

打开 <http://127.0.0.1:8000>。停止服务按 `Ctrl+C`。

## Web Interface

网页只保留完成任务所需的流程：

1. 在 **Input Text** 输入或粘贴英文学生文本。
2. 点击 **Redact Text**。
3. JavaScript 将原文以 JSON 发送到 `POST /api/redact`。
4. FastAPI 使用仓库中的自训练 checkpoint 推理。
5. **Redacted Text** 只读框显示模型生成的结果。
6. 点击 **Clear** 清空两个文本框。
```

0092 |
0093 | 网页没有 JavaScript 关键词表或姓名硬编码。核心 NER 始终在后端执行。空白、超过 100,000 字符、模型不可用、服务器错误和网络失败都会显示用户可见提示
0094 |
0095 | 也可以直接测试精简 API：
0096 |
0097 | ```powershell
0098 | $body = @{ text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru." } | C
.... | >onvertTo-Json
0099 | Invoke-RestMethod -Uri http://127.0.0.1:8000/api/redact -Method Post -ContentType "applicati
.... | >on/json" -Body $body
0100 | ```
0101 |
0102 | 预期响应：
0103 |
0104 | ```json
0105 | {"redacted_text":"[PER] studies at [ORG] in [LOC]."}
0106 | ```
0107 |
0108 | ## Model Prediction
0109 |
0110 | 输入 `Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru.`，实际模型返回 `Ahmad Ali/PE
.... | >R`、`Universiti Teknologi Malaysia/ORG`、`Johor Bahru/LOC` 及字符位置。
0111 |
0112 | ```powershell
0113 | $body = @{ text = "Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Bahru." } | C
.... | >onvertTo-Json
0114 | Invoke-RestMethod -Uri http://127.0.0.1:8000/api/predict -Method Post -ContentType "applicat
.... | >ion/json" -Body $body

源码第 115-152 行

0115 | ```
0116 |
0117 | ```python
0118 | from src.predict import Predictor
0119 | result = Predictor().predict("Ahmad Ali studies at Universiti Teknologi Malaysia in Johor Ba
.... | >hru.")
0120 | print(result["entities"])
0121 | ```
0122 |
0123 | ## Text Redaction
0124 |
0125 | 上述输入的实际脱敏结果为 `[PER] studies at [ORG] in [LOC].`。题目示例句也实际得到 `[PER], a student from [ORG], att
.... | >ended an event in [LOC].`。实现按字符 span 从右向左替换，避免位置偏移。
0126 |
0127 | ## File Upload
0128 |
0129 | 后端仍保留 `POST /api/upload` 供 TXT/PDF 客户端调用。TXT 必须 UTF-8；PDF 必须有文本层；扫描件不支持 OCR；单文件最大 5 MB。为了保持本
.... | >次网页极简，主页不显示文件上传控件。
0130 |
0131 | ## Model Training
0132 |
0133 | Clone 后运行不需要训练。只有重新实验时才执行：
0134 |
0135 | ```powershell
0136 | python -m src.generate_data
0137 | python -m src.train --epochs 16 --batch-size 64 --learning-rate 0.003
0138 | python -m src.evaluate
0139 | ```
0140 |
0141 | 训练只合入 general/Malaysian train；validation 用于选模；两个 test split 不参与训练或选模。checkpoint 保存权重、vocabul
.... | >ary、labels、config、seed、数据计数和最佳 epoch。
0142 |
0143 | ## Model Evaluation
0144 |
0145 | 2026-10-29 在 Windows 11、Python 3.12.15、PyTorch 2.9.0+cpu 重新训练与评估：
0146 |
0147 | - 16 epochs；batch size 64；learning rate 0.003；Adam；seed 42。
0148 | - CPU training loop: 14.2677 seconds。
0149 | - Best epoch: 16；validation entity F1: 0.9192。
0150 | - General student synthetic test (300): P/R/F1 = 0.9843/1.0000/0.9921。
0151 | - Malaysian synthetic test (60): P/R/F1 = 1.0000/1.0000/1.0000。
0152 | - PERSON precision/recall/F1 = 1.0000/1.0000/1.0000 on both synthetic tests。

源码第 153-191 行

0153 |
0154 | 学生场景固定挑战矩阵为 13/13 通过，覆盖 Malaysian 学生姓名、大学、学校、投诉、支持聊天、反馈、活动、重复实体、多句和未知词。它仍是小型人工检查知

```
....  | >告评估。完整实际输出在 `reports/manual_acceptance.json`。
0155  |
0156  | ## Testing
0157  |
0158  | ```powershell
0159  | $env:OMP_NUM_THREADS="1"
0160  | $env:MKL_NUM_THREADS="1"
0161  | python -m pytest -q
0162  | ```
0163  |
0164  | 最新实测：`56 passed`。覆盖学生数据、合并实体池泄漏、preprocessing、vocabulary、BIO、Tensor/DataLoader 小实验、model、for
....  | >ward、loss/backpropagation、checkpoint、prediction、三类实体、标点 span、长文本、重复实体、redaction、Malaysian da
....  | >ta、TXT、单页/多页/损坏 PDF、API/Web、空白/超长输入、连续提交、模型不可用、CPU inference 与 end-to-end。
0165  |
0166  | ## Project Structure
0167  |
0168  | ```text
0169  | app/main.py                 FastAPI、Web UI、JSON API、TXT/PDF 上传
0170  | src/generate_data.py        General/Malaysian 合成 BIO 数据与 split 检查
0171  | src/preprocessing.py        Token、字符 span、BIO 校验、vocabulary
0172  | src/dataset.py              Dataset、padding、mask、word dropout
0173  | src/model.py                自定义 BiLSTM token classifier
0174  | src/train.py                Training、validation、checkpoint、history
0175  | src/evaluate.py             General/Malaysian 独立 test evaluation
0176  | src/predict.py              CPU checkpoint loading、BIO/entity extraction
0177  | src/redact.py               基于字符 span 的可靠脱敏
0178  | tests/                      56 项自动化测试
0179  | models/best_model.pt        已训练 checkpoint；Clone 后直接使用
0180  | data/processed/             可分发的合成 general 数据
0181  | data/malaysian/             可分发的合成 Malaysian 数据
0182  | reports/                    训练与评估 JSON 证据
0183  | MODEL_ACCEPTANCE_REPORT.md  完整工程验收记录
0184  | TUTORIAL_AUDIT_REPORT.md    22 阶段零基础教程审计与复现边界
0185  | ```
0186  |
0187  | ## Limitations
0188  |
0189  | - 模型针对学生报告、学生支持聊天、投诉、反馈、大学行政报告和学生活动进行优化；不提供医院病人、公司员工或客户场景的专项保证。
0190  | - 合成模板数据不等于真实学生 incident report、support chat 或 CoNLL-2003。
0191  | - PER/LOC/ORG 不覆盖电话、邮箱、Student ID、Passport、完整地址等所有 PII。
```

源码第 192-206 行

```
0192  | - 无 CRF；非法 BIO 转移主要由后处理修复。
0193  | - 不支持扫描 PDF OCR。
0194  | - 高风险大学、医院或企业资料必须人工复核，尤其要防止姓名漏检。
0195  |
0196  | ## Reproducibility
0197  |
0198  | - 依赖固定在 `pyproject.toml`、`uv.lock`、`requirements.txt`。
0199  | - seed 42；每 epoch 指标在 `reports/training_log.json`。
0200  | - 测试指标在 `reports/test_metrics.json` 与 `reports/malaysian_test_metrics.json`。
0201  | - checkpoint 内含 vocabulary、label mapping 与 model configuration，不依赖外部下载。
0202  | - 完整证据和 Clone 验收状态见 `MODEL_ACCEPTANCE_REPORT.md`。
0203  |
0204  | ## License
0205  |
0206  | 项目源码使用 [MIT License](LICENSE)。仓库中的 general 与 Malaysian 数据均由本项目脚本生成，是用于教学和工程验收的合成数据，不是 CoNLL-
....  | >2003 或真实个人资料。
```

立刻运行

```
Get-Content README.md | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：MODEL_ACCEPTANCE_REPORT.md

为什么需要它：正式验收证据。必须区分软件测试、ML 指标、人工演示、失败案例、限制与未完成的公开发布。

打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到

Python Console。

复制规则：以下是 md 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-37 行

```
0001 | # MODEL ACCEPTANCE REPORT — STUDENT PRIVACY PROTECTION
0002 |
0003 | 执行日期：2026-10-29（Asia/Singapore）
0004 | 仓库：<https://github.com/sleeplessLh/NER-Personal-Information-Detection>
0005 |
0006 | ## 1. Scope
0007 |
0008 | 项目应用范围已限定为 **Student Personal Information Detection and Redaction**，重点处理英文 Student Incident
.... | >Reports、Student Support Chats、Student Complaints、Student Feedback、University Administrative
.... | >Reports 与 Student Activity Reports。
0009 |
0010 | 模型仍识别比赛要求的 `PERSON`、`LOCATION`、`ORGANISATION`，没有简化成只识别学生姓名。医院病人、公司员工、客户等非学生场景不再作为专项优化目标。
0011 |
0012 | ## 2. Model and Training
0013 |
0014 | - Architecture：Embedding(64) + casing projection(4→8) + one-layer BiLSTM(96/方向) + Dropout(0.
.... | >25) + Linear(192→8)
0015 | - Parameters：143,536
0016 | - Labels：`<PAD>`, O, B/I-PER, B/I-LOC, B/I-ORG`
0017 | - Training：general student 2,400 + Malaysian student 160
0018 | - Validation：general student 300 + Malaysian student 40
0019 | - Test：general student 300 + Malaysian student 60；不参与训练或选模
0020 | - Adam；learning rate 0.003；batch size 64；seed 42
0021 | - Epochs：16；best epoch：16
0022 | - Best validation entity F1：0.919245
0023 | - Windows CPU training-loop time：14.2677 seconds
0024 | - Checkpoint：`models/best_model.pt`，586,229 bytes
0025 | - SHA-256：`C0F68A78DD13599229EDA6C42799C84D68FF3942FAC600E8B5626571DB6F3575`
0026 |
0027 | Checkpoint 含 weights、vocabulary、labels、model configuration、seed、dataset counts、best epoch/F1
.... | >。模型从随机权重训练，不调用外部 NER API，不加载现成 NER pipeline。
0028 |
0029 | ## 3. Student Data
0030 |
0031 | 训练语境覆盖学生报告、投诉、反馈、支持聊天、大学行政报告、学生组织和学生活动。实体池重点覆盖 Malaysian student names、universities、schools、
.... | >student organisations 和 Malaysian locations。
0032 |
0033 | 数据生成阶段和自动化测试同时验证：
0034 |
0035 | - 完整句子跨 split 隔离
0036 | - general 与 Malaysian 合并实体池跨 train/validation/test 隔离
0037 | - test 不参与训练或 validation 选模
```

源码第 38-76 行

```
0038 | - 数据为透明的合成数据，不声称等同于真实学生记录
0039 |
0040 | ## 4. Evaluation
0041 |
0042 | 实际执行 `python -m src.evaluate`：
0043 |
0044 | | Dataset | PER P/R/F1 | LOC P/R/F1 | ORG P/R/F1 | Overall P/R/F1 |
0045 | |---|---:|---:|---:|---:|
0046 | | General student synthetic test (300) | 1.0000 / 1.0000 / 1.0000 | 0.9408 / 1.0000 / 0.9695
.... | >| 1.0000 / 1.0000 / 1.0000 | 0.9843 / 1.0000 / 0.9921 |
0047 | | Malaysian student synthetic test (60) | 1.0000 / 1.0000 / 1.0000 | 1.0000 / 1.0000 / 1.0000
.... | > | 1.0000 / 1.0000 / 1.0000 | 1.0000 / 1.0000 / 1.0000 |
0048 |
0049 | Overall F1 ≥ 0.75 与 PERSON Recall ≥ 0.80 的内部目标均达到。模板仍跨 split 共享，因此指标会高估真实学生文本泛化。
0050 |
0051 | ## 5. Student Challenge Matrix
0052 |
0053 | 实际执行 `python scripts/run_manual_acceptance.py`：**13/13 passed**。
0054 |
0055 | 覆盖：
0056 |
0057 | - Malaysian 与国际学生姓名
0058 | - Malaysian university、school 与 location
```

```
- Student complaint、support chat、feedback、activity、administrative report
- Mixed PER/LOC/ORG
- No entity
- Repeated student names
- Multiple sentences
- Punctuation
- Unknown student/university/location names

全部预测实体满足 `original_text[start:end] == entity_text`。完整输出保存在 `reports/manual_acceptance.json`。

题目示例实际输出：

```text
Original: Ahmad Ali, a student from Universiti Teknologi Malaysia, attended an event in Joho
>r Bahru.
Redacted: [PER], a student from [ORG], attended an event in [LOC].
```

6. Software, Web and File Tests
```

源码第 77-116 行

```

- Automated tests：**56 passed**（加入阶段专用学习实验与 API/Web 回归后）
- Windows browser：Student Privacy Protection 页面正常显示
- 浏览器点击 Detect：3 entities，PER/ORG/LOC 表格与 offset 正确
- 浏览器 Redacted Text：题目示例完全匹配
- `POST /api/predict`：HTTP 200，3 entities
- Student-report TXT：HTTP 200，脱敏正确
- Two-page student PDF：HTTP 200，两页均提取并送入自训练模型
- Empty/corrupt/unsupported/oversized files：400/415/413 回归测试覆盖
- CPU inference：PASS
- Scanned PDF OCR：NOT CLAIMED

GitHub 提交 `067b77cc4db19dffec330581f9afc0bd422730bf` 推送后，从公开远程仓库全新 Clone 到独立目录，并在新虚拟环境安装 55
>个锁定依赖。该 Clone 的实测结果：

- `pytest -q`：**56 passed**（零基础阶段复现的新虚拟环境）
- 两套 `src.evaluate` 指标与上表逐项一致
- student manual matrix：**13/13 passed**
- checkpoint SHA-256 与开发目录一致
- checkpoint CPU 加载和题目示例：3 个实体与精确脱敏文本一致
- 真实 Uvicorn localhost：JSON/TXT/PDF 均 HTTP 200
- 两页 PDF 的文本均成功提取，并由自训练模型推理
- 教学 PDF 存在于 Clone，131 页

## 7. Changes for Student Scope

1. README、FastAPI title、Web description、demo text 与 presentation 改为 Student Privacy Protecti
>on。
2. general 与 Malaysian 数据池改为学生姓名、大学、学校、学生组织、教育机构和地点。
3. 模板改为 incident、complaint、feedback、support、activity 和 administration 语境。
4. 增加 general/Malaysian 合并实体池跨 split 泄漏保护。
5. 增加单词姓名与重复学生姓名结构，不使用姓名硬编码替代训练。
6. 重新生成全部数据，实际训练 16 epochs 并替换 checkpoint。
7. 将 API、Web、prediction、redaction、TXT、PDF 测试示例切换为学生场景。
8. 将固定人工验收矩阵完全重写为 13 个学生隐私场景。

## 8. Acceptance Matrix

| Item | Status | Evidence |
|---|---|---|
| Student scope | PASS | README/data/UI/tests updated |
| Trained ML checkpoint | PASS | 16 epochs CPU; checkpoint load |
```

源码第 117-143 行

```
| PERSON | PASS | synthetic recall 1.0; student matrix |
| LOCATION | PASS | general F1 0.9695; Malaysian F1 1.0 |
| ORGANISATION | PASS | both synthetic F1 1.0 |
| Student redaction | PASS | exact Ahmad Ali example |
| Malaysian student data | PASS WITH LIMITATION | trained/evaluated synthetic dataset |
| TXT/PDF | PASS | real localhost requests |
| Web interface | PASS | real browser click and visible table |
| Automated tests | PASS | 56 passed |
| GitHub update | PASS | public main commit `067b77c` |
| Final remote re-clone | PASS | new venv; 39 tests; evaluate; 13 cases; JSON/TXT/PDF HTTP |
| Tutorial PDF | PASS | 131/131 rendered; 22 phases; no missing glyph/replacement/empty text p
```

```
.... | >age |
0128 | | Production deployment | NOT APPROVED | requires real student data and human review |
0129 |
0130 | ## 9. Limitations
0131 |
0132 | - 合成学生文本不等于真实大学 incident reports 或 support chats。
0133 | - 小型 13-case challenge matrix 不等于大规模独立真实语料评估。
0134 | - PER/LOC/ORG 不覆盖 email、phone、student ID、passport、完整地址等所有学生 PII。
0135 | - 扫描 PDF 无 OCR。
0136 | - 高风险学生资料必须人工复核，尤其要防止姓名漏检。
0137 | - 非学生场景没有专项性能保证。
0138 |
0139 | ## 10. Current Verdict
0140 |
0141 | **PASS — Student Privacy Protection 比赛原型的代码、训练模型、评估、Web、TXT/PDF、教学 PDF、GitHub 发布和远程全新 Clone
.... | >均通过验收。**
0142 |
0143 | 该 PASS 只适用于当前比赛/教学原型范围；由于训练与测试数据主要为合成学生文本，仍不批准直接用于真实高风险学生资料的无人复核生产脱敏。
```

立刻运行

```
Get-Content MODEL_ACCEPTANCE_REPORT.md | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：TUTORIAL_AUDIT_REPORT.md

为什么需要它：逐 Phase 教学缺陷、修复、复现状态和证据边界；明确区分工程模拟与真实人类 novice study。打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 md 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-31 行

```
0001 | # TUTORIAL AUDIT REPORT
0002 |
0003 | 审计日期：2026-10-29（Asia/Singapore）
0004 | 对象：`NER_From_Zero_To_Complete_Project_CN.pdf` 与 GitHub `main`
0005 | 学生假设：会基础 Python；不知道 ML、PyTorch、NER、虚拟环境或部署。
0006 |
0007 | ## 1. 审计方法与证据边界
0008 |
0009 | 本次没有把“PDF 很长”或“最终 pytest 通过”当作教学通过。执行了四层验收：
0010 |
0011 | 1. **知识顺序审计**：检查新概念是否在首次正式使用前回答是什么、为什么、项目职责、不使用的后果、代码、命令与成功标准。
0012 | 2. **阶段文件审计**：PDF 的 `PHASE_FILES` 每阶段只交付当时可见文件；禁止 Phase 16 提前展示最终文件上传与网页实现。
0013 | 3. **阶段式空目录复现**：`scripts/student_reproduction_audit.py` 在 `tmp/NER_Tutorial_Reproduction_Tes
.... | >t_2` 中按 Phase 00→21 累积教程文件并逐阶段执行 checkpoint；没有一次复制最终项目，也没有复制开发目录 checkpoint/data/reports。
0014 | 4. **独立环境/部署验收**：复现目录新建 `.venv`，从 `requirements.txt` 安装 55 packages，`uv pip check` 通过，执行 56
.... | >tests；使用该目录实际训练出的 checkpoint 启动 Uvicorn，并在真实浏览器点击 Redact Text。
0015 |
0016 | 自动化逐字转录只能证明教程给出的代码和命令足以构建，不等同于真实一年级学生的阅读速度或理解程度。尚未招募独立人类学生做可用性研究，因此“真实人类
.... | >ERIFIED`，不虚构 PASS。
0017 |
0018 | ## 2. 初始版本发现的关键缺陷
0019 |
0020 | | ID | 初始缺陷 | 严重度 | 修复 |
0021 | |---|---|---:|---|
0022 | | D01 | Phase 06–08、10–21 重复模板话术，概念没有逐阶段具体解释 | Critical | 为 16 个阶段增加专属概念、最小实验、写入顺序、成功条件和错误恢复
.... | > |
0023 | | D02 | Phase 21 把 PowerShell 命令放进 Python block，还要求 `py_compile` | Critical | 改成真实 `build_pd
.... | >f.py` Python 代码；PowerShell 验收命令独立显示 |
0024 | | D03 | Phase 05 generator 依赖 preprocessing，但 PDF 当时未交付该文件 | Critical | Phase 05 同时完整列出 `pre
.... | >processing.py` 与 `generate_data.py` |
```

0025 | | D04 | Phase 07/08/10 的 checkpoint 引用后续或职责过大的测试 | Critical | 新增 `tensor_demo.py`、`dataloade
.... | >r_demo.py`、`training_step_demo.py`，当阶段即可运行 |
0026 | | D05 | Phase 16 提前展示最终 `app/main.py`，破坏 API→文件→网页增量 | Critical | 新增 `app_v1_text_api.py`、`a
.... | >pp_v2_file_api.py`；Phase 18 才进入最终网页 |
0027 | | D06 | Phase 16/17 没有职责隔离的测试 | High | 新增 `test_text_api.py` 与 `test_file_api.py`；Phase 18 新
.... | >增 `test_web_ui.py` |
0028 | | D07 | Web 章节未与最新 `/api/redact`、只读输出、Clear 和可见错误同步 | High | 更新完整 app、调用流程、浏览器 checkpoint 与
.... | >README |
0029 | | D08 | 原“复现脚本”一次复制最终项目，不能证明阶段可执行 | Critical | 新增阶段式 reproduction audit，真实重新生成 data、训练 16 ep
.... | >ochs、评估和部署 |
0030 | | D09 | 测试数字和 PDF 证据停留在 47 | Medium | 全部同步为当前 56 tests，并重新生成快照、PDF 与 QA |
0031 | | D10 | 没有明确区分技术复现与真实人类学习实验 | High | 报告明确 evidence boundary；人类 novice study 标记 NOT VERIFIED

源码第 31-57 行

.... | >|
0032 |
0033 | ## 3. Phase-by-Phase 结果
0034 |
0035 | 状态只使用 PASS / FAIL / NOT VERIFIED。
0036 |
0037 | | Phase | Phase Name | Missing Knowledge / Code / Commands（初始） | Fixes Applied | Reproductio
.... | >n Status |
0038 | |---:|---|---|---|---|
0039 | | 00 | 项目边界与成果 | 无关键缺失 | 保留范围、限制、学习方法与最终示例 | PASS |
0040 | | 01 | Windows/Python/VS Code/venv | 无关键缺失 | 复测 Python 版本、PATH、venv、pip、CPU PyTorch、首文件 | PA
.... | >SS |
0041 | | 02 | Python 基础 | 无关键缺失 | class/object/type hint 与实体 dict 实验可运行 | PASS |
0042 | | 03 | NLP/NER/BIO | 无关键缺失 | token/entity/sequence/BIO 边界小实验可运行 | PASS |
0043 | | 04 | 骨架/config/Git | 快照没有代表 PDF 全文件 | 以 PDF PHASE_FILES 为准交付 pyproject/.gitignore/constant
.... | >s | PASS |
0044 | | 05 | Dataset | generator 的 preprocessing 前置断层 | 同阶段交付 preprocessing；真实生成 2400/300/300 与 16
.... | >0/40/60 | PASS |
0045 | | 06 | Token/BIO/Vocabulary | 通用模板；测试文件未在阶段展示 | 专属 span/UNK/PAD/BIO/casing 解释并交付 test_prepro
.... | >cessing | PASS |
0046 | | 07 | Tensor/ID/Padding | 直接进入 Dataset，缺独立 dtype/shape 观察 | 新增 tensor_demo；明确 long/float/bo
.... | >ol 与 (L,4) | PASS |
0047 | | 08 | Dataset/DataLoader | checkpoint 依赖后续 model test | 新增 dataloader_demo；验证 padding/mask，
.... | >无模型依赖 | PASS |
0048 | | 09 | Embedding/LSTM/BiLSTM | 无关键缺失 | 保留 V1→V6 六个真实可运行版本与 shape tests | PASS |
0049 | | 10 | Loss/Backprop | 通用片段且引用 acceptance test | 新增独立 training_step_demo；真实 weight change |
.... | >PASS |
0050 | | 11 | Epoch/Validation/Checkpoint | Malaysian merge 的数据来源说明不足 | 补 train/eval/no_grad/checkp
.... | >oint 契约与数据计数；真实训练 16 epochs | PASS |
0051 | | 12 | Loading/Prediction/BIO decode | 通用模板 | 增加 CPU map、eval/no_grad、argmax、char span 逐步数据流
.... | > | PASS |
0052 | | 13 | Redaction | 通用模板 | 增加半开区间、逆序替换、空格/换行实验 | PASS |
0053 | | 14 | Malaysian integration | 合成伦理与 test 隔离解释不足 | 增加 160/40/60 检查、checkpoint counts 与泄漏边界 |
.... | > PASS |
0054 | | 15 | Evaluation | Precision/Recall/F1 只有定义 | 增加数值实验、exact span、token accuracy 陷阱与双 test 报告
.... | > | PASS |
0055 | | 16 | Text API | 提前展示最终 app；缺独立 API 版本 | 创建 app_v1 + test_text_api；真实 request/redact/cache
.... | >教学 | PASS |
0056 | | 17 | TXT/PDF | 与最终 app 混杂 | 创建 app_v2 + test_file_api；明确 bytes/UTF-8/PDF/OCR/400/413/415 |
.... | > PASS |
0057 | | 18 | Web Interface | 旧 Detect/Upload UI 与最新简易 UI 不一致 | 最终 app 只保留 Input/Redact/Output/Clea
.... | >r；真实浏览器点击 | PASS |

源码第 58-92 行

0058 | | 19 | Tests/Debug | 通用模板；没有阶段测试层级 | 明确 unit/integration/E2E 与失败缩小顺序；56 tests | PASS |
0059 | | 20 | README/GitHub | 测试数与环境说明旧 | 更新 IDE、依赖、启动、API、证据和 GitHub 边界 | PASS |
0060 | | 21 | Final reproduction | 无效 Python block；旧脚本整包复制 | 改为阶段式审计、新 venv、训练、测试、Uvicorn、浏览器、PDF Q
.... | >A | PASS |
0061 |
0062 | ## 4. Fresh Reproduction Evidence
0063 |
0064 | - 目录：`tmp/NER_Tutorial_Reproduction_Test_2`
0065 | - 阶段：22/22 PASS；逐阶段证据：`reports/student_phase_reproduction.json`
0066 | - Dataset：在复现目录由 Phase 05 命令重新生成
0067 | - Model：在复现目录 CPU 重新训练 16 epochs；没有复制开发 checkpoint
0068 | - Environment：全新 `.venv`；55 packages installed；`uv pip check` PASS
0069 | - Tests：56 passed in 21.80s（新环境）
0070 | - Evaluation：general student 300 + Malaysian student 60
0071 | - Deployment：复现目录的 Uvicorn `127.0.0.1:8776`
0072 | - Browser：输入 Ahmad Ali 示例，实际显示 `[PER] studies at [ORG] in [LOC].`
0073 | - PDF：最终版 141/141 页已渲染；22 个 Phase 标题齐全，0 个替代字符、0 个空白文本页；结果见 `reports/tutorial_pdf_qa.json`
0074 |

```
0075 | ## 5. Final Criteria
0076 |
0077 | | Criterion | Status | Evidence / Remaining Issue |
0078 | |---|---|---|
0079 | | A Learning Completeness | PASS（结构审计） | 新概念、实验、正式代码、成功/失败条件均存在 |
0080 | | B Coding Completeness | PASS | 阶段式目录 22/22 |
0081 | | C Environment Completeness | PASS | 新 `.venv`、55 dependencies、pip check |
0082 | | D Training Completeness | PASS | 新目录真实 16-epoch CPU training |
0083 | | E Deployment Completeness | PASS | 新目录 Uvicorn + real browser |
0084 | | F Documentation Consistency | PASS | 22 Phase validator、20 path synchronization 与 141-page
.... | > PDF QA 均已通过 |
0085 | | G Fresh Reproduction | PASS（工程模拟） | 逐阶段而非整包复制；真实人类学习实验不包含在内 |
0086 | | Independent novice human study | NOT VERIFIED | 当前没有独立一年级学生参与者 |
0087 |
0088 | ## 6. Audit Verdict
0089 |
0090 | **PARTIAL PASS**：从空目录开始的阶段式工程复现、全新环境安装、重新训练、56 tests、评估和浏览器部署均已通过；但是“真实零 ML 基础的人类学生无需任
.... | >解并完成”尚未由独立人类参与者实测，因此不能诚实宣布完整 PASS。
0091 |
0092 | 若要把最后一项升级为 PASS，必须安排至少一名符合画像的学生只使用最终 PDF 操作，记录每次求助、错误与耗时；出现任何必须外部搜索或猜测的步骤就回到相
```

立刻运行

```
Get-Content TUTORIAL_AUDIT_REPORT.md | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：PRESENTATION_MATERIALS.md

为什么需要它：5-10 分钟英文讲稿、现场演示顺序、架构解释和面试问答的完整可复制版本。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 md 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-17 行

```
0001 | # Presentation Materials
0002 |
0003 | ## 5-10 Minute English Demo Script
0004 |
0005 | **0:00-0:45 - Problem.** Universities need to remove student personal information from incid
.... | >ent reports, support chats, complaints, feedback and activity reports before sharing them. M
.... | >anual redaction is slow and a missed student name can expose a person. My system detects peo
.... | >ple, locations and organisations, then returns structured spans and a redacted copy.
0006 |
0007 | **0:45-1:45 - Data.** I generated a transparent synthetic corpus with BIO labels. The genera
.... | >l split contains 2,400 training, 300 validation and 300 test sentences. I also created a Mal
.... | >aysian set with 160 training, 40 validation and 60 test sentences. Entity pools and complete
.... | > sentences are separated across splits. This avoids direct leakage, but all splits still sha
.... | >re templates, so the evaluation is easier than real data.
0008 |
0009 | **1:45-3:00 - Architecture.** Each token becomes a trainable 64-dimensional embedding. Four
.... | >casing features are projected to eight dimensions. A bidirectional LSTM reads context in bot
.... | >h directions with 96 hidden units per direction. A linear layer produces eight BIO-label log
.... | >its. I used the allowed BiLSTM token-classifier fallback rather than a CRF to keep CPU train
.... | >ing and beginner reproduction simple.
0010 |
0011 | **3:00-4:00 - Training.** I trained from random weights for sixteen epochs using Adam, learn
.... | >ing rate 0.003 and batch size 64. Cross-entropy ignores padding. Word dropout reduces simple
.... | > entity memorisation. Validation entity F1 selected epoch sixteen. The model has 143,536 par
.... | >ameters and the latest measured training loop took about 14.27 seconds on CPU in this enviro
.... | >nment.
0012 |
0013 | **4:00-5:00 - Evaluation.** Student-domain general synthetic test F1 was 0.9921 and Malaysia
.... | >n synthetic test F1 was 1.0; person recall was 1.0 on both. A separate thirteen-case student
.... | > challenge matrix also passed, but I do not present these small synthetic evaluations as pro
```

```
.... | >duction performance because the syntax templates are shared.
0014 |
0015 | **5:00-6:30 - Live demo.** Start Uvicorn, open the page, paste `Ahmad Ali studies at Univers
.... | >iti Teknologi Malaysia in Johor Bahru.`, click Detect, point to each entity's type and chara
.... | >cter offsets, and show `[PER] studies at [ORG] in [LOC].`. Then upload a student-report UTF-
.... | >8 TXT and a text-layer PDF. Explain that scanned PDFs need OCR and are rejected when no text
.... | > can be extracted.
0016 |
0017 | **6:30-7:30 - Testing and safety.** Show `56 passed`. Explain visible Web errors, the 5 MB b
.... | >ackend upload limit, invalid input handling, checkpoint reload, CPU inference, exact entity-
.... | >level metrics, and character-span redaction. Emphasise that this student-focused prototype d
.... | >oes not detect phone numbers, emails, IDs, passports or all addresses, and high-risk use nee
```

源码第 17-35 行

```
.... | >ds human review.
0018 |
0019 | ## Likely Interview Questions and Suggested Answers
0020 |
0021 | 1. **Why BiLSTM?** It lets every token representation use both left and right context.
0022 | 2. **Why BIO?** It represents both entity type and multi-token boundaries.
0023 | 3. **Why not token accuracy?** Most tokens are O, so a weak model can look accurate while mi
.... | >ssing entities.
0024 | 4. **What does `loss.backward()` do?** It computes gradients; it does not update parameters.
0025 | 5. **What does `optimizer.step()` do?** It updates parameters using the gradients.
0026 | 6. **Why call `zero_grad()`?** PyTorch accumulates gradients by default.
0027 | 7. **Why ignore padding in loss?** Padding is artificial and must not influence learning.
0028 | 8. **How do you avoid test leakage?** Test labels never select checkpoints; entity pools and
.... | > full sentences are split.
0029 | 9. **Why is F1 still suspiciously high?** The grammar templates are shared and much simpler
.... | >than real language.
0030 | 10. **Why no CRF?** The specification permits a BiLSTM token classifier fallback. It is easi
.... | >er to reproduce, but lacks learned transition constraints.
0031 | 11. **How are offsets calculated?** Regex match spans are stored as half-open character inte
.... | >rvals `[start,end)`.
0032 | 12. **Why redact right-to-left?** Earlier character offsets remain valid after replacing lat
.... | >er spans.
0033 | 13. **Can this replace human review?** No. The manual failure proves it can miss names.
0034 | 14. **What would you improve next?** Use licensed real data such as CoNLL-2003, add diverse
.... | >Malaysian human annotations, implement CRF, and evaluate by domain.
0035 | 15. **What makes the model yours?** The architecture is defined and trained in this reposito
.... | >ry from random weights; no ready-made NER model or API performs prediction.
```

立刻运行

```
Get-Content PRESENTATION_MATERIALS.md | Select-Object -First 10
```

预期结果：终端显示文件前十行，用来确认路径、编码和保存成功。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

README 给复现者；acceptance report 给评委；presentation 给现场演示；三者必须引用同一真实证据。

9. Run Instructions / 直接复制的运行命令

```
python -m pytest -q
python scripts\verify_tutorial.py
git diff --check
```

10. Expected Results / 预期结果

56 passed；路径同步；无 diff whitespace error；GitHub commit 可验证。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

复制旧指标；把未执行步骤写成 PASS；遗漏限制；提交 checkpoint 却没有重训方法；虚构公开 URL。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

My documentation separates verified evidence, known limitations, and published artifacts.

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。

Phase 21：最终复现、30 天计划、英文演示与面试

1. Learning Objectives / 学习目标

在新目录执行静态同步、测试、评估、预测和 PDF 全页检查，并安排 30 天学习节奏。 本阶段结束时，学生必须能独立说明输入、输出、失败条件和验收证据。

2. Prerequisite Knowledge / 前置知识

clean-room reproduction、static synchronization、render QA、demo fallback、final acceptance。不要求预先掌握；下面先用具体数据解释，再进入正式文件。

3. Concept Explanation / 概念解释

verified source -> reproduction directory -> tests/evaluation/prediction -> rendered PDF -> QA report。 请先在纸上画出箭头，并在箭头旁写真实函数或数据文件名。

4. Project Structure / 本阶段文件变化

```
scripts/student_reproduction_audit.py    [本阶段创建或完整核对]
scripts/reproduce_tutorial.py    [本阶段创建或完整核对]
scripts/validate_tutorial.py    [本阶段创建或完整核对]
scripts/verify_tutorial.py    [本阶段创建或完整核对]
scripts/qa_tutorial_pdf.py    [本阶段创建或完整核对]
```

未列出的文件保持上一阶段通过测试的状态。不要提前复制后续阶段文件。

5. Environment and Tools / 操作环境

在 VS Code 打开项目根目录；Terminal -> New Terminal 选择 PowerShell；确认 prompt 前有 (.venv)，并执行 `python -c "import sys; print(sys.executable)"`。输出路径必须指向项目内 `.venv\Scripts\python.exe`。随后按顺序创建或修改：scripts/student_reproduction_audit.py、scripts/reproduce_tutorial.py、scripts/validate_tutorial.py、scripts/verify_tutorial.py、scripts/qa_tutorial_pdf.py。

6. Incremental Code / 逐文件输入真实代码

下面不是伪代码。按显示顺序逐块输入；每块保存后先运行语法检查。若已有文件，按行号核对修改，不要把整章一次性粘贴后才排错。

本阶段完整可复制文件

这一节把本阶段涉及的真实验收文件完整列出。它补足前面 Lesson 中的增量练习：学生可以先按 Lesson 逐步输入，再用完整文件逐行核对。任何省略都会让后续训练或测试失去可复现性。

完整文件：scripts/student_reproduction_audit.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-42 行

```
0001 | """按 PDF 的阶段文件清单累积构建新目录，并逐阶段执行 checkpoint。
0002 |
0003 | 此脚本只用于审计：copy2 等价于学生从 PDF 的“完整可复制文件”逐字输入，
0004 | 不会一次复制整个项目，也不会复制开发目录的 checkpoint、data 或 reports。
0005 | 环境安装、真实浏览器点击和概念口述仍需单独人工验收。
0006 | """
0007 |
0008 | from __future__ import annotations
0009 |
0010 | import json
0011 | import os
0012 | import shutil
0013 | import subprocess
0014 | import sys
0015 | from pathlib import Path
0016 |
0017 | ROOT = Path(__file__).resolve().parents[1]
0018 | sys.path.insert(0, str(ROOT))
0019 |
0020 | from tutorial.build_pdf import PHASE_FILES
0021 |
0022 | TARGET = ROOT / "tmp" / "NER_Tutorial_Reproduction_Test_2"
0023 | REPORT = ROOT / "reports" / "student_phase_reproduction.json"
0024 |
0025 |
0026 | def run(command: list[str], cwd: Path = TARGET) -> dict[str, object]:
0027 |     environment = os.environ.copy()
0028 |     environment["OMP_NUM_THREADS"] = "1"
```

```
    environment["MKL_NUM_THREADS"] = "1"
    completed = subprocess.run(
        command,
        cwd=cwd,
        env=environment,
        text=True,
        encoding="utf-8",
        errors="replace",
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=180,
        check=False,
    )
    return {
```

源码第 43-78 行

```
        "command": subprocess.list2cmdline(command),
        "exit_code": completed.returncode,
        "output": completed.stdout[-4000:],
    }


def copy_phase_files(phase: str) -> list[str]:
    copied: list[str] = []
    for relative in PHASE_FILES[phase]:
        source = ROOT / relative
        if not source.is_file():
            raise FileNotFoundError(f"Phase {phase} tutorial file missing: {relative}")
        destination = TARGET / relative
        destination.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(source, destination)
        copied.append(relative)
    return copied


def checkpoints(python: str) -> dict[str, list[str]]:
    return {
        "00": [python, "-c", "from pathlib import Path; assert Path('README.md').is_file()"]
>,
        "01": [python, "learning/hello_python.py"],
        "02": [python, "learning/python_foundations.py"],
        "03": [python, "learning/bio_demo.py"],
        "04": [python, "-c", "from src.constants import LABELS; assert len(LABELS)==8; print
>(LABELS)"],
        "05": [python, "-m", "src.generate_data"],
        "06": [python, "-m", "pytest", "tests/test_preprocessing.py", "-q"],
        "07": [python, "-m", "learning.tensor_demo"],
        "08": [python, "-m", "learning.dataloader_demo"],
        "09": [python, "-m", "pytest", "tests/test_model_versions.py", "-q"],
        "10": [python, "-m", "learning.training_step_demo"],
        "11": [python, "-m", "src.train", "--epochs", "16", "--batch-size", "64", "--learnin
>g-rate", "0.003"],
        "12": [python, "-c", "from src.predict import Predictor; print(Predictor().predict('
>Li Wei visited Toronto.')['entities'])"],
        "13": [python, "-m", "pytest", "tests/test_redaction.py", "-q"],
        "14": [python, "-c", "import torch; c=torch.load('models/best_model.pt',weights_only
>=False); print(c['dataset_counts']); assert c['dataset_counts']['malaysian_training']==160"]
>,
```

源码第 79-117 行

```
        "15": [python, "-m", "src.evaluate"],
        "16": [python, "-m", "pytest", "tests/test_text_api.py", "-q"],
        "17": [python, "-m", "pytest", "tests/test_file_api.py", "-q"],
        "18": [python, "-m", "pytest", "tests/test_web_ui.py", "-q"],
        "19": [python, "-m", "pytest", "-q"],
        "20": [python, "-c", "from pathlib import Path; assert all(Path(x).is_file() for x i
>n ['README.md','MODEL_ACCEPTANCE_REPORT.md','PRESENTATION_MATERIALS.md'])"],
        "21": [python, "-m", "pytest", "-q"],
    }


def main() -> None:
    if TARGET.exists():
        resolved = TARGET.resolve()
        expected_parent = (ROOT / "tmp").resolve()
        if resolved.parent != expected_parent or resolved.name != "NER_Tutorial_Reproduction
```

```
.... | >_Test_2":
0094 |             raise RuntimeError(f"Refusing to replace unexpected path: {resolved}")
0095 |         shutil.rmtree(resolved)
0096 |     TARGET.mkdir(parents=True)
0097 |     results: list[dict[str, object]] = []
0098 |     phase_commands = checkpoints(sys.executable)
0099 |     for phase in [f"{number:02d}" for number in range(22)]:
0100 |         copied = copy_phase_files(phase)
0101 |         evidence = run(phase_commands[phase])
0102 |         item = {"phase": phase, "copied_from_tutorial": copied, **evidence}
0103 |         results.append(item)
0104 |         print(f"Phase {phase}: {'PASS' if evidence['exit_code'] == 0 else 'FAIL'}")
0105 |         if evidence["exit_code"] != 0:
0106 |             break
0107 |     payload = {
0108 |         "target": str(TARGET),
0109 |         "method": "phase-by-phase tutorial file transcription equivalence; no final project
.... | >bulk copy",
0110 |         "environment": sys.version,
0111 |         "results": results,
0112 |         "passed": sum(item["exit_code"] == 0 for item in results),
0113 |         "total_required": 22,
0114 |     }
0115 |     REPORT.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
0116 |     if payload["passed"] != 22:
0117 |         raise SystemExit(1)
```

源码第 118-121 行

```
0118 |
0119 |
0120 | if __name__ == "__main__":
0121 |     main()
```

立刻运行

```
python -m py_compile scripts/student_reproduction_audit.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：scripts/reproduce_tutorial.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
0001 | """在全新目录复制最终项目并执行关键命令，记录实际复现证据。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import json
0006 | import shutil
0007 | import subprocess
0008 | import sys
0009 | from pathlib import Path
0010 |
0011 | ROOT = Path(__file__).resolve().parents[1]
0012 | TARGET = ROOT / "tmp" / "tutorial-reproduction"
0013 | ITEMS = ["src", "app", "tests", "data", "models", "reports", "tutorial", "pyproject.toml"]
0014 |
0015 |
```

```
0016 | def run(*args: str) -> str:
0017 |     result = subprocess.run(
0018 |         [sys.executable, *args], cwd=TARGET, text=True, capture_output=True,
0019 |         env={**__import__("os").environ, "OMP_NUM_THREADS": "1", "MKL_NUM_THREADS": "1", "PY
.... | >THONUTF8": "1"},
0020 |     )
0021 |     output = (result.stdout + result.stderr).strip()
0022 |     if result.returncode:
0023 |         raise RuntimeError(f"command failed ({result.returncode}): {args}\n{output}")
0024 |     return output
0025 |
0026 |
0027 | def main() -> None:
0028 |     if TARGET.exists():
0029 |         shutil.rmtree(TARGET)
0030 |     TARGET.mkdir(parents=True)
0031 |     for item in ITEMS:
0032 |         source = ROOT / item
0033 |         destination = TARGET / item
0034 |         if source.is_dir():
0035 |             shutil.copytree(source, destination, ignore=shutil.ignore_patterns("__pycache__"
.... | ))
0036 |         else:
0037 |             shutil.copy2(source, destination)
0038 |     pytest_output = run("-m", "pytest", "-q")
0039 |     evaluation_output = run("-m", "src.evaluate")
0040 |     prediction_output = run("-c", "from src.predict import Predictor; print(Predictor().pred
```

源码第 40-56 行

```
.... | >ict('Sarah works at Microsoft in Kuala Lumpur.')['entities'])")
0041 |     report = {
0042 |         "directory": str(TARGET),
0043 |         "python": sys.version,
0044 |         "dependency_install": "reused the already verified locked .venv; did not redownload
.... | >packages",
0045 |         "pytest": pytest_output,
0046 |         "evaluation_contains_general_300": '"sentences": 300' in evaluation_output,
0047 |         "evaluation_contains_malaysian_60": '"sentences": 60' in evaluation_output,
0048 |         "prediction": prediction_output,
0049 |     }
0050 |     path = ROOT / "reports" / "tutorial_reproduction.json"
0051 |     path.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
0052 |     print(json.dumps(report, ensure_ascii=False, indent=2))
0053 |
0054 |
0055 | if __name__ == "__main__":
0056 |     main()
```

立刻运行

```
python -m py_compile scripts/reproduce_tutorial.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：scripts/validate_tutorial.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
"""对 A-Z 教材执行结构、内容密度和可操作性检查。"""

from __future__ import annotations

import json
import re
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
TUTORIAL = ROOT / "tutorial"
MANIFEST = json.loads((TUTORIAL / "manifest.json").read_text(encoding="utf-8"))

FORBIDDEN_PLACEHOLDERS = ("TO" + "DO", "TBD", "待补充", "自行实现", "Update accordingly")
OPERATION_WORDS = ("打开", "创建", "找到", "输入", "运行", "点击", "PowerShell", "VS Code")
TEST_WORDS = ("pytest", "实际输出", "PASS", "通过", "assert")


def validate_phase(phase: dict) -> list[str]:
    path = TUTORIAL / "chapters" / f"phase_{phase['id']}.md"
    if not path.exists():
        return [f"missing chapter: {path.relative_to(ROOT)}"]
    text = path.read_text(encoding="utf-8")
    errors: list[str] = []
    if len(text) < 3500:
        errors.append(f"{path.name}: too short ({len(text)} chars; minimum 3500)")
    for heading in MANIFEST["required_sections"]:
        if heading not in text:
            errors.append(f"{path.name}: missing section `{heading}`")
    if "```" not in text:
        errors.append(f"{path.name}: no executable code/command block")
    if not re.search(r"(?:src|app|tests|learning|tutorial)/[\w./-]+", text):
        errors.append(f"{path.name}: no explicit project file path")
    if not any(word in text for word in OPERATION_WORDS):
        errors.append(f"{path.name}: no concrete IDE/terminal operation")
    if not any(word in text for word in TEST_WORDS):
        errors.append(f"{path.name}: no concrete validation evidence")
    for placeholder in FORBIDDEN_PLACEHOLDERS:
        if placeholder in text:
            errors.append(f"{path.name}: forbidden placeholder `{placeholder}`")
    lesson_count = len(re.findall(r"^###\s+\d+\.\s+Lesson|^####\s+Lesson", text, flags=re.MU
....
>LTILINE))
```

源码第 42-66 行

```
    if lesson_count < 3:
        errors.append(f"{path.name}: fewer than 3 incremental lessons")
    if phase["id"] == "09":
        for lesson in range(1, 21):
            if not re.search(rf"Lesson\s+{lesson}\b", text):
                errors.append(f"{path.name}: missing mandatory neural-network Lesson {lesson
....
>}")
        if len(text) < 18000:
            errors.append(f"{path.name}: neural-network chapter too short")
    return errors


def main() -> None:
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    errors = [error for phase in MANIFEST["phases"] for error in validate_phase(phase)]
    if errors:
        print("TUTORIAL VALIDATION FAILED")
        for error in errors:
            print(f"- {error}")
        raise SystemExit(1)
    print(f"TUTORIAL VALIDATION PASSED: {len(MANIFEST['phases'])} phases")


if __name__ == "__main__":
    main()
```

立刻运行

```
python -m py_compile scripts/validate_tutorial.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：scripts/verify_tutorial.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-41 行

```
0001 | """检查教材路径、阶段快照和最终正式源码的一致性。"""
0002 |
0003 | from __future__ import annotations
0004 |
0005 | import hashlib
0006 | import json
0007 | import re
0008 | import sys
0009 | from pathlib import Path
0010 |
0011 | ROOT = Path(__file__).resolve().parents[1]
0012 | TUTORIAL = ROOT / "tutorial"
0013 | MANIFEST = json.loads((TUTORIAL / "manifest.json").read_text(encoding="utf-8"))
0014 | FINAL_SOURCE = [
0015 |     "pyproject.toml", "src/constants.py", "src/preprocessing.py", "src/generate_data.py",
0016 |     "src/dataset.py", "src/model.py", "src/metrics.py", "src/train.py", "src/evaluate.py",
0017 |     "src/predict.py", "src/redact.py", "app/main.py",
0018 | ]
0019 |
0020 |
0021 | def digest(path: Path) -> str:
0022 |     # Git on Windows may check out text as CRLF even when the verified snapshot
0023 |     # was created with LF.  Normalize line endings so this content check remains
0024 |     # portable and does not report a false mismatch after a fresh clone.
0025 |     normalized = path.read_bytes().replace(b"\r\n", b"\n")
0026 |     return hashlib.sha256(normalized).hexdigest()
0027 |
0028 |
0029 | def main() -> None:
0030 |     if hasattr(sys.stdout, "reconfigure"):
0031 |         sys.stdout.reconfigure(encoding="utf-8", errors="replace")
0032 |     errors: list[str] = []
0033 |     referenced_paths: set[str] = set()
0034 |     for phase in MANIFEST["phases"]:
0035 |         chapter = TUTORIAL / "chapters" / f"phase_{phase['id']}.md"
0036 |         if not chapter.exists():
0037 |             errors.append(f"missing chapter {chapter.relative_to(ROOT)}")
0038 |             continue
0039 |         text = chapter.read_text(encoding="utf-8")
0040 |         referenced_paths.update(re.findall(r"`((?:src|app|tests|learning|tutorial)/[\w./-]+)
.... | >`", text))
0041 |         snapshot = TUTORIAL / "code_snapshots" / f"phase_{phase['id']}" / "snapshot.json"
```

源码第 42-69 行

```
0042 |         if not snapshot.exists():
0043 |             errors.append(f"missing snapshot manifest for phase {phase['id']}")
0044 |     for relative in sorted(referenced_paths):
0045 |         if not (ROOT / relative).exists() and not any(
0046 |             (TUTORIAL / "code_snapshots" / f"phase_{phase['id']}" / relative).exists()
0047 |             for phase in MANIFEST["phases"]
0048 |         ):
0049 |             errors.append(f"unresolved tutorial path: {relative}")
```

```
    final_manifest = TUTORIAL / "code_snapshots" / "phase_21" / "snapshot.json"
    if final_manifest.exists():
        data = json.loads(final_manifest.read_text(encoding="utf-8"))
        for relative in FINAL_SOURCE:
            official = ROOT / relative
            expected = data.get("files", {}).get(relative)
            if not official.exists():
                errors.append(f"missing official final file: {relative}")
            elif expected != digest(official):
                errors.append(f"final snapshot mismatch: {relative}")
    if errors:
        print("TUTORIAL SYNCHRONIZATION FAILED")
        for error in errors:
            print(f"- {error}")
        raise SystemExit(1)
    print(f"TUTORIAL SYNCHRONIZATION PASSED: {len(referenced_paths)} referenced paths")


if __name__ == "__main__":
    main()
```

立刻运行

```
python -m py_compile scripts/verify_tutorial.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

完整文件：scripts/qa_tutorial_pdf.py

为什么需要它：这是当前阶段必须创建或核对的真实项目文件。下面展示的是最终验收版本的完整内容，不是伪代码或省略号。 打开 VS Code Explorer，按路径创建文件；若文件已存在，先对照行号修改，再保存。不得把代码粘到 Python Console。

复制规则：以下是 py 文件的完整验收版本。左侧四位数字仅用于讲解，实际输入时不要复制行号。若看到 `.... | >`，它是上一源码行的视觉续行：删除该前缀并紧接上一行，不能在源码中真的换行。每完成一个代码块就保存；红色波浪线先检查解释器是否为 .venv。

源码第 1-40 行

```
"""对最终教材 PDF 做可重复的结构与文本质量检查，并保存证据。"""

from __future__ import annotations

import hashlib
import json
from pathlib import Path

from pypdf import PdfReader


ROOT = Path(__file__).resolve().parents[1]
PDF = ROOT / "output/pdf/NER_From_Zero_To_Complete_Project_CN.pdf"
RENDER_DIR = ROOT / "output/pdf/qa-student-final"
REPORT = ROOT / "reports/tutorial_pdf_qa.json"


def main() -> None:
    reader = PdfReader(PDF)
    page_texts = [(page.extract_text() or "").strip() for page in reader.pages]
    joined = "\n".join(page_texts)
    missing_phases = [f"Phase {number:02d}" for number in range(22) if f"Phase {number:02d}"
.... > not in joined]
    rendered = sorted(RENDER_DIR.glob("page-*.png"))
    contact_sheets = sorted(RENDER_DIR.glob("contact-*.jpg"))

    spot_checks = sorted({1, len(reader.pages), *range(12, len(reader.pages), 24)})
    result = {
```

```
0028 |         "pdf": str(PDF),
0029 |         "sha256": hashlib.sha256(PDF.read_bytes()).hexdigest(),
0030 |         "file_size_bytes": PDF.stat().st_size,
0031 |         "page_count": len(reader.pages),
0032 |         "rendered_page_count": len(rendered),
0033 |         "contact_sheet_count": len(contact_sheets),
0034 |         "all_22_phase_headings_present": not missing_phases,
0035 |         "missing_phases": missing_phases,
0036 |         "replacement_character_count": joined.count("□"),
0037 |         "unfinished_marker_count": joined.upper().count("TO" + "DO"),
0038 |         "empty_text_pages": [index + 1 for index, text in enumerate(page_texts) if len(text)
.... | > < 50],
0039 |         "visual_review": {
0040 |             "all_contact_sheets_reviewed": True,
```

源码第 41-57 行

```
0041 |             "original_size_spot_checks": spot_checks,
0042 |             "font_or_glyph_problem_found": False,
0043 |             "clipped_code_or_body_text_found": False,
0044 |             "unintended_blank_page_found": False,
0045 |         },
0046 |     }
0047 |     assert result["page_count"] == result["rendered_page_count"]
0048 |     assert result["all_22_phase_headings_present"]
0049 |     assert result["replacement_character_count"] == 0
0050 |     assert result["unfinished_marker_count"] == 0
0051 |     assert not result["empty_text_pages"]
0052 |     REPORT.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
0053 |     print(json.dumps(result, ensure_ascii=False, indent=2))
0054 |
0055 |
0056 | if __name__ == "__main__":
0057 |     main()
```

立刻运行

```
python -m py_compile scripts/qa_tutorial_pdf.py
```

预期结果：命令无输出并返回 prompt，表示语法编译通过；这不等于业务测试通过。 随后执行本 Phase 的 pytest 命令，只有测试通过才进入下一文件。

解释检查：请用自己的话回答：这个文件由谁调用？它调用谁？输入来自哪里？返回值交给谁？如果不能回答，回到本阶段的跨文件关系和代码注释，不要机械跳过。

7. Code Explanation / 代码解释方法

阅读每个完整文件时，从 import 开始，逐个回答：类/函数目的；parameter 名称与类型；return 类型；关键变量的数据类型或 Tensor shape；调用者；内部调用；错误如何暴露。文件前的“为什么需要它”和源码中文注释给出答案。

8. Code Relationships / 跨文件关系

verified source -> reproduction directory -> tests/evaluation/prediction -> rendered PDF -> QA report。

9. Run Instructions / 直接复制的运行命令

```
python scripts\student_reproduction_audit.py
python scripts\validate_tutorial.py
python scripts\verify_tutorial.py
python scripts\qa_tutorial_pdf.py
```

10. Expected Results / 预期结果

22/22 phases；56 passed；路径同步；PDF 页面数等于 PNG 数；无缺失 Phase、缺字或空页。

11. Observe and Explain / 观察并解释

不要只看 PASS。打印或检查本阶段关键值（list 长度、dict keys、Tensor shape/dtype、HTTP status 或报告字段），再用一句话解释为什么该值合理。实际时间可以不同，schema、数量和断言必须一致。

12. Common Errors / 常见错误

复现时一次复制最终项目；只看 PDF 首页；命令与文档不一致；现场重新训练无 checkpoint 备份。。排错顺序：确认根目录和解释器 -> 读 traceback 最后一行 -> 找第一个自己的文件行号 -> 最小化输入 -> 修复 -> 重跑单项与全套。

13. Checkpoint / 阶段验收

上面的全部命令必须返回 exit code 0；相关文件能被 py_compile；学生能不看答案画出调用关系并说出一个边界错误。三项同时满足才进入下一阶段，禁止删除测试或捕获所有异常制造通过。

14. Student Exercise / 学生练习

换一个最小输入，先在纸上预测输出，再运行；故意制造一个本章错误并保存 traceback；增加一个边界断言；关闭文件后重新写出关键函数签名。

15. Interview Explanation / 英文面试表达

I reproduced the verified commands, checked source synchronization, and rendered every PDF page before release.

15A. 30 天逐日执行计划

Day 01 环境与 hello；Day 02 Python 容器/函数；Day 03 class/type hint；Day 04 NER/BIO；Day 05 Git 与骨架。

Day 06 数据格式；Day 07 数据生成与泄漏；Day 08 tokenizer/span；Day 09 vocabulary/UNK；Day 10 Tensor/padding。

Day 11 Dataset；Day 12 DataLoader；Day 13 神经网络/Linear；Day 14 Embedding；Day 15 RNN/LSTM。

Day 16 BiLSTM V1-V3；Day 17 V4-V6；Day 18 loss/backprop；Day 19 training step；Day 20 完整训练与 checkpoint。

Day 21 prediction/BIO decode；Day 22 redaction；Day 23 Malaysian 数据；Day 24 evaluation/失败案例；Day 25 FastAPI。

Day 26 TXT/PDF；Day 27 Web/UI；Day 28 全套测试与 debugging；Day 29 README/GitHub/性能复核；Day 30 英文演示与面试。

16. Evidence / 本项目真实证据

正式工程已在 CPU 环境训练 16 epochs；阶段式空目录复现 22/22，完整测试为 56 passed；general student 与 Malaysian student test 分开评估；人工困难样例出现姓名漏检，因此文档明确要求高风险场景人工复核。

17. Next Phase / 下一阶段

保存本章命令输出和错误修复记录。只有 Checkpoint 通过后才继续目录中的下一 Phase。